Digital Signal Processing

Getting Started: Wokwi and JSLinux

Exercise Sheet 0

Script: Preface, Chapter 1.1

with solutions

Learning goals

- Run, modify and share an Arduino project in the Wokwi simulator.
- *Bonus (optional):* compile and run C programs in JSLinux, plot results in the terminal, and transfer measured data from Wokwi (Serial Monitor) to JSLinux.
- Get to know the course library `code/common/engine_signals.h`, the *virtual engine* used on all following sheets.

The virtual engine

Throughout the exercises we cannot measure on a real engine. Instead, a *virtual engine* generates the sensor signals introduced in Chapter 1 of the script. Exactly the same C model is used in three places:

Where	How
JSLinux	`#include "engine_signals.h"` in your C program
Wokwi (sketch)	add `engine_signals.h` as an extra file to the project
Wokwi (custom chip)	part `chip-engine-sim`: outputs CRANK, CAM, KNOCK, ION, and the ground-truth signals TDC and KNK

The conventions of the model are fixed for all sheets:

- 4-cylinder 4-stroke engine, crank angle $\theta \in [0°, 720°)$, firing order 1–3–4–2, combustion TDC at 120°, 300°, 480°, 660°.
- 60-2 trigger wheel: tooth $k = 0, \ldots, 57$ is HIGH for $\theta \bmod 360° \in [6k°, 6k° + 3°)$; teeth 58 and 59 are missing.
- Camshaft signal HIGH for $0° \le \theta < 180°$.
- Knock: damped oscillation with modes at 6.5 kHz and 10.5 kHz, decay time 0.8 ms, onset 5–25° after TDC.
- Disturbances: broadband noise, valve impacts (8 kHz, 90° after TDC), low-frequency combustion vibration, a 1 % speed ripple at twice crankshaft frequency.

Alternative: Wokwi in VS Code

All Wokwi exercises run in the browser; nothing has to be installed. If you prefer working locally, every Wokwi project of the course is also available as a project for the *Wokwi for VS Code* extension in `code/wokwi_vscode/` (same sketches and circuits). There you build the firmware with PlatformIO and Wokwi simulates it; the custom chips come precompiled. You need VS Code with the extensions *PlatformIO IDE* and *Wokwi Simulator* and a free Wokwi license. Setup and workflow are described step by step in `code/wokwi_vscode/README.md`. In the exercises, "open the project in Wokwi" then means: open the folder of the same name in `code/wokwi_vscode/` and edit `src/sketch.ino`. The logic analyzer recording is written to `wokwi.vcd` in the project folder when you stop the simulation.

Exercise 0.1 First steps in Wokwi

WOKWI

Open `https://wokwi.com`, create a new *Arduino Uno* project and replace `sketch.ino` and `diagram.json` by the files in `code/wokwi/sheet00_hello/`.

a) Start the simulation, turn the potentiometer and watch the Serial Monitor. Which ADC value corresponds to 2.5 V? What is the voltage resolution (1 LSB) of the 10-bit ADC?

b) The sketch toggles pin 13 at every sample. Stop the simulation after about one second, download the logic analyzer recording (`.vcd`) and determine the sampling period. Does it match `TS_US`?

c) Change the sampling rate to 1 kHz. Explain why the printed `t_us` values still show regular increments although the `Serial.print` calls take time.

d) Wokwi runs in *simulated time*. Compare the simulated time in the status bar with the time on your wall clock. Why is this important when we later measure execution times?

Solution

a) $V = \text{adc} \cdot 5/1023$, so $2.5\,\text{V} \approx 511/512$. 1 LSB $= 5/1023\,\text{V} \approx 4.9\,\text{mV}$.
b) Pin 13 toggles at every sample, i.e. the period of the rectangular signal on D0 is $2T_s = 20\,\text{ms}$; consecutive edges are $T_s = 10\,\text{ms}$ apart.
c) The loop waits for an absolute deadline `t_next` that is incremented by `TS_US`. The time spent in printing only shortens the waiting time, it does not shift the next deadline (no drift), as long as one loop iteration takes less than T_s. At 1 kHz and 115200 baud a line of about 20 characters takes $\approx 1.7\,\text{ms}$ – longer than T_s! Then the deadlines are missed and the samples come out late; the `t_us` increments become irregular. Remedy: fewer characters, higher baud rate, or buffer samples and print blocks.
d) The simulator may run faster or slower than real time. `micros()` and the logic analyzer use simulated time, so timing *inside* the simulation is consistent. Wokwi is not cycle-accurate for all peripherals, therefore measured execution times are only indicative; we count operations instead.

Exercise 0.2 The engine simulator chip

WOKWI

Every Wokwi project of the following sheets contains the custom chip `engine-sim`. Add the files `engine-sim.chip.c` and `engine-sim.chip.json` from `code/wokwi/chips/` to the Uno project of Exercise 0.1 and add the part `{ "type": "chip-engine-sim", "id": "eng", "attrs": { "vmax": "5", "of` to `diagram.json`. *VS Code:* work in `code/wokwi_vscode/sheet00_hello/`; instead of copying the chip files, add the precompiled chip to `wokwi.toml` (as in `sheet01_crank/wokwi.toml`):

```
[[chip]]
name = 'engine-sim'
binary = '../chips/engine-sim.chip.wasm'
```

a) Connect `eng:CRANK` to logic analyzer D1, `eng:CAM` to D2 and `eng:TDC` to D3 (and `eng:GND` to ground). Record 100 ms at 3000 rpm and view the `.vcd` file (e.g. in PulseView or GTKWave). Identify the missing-tooth gap and verify the tooth period.

b) How many crank teeth lie between the gap and the first TDC pulse? Compare with the conventions above.

c) Change the engine speed with the slider of the chip and repeat the tooth-period measurement at 6000 rpm.

Solution

a) At 3000 rpm one revolution takes 20 ms; the tooth period (rising to rising edge) is $20\,\text{ms}/60 = 333\,\mu\text{s}$, across the gap $3 \cdot 333\,\mu\text{s} = 1\,\text{ms}$.

b) The reference edge (first tooth after the gap) is at $\theta = 0°$, TDC of cylinder 1 at 120°, i.e. 20 teeth after the reference edge.
c) At 6000 rpm: 167 µs per tooth, 500 µs across the gap.

Deliverables

Screenshot of the Wokwi logic analyzer view (or VCD plot) with the gap marked. Bonus (optional): terminal output of `hello_signals`; the CSV-plot program of 0.B1 d).

Bonus: JSLinux (optional)

The lab exercises above use only Wokwi. Interested students can in addition process the engine signals in plain C on a Linux system that runs in the browser. The JSLinux exercises of the following sheets build on this one.

Exercise 0.B1 First steps in JSLinux

JSLINUX

Open `https://bellard.org/jslinux` and start the *x86 Alpine Linux* (console) virtual machine.

a) Check the toolchain: `gcc -version` (or `tcc -v`). Upload `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and `code/bonus/jslinux/sheet00/hello_signals.c` with the upload button below the terminal.

```
gcc -O2 -o hello_signals hello_signals.c -lm
./hello_signals
```

b) Explain the plot of the knock signal: where does the burst start, how long does it last, and what is visible before the burst?

c) Modify the program such that it plots one full engine cycle (720°) at 3000 rpm with `knock_prob = 0`. How many samples are needed at $f_s = 50$ kHz? Identify the valve impacts and the combustion vibration in the plot.

d) Copy 2–3 seconds of the Wokwi Serial Monitor output from Exercise 0.1 into a file `pot.csv` in JSLinux (`cat > pot.csv`, paste, `Ctrl-D`). Write a short program that reads it with `csv_read()` and plots the voltage column.

Solution

b) The program starts the record at $\theta = 125°$, i.e. 5° after TDC of cylinder 1. The knock onset lies between 5° and 25° ATDC; at 3000 rpm (18 000 °/s) 1° corresponds to 55.6 µs, so the burst appears within the first 1.1 ms. It decays with $\tau = 0.8$ ms and is practically gone after $3\tau \approx 2.4$ ms. Before the burst only noise and the slow combustion vibration (a half-sine over 60°) are visible.
c) One cycle lasts $2 \cdot 60/3000\,\text{s} = 40$ ms, i.e. 2000 samples. Valve impacts appear 90° after each TDC (8 kHz bursts at $\theta = 210°, 390°, 570°, 30°$), the combustion vibration as broad positive humps directly after each TDC.
d) Example:

```c
csv_t d;
if (csv_read("pot.csv", &d) == 0) {
    double *v = csv_col(&d, 3);          /* column "volt" */
    ap_plot(v, d.rows, "potentiometer voltage");
    free(v); csv_free(&d);
}
```

Digital Signal Processing Exercise Sheet 1

The ECU Sees Only Signals

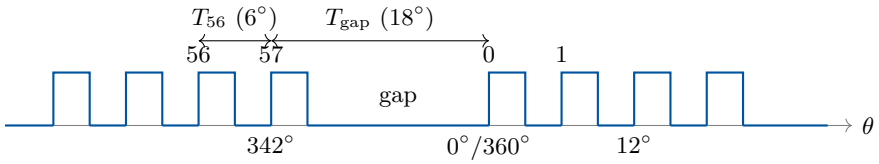
Script: Chapter 1 (The ECU Sees Only Signals) **with solutions**

Learning goals

- Classify the engine sensor signals (continuous/discrete time, analog/binary amplitude, deterministic/stochastic, periodic/aperiodic) and know their characteristic time scales.
- *Bonus (optional):* describe sensor signals quantitatively: mean, RMS, extreme values, autocorrelation as a numerical test for periodicity.
- Extract a first piece of information from a raw signal: crank angle and engine speed from the 60-2 tooth timing, including the effect of the timer resolution.
- Synchronise crank and cam signal to determine the 720° phase of the engine cycle (cylinder identification).

Background: the 60-2 trigger wheel

The crank wheel of the virtual engine has 60 tooth positions (6° each), two of which are missing. The Hall sensor delivers a binary signal; the rising edge of tooth $k = 0, \ldots, 57$ occurs at $\theta = 6k°$ (modulo 360°), the reference mark is the first rising edge after the gap.

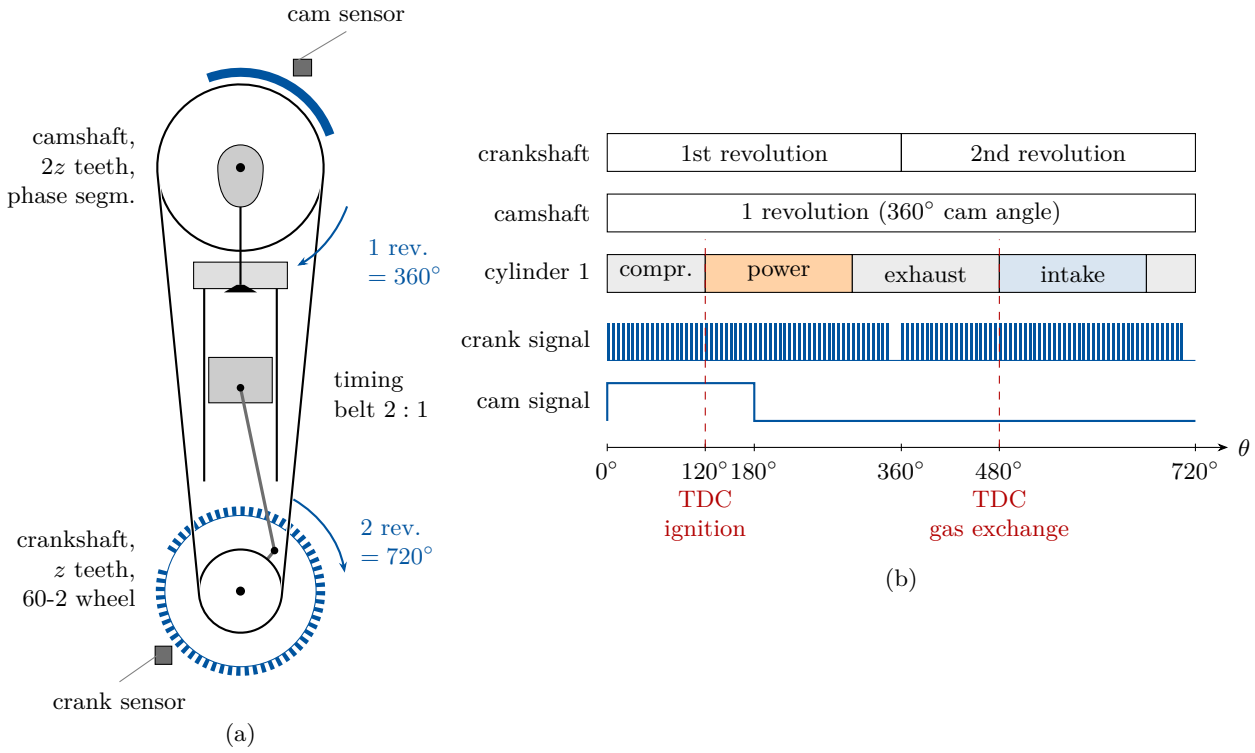


The tooth period T_k is measured from rising edge to rising edge. Because one tooth corresponds to $6° = 1/60$ revolution, the engine speed in rpm is

$$n = \frac{60\,\text{s/min}}{60\,T_k} = \frac{1}{T_k/\text{s}}\ \text{rpm}, \qquad n = \frac{3}{T_{\text{gap}}/\text{s}}\ \text{rpm (gap)}, \qquad n = \frac{60}{T_{\text{rev}}/\text{s}}\ \text{rpm (revolution)}. \tag{1}$$

Background: crankshaft, camshaft and the 720° cycle

A four-stroke engine needs *two* crankshaft revolutions for one working cycle (intake, compression, power, exhaust), but every valve opens only once per cycle. The camshaft is therefore driven by a timing belt (or chain) with a ratio of 2 : 1: its sprocket has twice as many teeth as the crankshaft sprocket, so it turns once while the crankshaft turns twice. The crank angle θ is counted over the full cycle, $0° \le \theta < 720°$, which is exactly one camshaft revolution (360° cam angle). The 60-2 wheel on the crankshaft repeats after 360° crank angle, the single segment of the cam phase wheel only after 720°.



(a) Schematic front view of one cylinder with crank drive and overhead camshaft; the timing belt turns the camshaft once per two crankshaft revolutions. (b) One working cycle of cylinder 1 over the crank angle: the crank wheel pattern and the piston position repeat after 360°, the strokes and the cam signal only after 720°.

Exercise 1.1 Signal classification and time scales

PAPER

a) Classify the five engine signals of Table 1.1 of the script (crank, cam, knock, ion current, lambda) with respect to (i) time: continuous/discrete, (ii) amplitude: analog/binary (digital), (iii) deterministic/stochastic, (iv) periodic/quasi-periodic/aperiodic. Collect the result in a table and justify every entry in one line. Note that the answer may differ for the physical sensor output and for the representation inside the ECU.

b) Discuss the borderline cases: (i) Is the crank signal periodic during an acceleration? Is it periodic in another independent variable? (ii) The knock signal is often called *quasi-periodic*, although a single knock event is a transient. Why? Introduce the notion of a signal whose *statistics* are periodic in the crank angle.

c) Fill in the following table for an engine speed of 3000 rpm: crank revolution period, tooth period and tooth frequency, duration of the gap (rising to rising edge), cam period, cam HIGH time, interval between two combustions (firing frequency), duration of 1° crank angle, time window of a knock onset (5°–25° after TDC), number of 6.5 kHz knock oscillations within one decay time $\tau = 0.8\,\text{ms}$.

d) Crank and cam: the crankshaft turns 720° per engine cycle, the camshaft 360°. Why is the crank signal alone not sufficient to decide whether the piston of cylinder 1 is in its compression or in its exhaust stroke? After switching on the ignition with the engine cranking at 200 rpm, what is the worst-case time until the ECU knows (i) the crank angle modulo 360°, (ii) the full 720° phase, if the cam level is evaluated at $\theta = 60°$ after the reference mark?

Solution

a) Classification. Go through the four criteria one at a time and ask each time: *what does the sensor deliver*, and *what does the ECU store*?

Step 1: *Time.* Every sensor delivers a voltage at every instant, so all five physical signals are continuous-time (CT). Inside the ECU they become discrete-time (DT) signals, in two different ways: analog signals are *sampled* at fixed times $t = nT_s$ by the ADC; binary signals are *time-stamped* at their edges by a capture timer, so the index is the tooth number k, not a fixed sampling time.

Step 2: *Amplitude.* A Hall sensor switches between two levels. This makes crank and cam binary. Knock, ion current and lambda are analog voltages, and the ADC quantises them (e.g. 10 or 12 bits). The lambda sensor is analog, but its characteristic is so steep that in practice it shows only two values (0.1 V lean, 0.9 V rich).

Step 3: *Predictability.* If the engine speed is known, the tooth wheel and the cam pattern follow exactly from the geometry, so they are deterministic. Knock and noise have random onset, amplitude and phase, so they are stochastic. Combustion varies from cycle to cycle, so the ion current is only partly deterministic.

Step 4: *Periodicity.* Ask after which shift the signal repeats, in time t or in crank angle θ: crank 360°, cam 720°, combustion-related signals $720°/4 = 180°$ (one event per TDC).

This gives the table (physical signal → inside the ECU):

Signal	time	amplitude	predictability	periodicity
crank	CT → event times t_k	binary (Hall)	deterministic up to small speed fluctuations	periodic in θ (360°); in t only at constant speed
cam	CT → level sampled at tooth events	binary	deterministic	periodic in θ (720°)
knock	CT → DT after ADC	analog → quantised	stochastic (noise, random knock onset, amplitude, phase)	quasi-periodic: deterministic parts repeat every 180°, knock is random
ion current	CT → DT	analog	partly deterministic (shape of the peaks), cycle-to-cycle variation	aperiodic per event; event pattern repeats per 180°
lambda	CT → DT (slow, 10–100 Hz)	analog, but practically two-valued (0.1/0.9 V)	slowly varying, disturbed	aperiodic; in the closed loop a limit cycle (quasi-periodic, ≈1 Hz)

Crank and cam are physically continuous-time voltages with two levels. The ECU, however, does not sample them. A capture timer time-stamps their edges, so the information is in the *sequence of edge times* t_k, which is a discrete-time signal indexed by the tooth number. All analog signals become discrete-time and quantised (= digital) after the ADC.

b) Borderline cases. (i) A signal is periodic in t if $x(t) = x(t+T)$ for all t and one fixed $T > 0$. During an acceleration $\omega(t)$ increases, so the tooth period $T_k = 6°/\omega$ gets shorter from tooth to tooth and no single T works: the crank signal is *not* periodic in time. Write it as a function of the crank angle instead, $x(t) = x_c(\theta(t))$ with $\theta(t) = \int_0^t \omega(t')\,\mathrm{d}t'$. The wheel geometry fixes x_c, so $x_c(\theta) = x_c(\theta + 360°)$ holds *exactly*, whatever the speed. This is why many engine algorithms work in the angle domain (Sheet 4).

(ii) Look at a single burst: it is a damped oscillation $A\,\mathrm{e}^{-(t-t_0)/\tau}\sin(2\pi f_1(t-t_0)+\varphi)$ with random t_0, A and φ, and in many cycles there is no burst at all. That is a transient, not a periodic signal. Now look at *averages over many cycles* at a fixed crank angle: the knock *probability*, the

valve impacts, the combustion vibration and the noise level are the same after every TDC. So the mean $\mu(\theta) = \mathrm{E}\{x(\theta)\}$ and the autocorrelation $R(\theta, \theta+\Delta) = \mathrm{E}\{x(\theta)\,x(\theta+\Delta)\}$ satisfy

$$\mu(\theta + 180^\circ) = \mu(\theta), \qquad R(\theta+180^\circ, \theta+180^\circ+\Delta) = R(\theta, \theta+\Delta).$$

A signal whose statistics are periodic like this is called *cyclostationary*. This is exactly what "quasi-periodic" means for the knock signal.

c) Time scales at 3000 rpm**.** Everything follows from two numbers:

$$T_{\mathrm{rev}} = \frac{60}{n} = \frac{60}{3000}\,\mathrm{s} = 20\,\mathrm{ms}, \qquad \omega = \frac{360^\circ}{T_{\mathrm{rev}}} = 18\,000\,^\circ/\mathrm{s}.$$

Each other entry is then an angle divided by ω:

- tooth: $T_z = 6^\circ/\omega = 20\,\mathrm{ms}/60 = 333.3\,\mu\mathrm{s}$, i.e. $f_z = 3\,\mathrm{kHz}$;
- gap: the rising edge of tooth 57 is at 342°, that of tooth 0 at 360°, so $18^\circ/\omega = 3\,T_z = 1\,\mathrm{ms}$;
- cam period = engine cycle: $720^\circ/\omega = 2\,T_{\mathrm{rev}} = 40\,\mathrm{ms}$; cam HIGH for 180°: 10 ms;
- 4 combustions per 720°: one every 180°, i.e. 10 ms, firing frequency 100 Hz = 2× the rotation frequency (2nd order);
- 1°: $1/\omega = 55.6\,\mu\mathrm{s}$, so the knock window 5° … 25° ATDC is 0.278 ms … 1.389 ms after TDC;
- oscillations within τ: $f_1\tau = 6500\,\mathrm{Hz} \cdot 0.8\,\mathrm{ms} = 5.2$.

quantity	formula	value
crank revolution	$60/n$	20 ms (50 Hz)
tooth period / frequency	$T_{\mathrm{rev}}/60$	333 µs / 3 kHz
gap (rising to rising)	$3\,T_z$	1 ms
engine cycle = cam period	$2\,T_{\mathrm{rev}}$	40 ms (25 Hz)
cam HIGH time	180°	10 ms
combustion interval	$720^\circ/4 = 180^\circ$	10 ms (firing frequency 100 Hz = 2nd order)
1° crank angle	$1/\omega$	55.6 µs
knock onset window	5° … 25° ATDC	0.28 ms … 1.39 ms after TDC
knock oscillations in τ	$f_1\tau$	$6.5 \cdot 0.8 = 5.2$

These numbers are easy to check with a few lines of C. Change `n` to get the table for another speed:

```c
double n = 3000.0;                  /* engine speed, rpm      */
double T_rev = 60.0 / n;            /* s per revolution       */
double w = 360.0 / T_rev;           /* crank speed, deg/s     */
printf("tooth       %7.1f us  (%.0f Hz)\n", 1e6 * 6.0 / w, w / 6.0);
printf("gap         %7.3f ms\n", 1e3 * 18.0 / w);
printf("combustion  %7.3f ms  (%.0f Hz)\n", 1e3 * 180.0 / w, w / 180.0);
printf("1 deg       %7.2f us\n", 1e6 / w);
printf("knock onset %7.3f .. %.3f ms ATDC\n", 1e3 * 5.0 / w, 1e3 * 25.0 / w);
```

```
tooth         333.3 us  (3000 Hz)
gap           1.000 ms
combustion   10.000 ms  (100 Hz)
1 deg         55.56 us
knock onset   0.278 .. 1.389 ms ATDC
```

The signals thus span four decades of time scales: knock (100 µs), teeth (300 µs), combustions (10 ms), lambda (1 s).

d) Crank and cam. The crank wheel looks identical in both revolutions of a cycle: at $\theta = 120^\circ$ (compression TDC of cylinder 1, ignition) and at $\theta = 480^\circ$ (gas-exchange TDC of cylinder 1,

valve overlap) the crank signal is the same, and so is the piston position. Only the camshaft, which turns once per cycle, tells the two revolutions apart.
At 200 rpm: $T_{\text{rev}} = 60/200\,\text{s} = 300\,\text{ms}$, so $1° \mathrel{\widehat{=}} 0.833\,\text{ms}$.
(i) The ratio test at the reference edge (360°) needs *three* consecutive rising edges: tooth 56 (336°) and tooth 57 (342°) give T_{prev}, and tooth 57 and the reference edge give T_{gap}. The worst case is to switch on just *after* the edge at 336°. The current gap then cannot be tested (no T_{prev}), and the ECU has to wait for the next reference edge at 720°:

$$t_{360} = (720° - 336°) \cdot 0.833\,\text{ms}/° = 384° \mathrel{\widehat{=}} 320\,\text{ms}.$$

(Best case: switch on just before 336°, sync after $24° = 20\,\text{ms}$.)
(ii) After the reference edge the ECU still has to reach the cam evaluation at 60°:

$$t_{720} = 384° + 60° = 444° \mathrel{\widehat{=}} 370\,\text{ms}.$$

Until then the ECU can use only "wasted spark" (fire cylinders 1 and 4 together at every crank TDC), and it must not inject sequentially.

Exercise 1.2 60-2 crank wheel and engine speed

WOKWI

Open the Wokwi project `code/wokwi/sheet01_crank/` (Arduino Uno, engine simulator chip, logic analyzer, two LEDs, push button). `eng:CRANK` is connected to pin 2 (external interrupt INT0), `eng:CAM` to pin 4. The ISR `crank_isr()` is called at every rising crank edge and time-stamps it. With `TIMEBASE_TIMER1 = 1` the time stamp is the 16-bit Timer1 counter (prescaler 8, 0.5 µs per tick), with 0 it is `micros()` (resolution 4 µs on the Uno).

a) *Preparation (paper).* Derive (1). A timer of resolution Δ quantises the tooth period. Show that the resulting quantisation step of the per-tooth speed is $\Delta n \approx n^2 \Delta$ (with n in rpm and Δ in s) and evaluate it for $\Delta = 4\,\mu\text{s}$ and $0.5\,\mu\text{s}$ at 600, 3000 and 7000 rpm. Compare with the amplitude of the 1 % speed ripple of the model. The gap is detected by the ratio test $T_k > 2\,T_{k-1}$: estimate the angular acceleration (in rpm/s) at which the test would fail (missed gap, or false gap), and compare with realistic engine accelerations ($\lesssim 10\,000$ rpm/s).

b) Complete the ISR: gap detection, tooth counter, per-revolution statistics, TDC LED (pin 12) on for 30° starting at each TDC edge. Which tooth numbers belong to $\theta = 120°$ and 300°? Check with the logic analyzer (D0 = crank, D2 = TDC output of the chip, D3 = your LED) and report the column `teeth` and `sync_err`.

c) Complete the speed computation in `loop()` (per revolution, last tooth, min/max per tooth). At 3000 rpm, compare `rpm_rev` with the spread `rpm_min`…`rpm_max` for both time bases. Press the button to dump 120 tooth periods, copy only this block and plot the per-tooth speed (column `rpm`), e.g. in a spreadsheet or with your CSV plot program of Bonus Exercise 0.B1 d) in JSLinux. Which part of the "jitter" is information, which part is measurement error?

d) Move the speed slider (600…7000 rpm) and repeat c) at 7000 rpm. Why is printing one CSV line per *tooth* not an option (estimate the UART load)?

Solution

a) Speed formulas. Step 1: one tooth is $6° = \frac{1}{60}$ rev. If it takes T_k seconds, the speed is

$$n = \frac{1/60\ \text{rev}}{T_k} = \frac{1}{60\,T_k}\,\frac{\text{rev}}{\text{s}} = \frac{60}{60\,T_k}\,\frac{\text{rev}}{\text{min}} = \frac{1}{T_k/\text{s}}\ \text{rpm}.$$

The gap covers $18° = \frac{3}{60}$ rev and a revolution covers 1 rev, so the same steps give $3/T_{\text{gap}}$ and $60/T_{\text{rev}}$.
Step 2, quantisation: the timer can only measure T_k as an integer multiple of the tick Δ. A

period error of one tick causes a speed error

$$\Delta n \approx \left|\frac{\mathrm{d}n}{\mathrm{d}T}\right| \Delta = \frac{1}{T^2}\,\Delta = n^2\Delta \qquad (n = 1/T \text{ in rpm}, T, \Delta \text{ in s}).$$

Example: at 3000 rpm and $\Delta = 4\,\mu\text{s}$, $\Delta n = 3000^2 \cdot 4 \cdot 10^{-6} = 36$ rpm. Equivalently, the relative error is one tick out of $T_k/\Delta = 333/4 \approx 83$ ticks, i.e. 1.2 %.
Step 3, compare with the ripple: the model speed is $\omega = \omega_0(1 + \varepsilon \sin 2\theta)$ with $\varepsilon = 0.01$, i.e. amplitude $\pm 0.01\,n$:

n / rpm	T_k	Δn ($\Delta = 4\,\mu$s)	Δn ($\Delta = 0.5\,\mu$s)	ripple (±1 %)
600	1667 µs	1.4 rpm	0.2 rpm	±6 rpm
3000	333 µs	36 rpm	4.5 rpm	±30 rpm
7000	143 µs	196 rpm	24.5 rpm	±70 rpm

The relative resolution $\Delta n/n = \Delta/T_k = n\Delta$ gets worse linearly with speed. The quantisation step equals the ripple amplitude when $n^2\Delta = 0.01\,n$, i.e. at $n = 0.01/\Delta$: 2500 rpm for `micros()`, 20 000 rpm for Timer1. Above 2500 rpm the ripple is therefore buried in the `micros()` quantisation. With the 0.5 µs timer it is resolved over the whole speed range. Real ECUs use capture timers with ≤ 100 ns resolution.
Step 4, robustness of the gap test. Let ω_1 be the speed during the tooth before the gap and ω_2 the mean speed across the gap. Then

$$\frac{T_{\text{gap}}}{T_{\text{prev}}} = \frac{18^\circ/\omega_2}{6^\circ/\omega_1} = 3\,\frac{\omega_1}{\omega_2}.$$

Missed gap: the ratio drops to 2 or less if $\omega_2 \geq 1.5\,\omega_1$. The centres of the two intervals are $3^\circ + 9^\circ = 12^\circ$ apart, i.e. $\Delta t \approx 12^\circ/\omega_1$. The speed must therefore rise by $0.5\,\omega_1$ within Δt:

$$\dot\omega \approx \frac{0.5\,\omega_1}{12^\circ/\omega_1} = \frac{\omega_1^2}{24^\circ}.$$

Convert to rpm with $\omega = 6n$ (ω in °/s, n in rpm): $6\dot n = 36n^2/24$, so $\dot n \approx n^2/4$ (rpm/s). At 600 rpm this is $600^2/4 = 90\,000$ rpm/s.
False gap: a normal tooth gives a ratio $T_k/T_{k-1} > 2$ only if the speed halves within one tooth ($\Delta t \approx 6^\circ/\omega$): $\dot\omega \approx -0.5\,\omega^2/6^\circ$, i.e. $\dot n \approx -n^2/2$, which is $-180\,000$ rpm/s at 600 rpm.
Both limits are an order of magnitude beyond real engines ($\lesssim 10\,000$ rpm/s). The offline model confirms the large margin: at 3000 and 7000 rpm the measured gap ratio lies in $2.97 \ldots 3.09$ and the ratio of normal teeth stays below 1.03. So threshold 2 is robust in normal operation (the geometric mean of 1 and 3 would be 1.73). Only the start is critical: when cranking at 200 rpm, the compression strokes cause ±30 % speed fluctuations, and the limit is $\dot n \approx 200^2/4 = 10\,000$ rpm/s. That is why ECUs add plausibility checks there (tooth count 58 between gaps, cam edge position).
b) ISR. The tooth numbers follow from $\theta = 6^\circ k$: $k = 120/6 = 20$ and $k = 300/6 = 50$. The LED stays on for $30^\circ/6^\circ = 5$ teeth. The reference solution (`code/solutions/wokwi/sheet01_crank/sketch.ino`, line numbers as in the file) works in three steps.
Step 1 – time stamp and tooth period. Unsigned subtraction gives the right period even when the counter wraps around. The 16-bit Timer1 wraps every $65536 \cdot 0.5\,\mu\text{s} = 32.8$ ms, so periods up to 32.8 ms are measured correctly. The longest interval is the gap, which limits the method to $n > 3/32.8\,\text{ms} \approx 92$ rpm; that is enough for cranking. The first edge only initialises `t_last`:

```
52 void crank_isr(void)
53 {
54     tick_t now = now_ticks();
```

```
    tick_t T = now - t_last;              /* unsigned: wrap-around safe */
    t_last = now;
    if (!have_edge) { have_edge = true; return; }

    bool gap = (T_prev != 0) && (T > 2 * T_prev);
    T_prev = T;
```

Note `T_prev != 0`: right after the start there is no previous period, so no gap decision is made (this is the extra edge needed in 1.1 d).
Step 2 – gap and tooth counter. At the gap the counter is reset to 0 and the accumulated revolution statistics go to `loop()`. If the previous revolution did not end at tooth 57, an edge was lost or counted twice, and `sync_errors` is incremented. Every other edge increments `tooth` and updates sum, minimum and maximum of T:

```
    if (gap) {
        if (synced && tooth != 57) sync_errors++;
        /* revolution complete: hand over statistics to loop() */
        rev_ticks = rev_acc + T;
        Tmin_rev = Tmin_acc; Tmax_rev = Tmax_acc;
        teeth_rev = teeth_acc + 1;        /* edges incl. this one */
        rev_ready = synced;
        rev_acc = 0; teeth_acc = 0; Tmin_acc = 0xFFFF; Tmax_acc = 0;
        synced = true;
        tooth = 0;
        if (phase >= 0) phase = 360 - phase;   /* next revolution */
    } else {
        if (!synced) return;
        tooth++;
        rev_acc += T;
        teeth_acc++;
        if (T < Tmin_acc) Tmin_acc = T;
        if (T > Tmax_acc) Tmax_acc = T;
        T_tooth_rev = T;
    }
```

Step 3 – TDC LED. Switch it on at teeth 20/50 and off 5 teeth later:

```
    /* crank angle of this edge = phase + 6 * tooth */
    if (tooth == TDC_TOOTH_A || tooth == TDC_TOOTH_B) {
        digitalWrite(TDC_LED, HIGH);
        /* cylinder 1: TDC at 120 deg, only known with cam phase */
        if (phase == 0 && tooth == TDC_TOOTH_A) digitalWrite(CYL1_LED, HIGH);
    }
    if (tooth == TDC_TOOTH_A + LED_TEETH || tooth == TDC_TOOTH_B + LED_TEETH) {
        digitalWrite(TDC_LED, LOW);
        digitalWrite(CYL1_LED, LOW);
    }
```

Expected result: every revolution reports `teeth` = 58 edges (57 normal teeth + the gap edge) and `sync_err` = 0. In the logic analyzer the LED (D3) rises a few microseconds (interrupt latency) after the rising edge of tooth 20 (50) on D0. This is practically together with the 50 µs TDC pulse of the chip on D2, which is emitted exactly at 120°/300°. At 3000 rpm the LED is lit for $5 \cdot 333\,\mu s = 1.67\,ms$ every 10 ms. The LED marks the crank-angle *event*.
c) Speed computation. `loop()` copies the shared variables with interrupts disabled. Without that, the ISR could change a multi-byte value halfway through the read. It then applies (1). The *longest* tooth period gives the *lowest* speed, so `rpm_min` uses `tmax`:

```
        noInterrupts();
        rt = rev_ticks; tmin = Tmin_rev; tmax = Tmax_rev; tl = T_tooth_rev;
        nt = teeth_rev; ph = phase; se = sync_errors; ce = cam_errors;
        rev_ready = false;
        interrupts();
```

```
144         /* rpm = 60 / T_rev[s];  per tooth (6 deg): rpm = 1 / T_tooth[s] */
145         float rpm_rev = 60.0f * TICKS_PER_S / rt;
146         Serial.print(rev++);              Serial.print(',');
147         Serial.print(millis());           Serial.print(',');
148         Serial.print(rpm_rev, 1);         Serial.print(',');
149         Serial.print(TICKS_PER_S / tl, 1); Serial.print(',');
150         Serial.print(TICKS_PER_S / tmax, 1); Serial.print(',');
151         Serial.print(TICKS_PER_S / tmin, 1); Serial.print(',');
```

Step 1 – per-revolution resolution. One revolution lasts 20 ms = 5000 ticks (`micros()`) or 40000 ticks (Timer1). One tick therefore gives $\Delta n = 3000/5000 = 0.6$ rpm, resp. $3000/40000 = 0.075$ rpm.
Step 2 – expected per-tooth values. With the ripple, the tooth period varies by $\pm 1\,\%$, i.e. $T_k = 330.0 \ldots 336.7$ µs. Since `micros()` returns multiples of 4 µs, the measured period is one of 328, 332, 336, 340 µs, which gives the speed levels 3048.8, 3012.0, 2976.2, 2941.2 rpm. These levels are ≈ 36 rpm apart, as predicted in a).
Step 3 – check with the offline model. `code/solutions/wokwi/sheet01_crank/tooth_sim.c` reproduces exactly this measurement over 200 revolutions:

```
$ ./tooth_sim 3000 4
rpm 3000, tick 4.00 us: per-tooth rpm mean 3000.7 std 25.7 min 2941.2 max 3048.8 (11286 teeth)
  per-revolution rpm 2999.40..3000.00
$ ./tooth_sim 3000 0.5
rpm 3000, tick 0.50 us: per-tooth rpm mean 3000.6 std 21.7 min 2967.4 max 3030.3 (11286 teeth)
  per-revolution rpm 2999.85..2999.85
```

(In Wokwi, rare extra outliers of a few µs can occur when the crank interrupt has to wait for the Timer0 or UART interrupt.)

time base	rpm_rev	per-tooth min. . . max	per-tooth std
`micros()`, 4 µs	2999.4. . . 3000.0	2941. . . 3049	25.7 rpm
Timer1, 0.5 µs	2999.85	2967. . . 3030	21.7 rpm

Step 4 – interpretation. The per-revolution speed is slightly *below* 3000. The reason is that the revolution time is $T_{\text{rev}} = \int_0^{360^\circ} \mathrm{d}\theta/\omega$, and the mean of $1/(1+\varepsilon\sin 2\theta)$ is $\approx 1+\varepsilon^2/2$. So $n_{\text{rev}} \approx n(1-\varepsilon^2/2) = 3000 - 0.15$ rpm.
The per-tooth spread splits into two independent parts, which add as variances:

- *Information*: the real ripple, a sine of amplitude 30 rpm with 2 periods per revolution. Its std is $30/\sqrt{2} = 21.2$ rpm. On a real engine these are the torque pulses of the individual combustions, used e.g. for misfire detection.
- *Measurement error*: each period is the difference of two time stamps, and each stamp has a rounding error that is uniform over one tick. The difference therefore has variance $2 \cdot \Delta^2/12 = \Delta^2/6$, i.e. a std of $n^2\Delta/\sqrt{6}$: 14.7 rpm for `micros()`, 1.8 rpm for Timer1.

$$\sigma_{\text{micros}} \approx \sqrt{21.2^2 + 14.7^2} = 25.8\,\text{rpm}, \qquad \sigma_{\text{Timer1}} \approx \sqrt{21.2^2 + 1.8^2} = 21.3\,\text{rpm},$$

in good agreement with 25.7 and 21.7 rpm. With Timer1 almost all of the "jitter" is information. With `micros()`, the measurement error is of the same size as the information. The plotted dump shows a clean sine with 2 periods per 58 teeth for Timer1, and a staircase-like, noisy version for `micros()`. The gap lines (`tooth` = 255) are already converted with $3/T$ in the dump, so they do not show up as outliers.
d) 7000 rpm. Use the same steps with $T_k = 142.9$ µs. `micros()` gives periods 140 . . . 148 µs, i.e. 7142.9 . . . 6756.8 rpm. The predicted std is $\sqrt{(70/\sqrt{2})^2 + (196/\sqrt{6})^2} = \sqrt{49.5^2 + 80.0^2} = 94$ rpm for `micros()` and $\sqrt{49.5^2 + 10.0^2} = 50.5$ rpm for Timer1. The model gives:

```
$ ./tooth_sim 7000 4
rpm 7000, tick 4.00 us: per-tooth rpm mean 7002.1 std 94.2 min 6756.8 max 7142.9 (11286 teeth)
```

```
  per-revolution rpm 6999.53..7002.80
$ ./tooth_sim 7000 0.5
rpm 7000, tick 0.50 us: per-tooth rpm mean 7001.2 std 51.3 min 6920.4 max 7092.2 (11286 teeth)
  per-revolution rpm 6999.53..6999.94
```

With `micros()` the measurement error is now larger than the ripple (±70 rpm). The per-revolution value stays within ±3 rpm (one tick out of 8.57 ms/4 µs = 2143 ticks) resp. 0.5 rpm. When the slider is moved, the speed changes abruptly in the simulator. The gap is still found (ratio 3 vs. 1, see a)) as long as the jump does not fall exactly into the gap.
UART load:

$$\text{edges/s} = \frac{7000}{60} \cdot 58 \approx 6770 \text{ s}^{-1},$$
$$\text{chars/s} \approx 6770 \cdot 20 \approx 135\,000 \text{ s}^{-1} \quad (\text{line } \texttt{1234,57,286,6993.0} + \text{CR/LF}),$$
$$\text{UART capacity} = \frac{115\,200 \text{ bit/s}}{10 \text{ bit/char}} = 11\,520 \text{ chars/s}.$$

Printing one line per tooth would need almost 12× the UART capacity. One line per revolution (≈ 50 chars, 117 rev/s) needs ≈ 5800 chars/s, i.e. half the capacity. Hence statistics per revolution, and a block dump from RAM that is printed after the recording (120 lines of ≈ 16 chars take ≈ 170 ms).

Exercise 1.3 Cam/crank synchronisation

CHALLENGE

Extend your sketch such that it determines the 720° phase from the cam signal (`eng:CAM`, HIGH for $0° \le \theta < 180°$).

a) Why should the cam level not be read at the reference edge ($\theta = 0°$), and which tooth is a good choice instead?

b) Implement `phase` (0° or 360°), toggle it at every gap, and check it at every revolution (`cam_err`). Light the LED on pin 11 only at the compression TDC of cylinder 1 and verify it with the logic analyzer (D1 = cam, D4 = LED).

c) Give the crank angle and the number of the cylinder that fires at each of the four TDC events.

Solution

a) Where to read the cam level. The cam edge at $\theta = 0°$ coincides with the reference edge of the crank signal. In the simulator both are written in the same event. On a real engine the cam edge has a tolerance of several degrees and drifts with cam phasing. Reading the level at the reference edge is therefore a race: the result depends on which edge comes first. The level must be read in the *middle* of a cam state, far from both cam edges (0° and 180°), e.g. at tooth 10 (60° or 420°):

$$\text{HIGH at tooth } 10 \Rightarrow \theta = 60° \text{ (phase } 0°), \qquad \text{LOW at tooth } 10 \Rightarrow \theta = 420° \text{ (phase } 360°).$$

Any tooth whose distance to 0° and 180° is ≥ 10° works, i.e. teeth 2 . . . 28.
b) Implementation. The phase logic consists of two parts. Step 1: *predict*. Every gap starts a new revolution, so the phase toggles between 0 and 360° (inside the gap branch of the ISR):

```
72          if (phase >= 0) phase = 360 - phase;   /* next revolution */
```

Step 2: *measure and check*. At tooth 10 the cam level gives the phase directly. If a prediction exists and the two disagree, `cam_err` is incremented. The measured value always wins, which also does the first synchronisation (`phase` = −1 before):

```
94      /* Ex 1.3: cam level in the middle of a cam state (60 or 420 deg) */
95      if (tooth == CAM_TOOTH) {
```

```
96          int16_t p = digitalRead(CAM_PIN) ? 0 : 360;
97          if (phase >= 0 && p != phase) cam_errors++;
98          phase = p;
99      }
```

Step 3: *use it.* The cylinder-1 LED is switched together with the TDC LED, but only at tooth 20 *and* phase 0°, i.e. at $\theta = 120°$ (line 87 in the listing of Exercise 1.2 b).
Expected result: `cam_err` stays 0 and the printed `phase` alternates 0, 360, 0, … The green LED (D4) is on only every second time the red TDC LED lights up at tooth 20, i.e. once per engine cycle (every 40 ms at 3000 rpm). It always lies inside the cam HIGH phase on D1 (at 120°, cam HIGH for 0 … 180°).
c) Cylinder assignment. The crank angle of an edge is $\theta = \text{phase} + 6°k$. Insert the two TDC teeth and the two phases, then use the TDC positions of the model (cylinders 1, 3, 4, 2 at 120, 300, 480, 660°, firing order 1-3-4-2):

phase	tooth	crank angle	cylinder
0°	20	$0 + 6 \cdot 20 = 120°$	1
0°	50	$0 + 6 \cdot 50 = 300°$	3
360°	20	$360 + 120 = 480°$	4
360°	50	$360 + 300 = 660°$	2

In code this is a small lookup, e.g. `const byte cyl[2][2] = {{1, 3}, {4, 2}};` indexed with `cyl[phase == 360][tooth == 50]`.

Deliverables

1.1: completed classification table and time-scale table. 1.2: sketch, screenshot of the logic analyzer (gap, TDC, LED), table of rpm statistics for both time bases at 3000 and 7000 rpm, plot of a tooth-period dump. 1.3 (optional): extended sketch, logic analyzer screenshot with cam and cylinder-1 LED. Bonus 1.B1 (optional): program, table of statistics and periods, short interpretation.

Bonus: JSLinux (optional)

For interested students: the following exercise analyses all engine signals offline in JSLinux (set up as in Bonus Exercise 0.B1).

Exercise 1.B1 Signal zoo

JSLINUX

The program `code/bonus/jslinux/sheet01/signal_zoo.c` generates 160 ms (four engine cycles) of the crank, cam, knock and ion signals at 3000 rpm, $f_s = 25$ kHz, and 20 s of a lambda sensor signal ($f_s = 200$ Hz) for a slowly oscillating $\lambda(t)$. Additionally, a crank signal during a speed ramp 2000 rpm to 4000 rpm is generated.

a) Complete `stats()`: mean, RMS value, minimum and maximum. Run the program and explain the values of the crank and cam signals analytically. Why is the mean of the knock signal not zero, although the knock sensor is AC coupled?

b) The normalised autocorrelation

$$r[m] = \frac{\hat{R}[m]}{\hat{R}[0]}, \qquad \hat{R}[m] = \frac{1}{N-m} \sum_{n=0}^{N-1-m} (x[n] - \bar{x})(x[n+m] - \bar{x})$$

is close to 1 at lags m where the signal resembles a shifted copy of itself. Complete `autocorr()` and `period_lag()` (first local maximum after the first zero crossing that reaches at least 90 % of the largest value). Report the estimated period and r for all six signals.

c) Interpret the results: Which signals are periodic, which only quasi-periodic? Why does the crank signal show a strong peak already at half a revolution? What happens to the periodicity of the crank signal during the ramp – and which lag is still detected? Why would the $1/N$-normalised (biased) estimate be misleading here?

Solution

a) Statistics. `stats()` needs only one loop over the samples. It accumulates $\sum x$ and $\sum x^2$ and tracks the extremes. Then $\bar{x} = \frac{1}{N}\sum x[n]$ and $x_{\text{rms}} = \sqrt{\frac{1}{N}\sum x[n]^2}$ (reference solution `code/solutions/bonus/jslinux/sheet01/signal_zoo.c`):

```c
static void stats(const char *name, const double *x, int n)
{
    double mean = 0.0, ms = 0.0, mn = x[0], mx = x[0];
    int i;
    for (i = 0; i < n; i++) {
        mean += x[i];
        ms += x[i] * x[i];
        if (x[i] < mn) mn = x[i];
        if (x[i] > mx) mx = x[i];
    }
    mean /= n;
    ms /= n;
    printf("%-14s %9.4f %9.4f %9.4f %9.4f\n", name, mean, sqrt(ms), mn, mx);
}
```

Output:

signal	mean	rms	min	max
crank	0.4808	0.6934	0.0000	1.0000
cam	0.2500	0.5000	0.0000	1.0000
knock	0.0433	0.1234	-1.1454	1.5144
ion	0.0993	0.2399	-0.1269	1.0440
lambda	0.5004	0.6195	0.1000	0.9000

Binary signals, analytically. Let $x \in \{0, 1\}$ with duty cycle d (fraction of samples that are 1). Then $\bar{x} = d$. Because $x^2 = x$, also $\overline{x^2} = d$, so $x_{\text{rms}} = \sqrt{d}$.

- Crank: 58 teeth with 3° HIGH each per 360°, so $d = 58 \cdot 3/360 = 0.4833$. Measured: 0.4808 and $\sqrt{0.4808} = 0.6934$. The small deviation (0.0025, i.e. about 10 of 4000 samples) is a sampling effect. One sample is $18\,000\,°/\text{s}/25\,\text{kHz} = 0.72°$, so a HIGH phase of 3° lasts only 4.17 sampling intervals and is represented by 4 or 5 samples, depending on where the sampling grid lies. Over the short record these roundings do not average out exactly.
- Cam: $d = 180/720 = 0.25$, RMS $= \sqrt{0.25} = 0.5$ – exact.

Knock mean. The knock signal contains a low-frequency combustion vibration: a positive half sine of amplitude $A = 0.2\,\text{V}$ during 60° after every TDC, i.e. during 60° of every 180°. The mean of a half sine over its own duration is $2A/\pi$, so

$$\bar{x} = A \cdot \frac{2}{\pi} \cdot \frac{60°}{180°} = 0.2 \cdot 0.637 \cdot 0.333 = 0.042\,\text{V},$$

which is the measured mean. (The noise, the valve bursts and the knock oscillations have zero mean.) A real piezo sensor is AC-coupled. The model represents the signal after a high-pass whose cut-off is far below the combustion frequency, so this component survives. The large extremes ($-1.15/+1.51\,\text{V}$) come from knock events.

Lambda RMS. The mean of 0.50 matches a signal that is lean and rich half of the time each. A pure two-level signal (0.1/0.9 V) would have RMS $\sqrt{0.5 \cdot 0.1^2 + 0.5 \cdot 0.9^2} = 0.640$. The measured value is lower (0.6195) because the sensor curve $V = 0.5 + 0.4\tanh((1-\lambda)/0.005)$ with $\lambda = 1 + 0.02\sin\varphi$ spends some time in the transition region: $\overline{V^2} = 0.25 + 0.16\,\overline{\tanh^2(4\sin\varphi)} = 0.25 + 0.16 \cdot 0.836 = 0.384$, so $\sqrt{0.384} = 0.6195$.

b) Autocorrelation and period search. *Small example by hand:* $x = [1,1,0,0,1,1,0,0]$ (period 4), $\bar{x} = 0.5$, deviations ± 0.5. At lag 2 every product is $(+0.5)(-0.5)$, at lag 4 every product is $(\pm 0.5)^2$, so $\hat{R}[0] = 0.25$, $\hat{R}[2] = -0.25$, $\hat{R}[4] = 0.25$, and $r[2] = -1$, $r[4] = 1$. The division by $N - m$ (not N) is what makes $r[4]$ exactly 1, although only 4 products exist at that lag.

`autocorr()` implements the formula literally. First it removes the mean, then it averages $N - m$ products per lag and normalises by $\hat{R}[0]$:

```
48 static void autocorr(const double *x, int n, double *rr, int maxlag)
49 {
50     double mean = 0.0, r0;
51     int i, m;
52     for (i = 0; i < n; i++) mean += x[i];
53     mean /= n;
54     for (m = 0; m <= maxlag; m++) {
55         double s = 0.0;
56         for (i = 0; i + m < n; i++) s += (x[i] - mean) * (x[i + m] - mean);
57         rr[m] = s / (n - m);
58     }
59     r0 = rr[0] > 0.0 ? rr[0] : 1.0;
60     for (m = 0; m <= maxlag; m++) rr[m] /= r0;
61 }
```

`period_lag()` takes three steps. (1) It skips the main lobe around $m = 0$ (every signal resembles itself for small shifts) up to the first zero crossing. (2) It finds the largest value $r_{\max}$ beyond. (3) It returns the *first* local maximum with $r \geq 0.9\, r_{\max}$. Taking the first one avoids picking a multiple of the period, whose peak is almost as high:

```
67 static int period_lag(const double *rr, int maxlag, double *peak)
68 {
69     int m0 = 1, m;
70     double rmax = -1.0;
71     while (m0 <= maxlag && rr[m0] > 0.0) m0++;        /* first zero crossing */
72     for (m = m0; m <= maxlag; m++) if (rr[m] > rmax) rmax = rr[m];
73     for (m = m0 + 1; m < maxlag; m++)
74         if (rr[m] >= rr[m - 1] && rr[m] >= rr[m + 1] && rr[m] >= 0.9 * rmax) {
75             *peak = rr[m];
76             return m;
77         }
78     *peak = 0.0;
79     return 0;
80 }
```

Result:

```
crank 3000   first peak: lag  250 =   10.00 ms r = 0.929 | max: lag  500 r = 0.988
crank ramp   first peak: lag    8 =    0.32 ms r = 0.348 | max: lag    8 r = 0.348
cam          first peak: lag 1000 =   40.00 ms r = 1.000 | max: lag 1000 r = 1.000
knock        first peak: lag  244 =    9.76 ms r = 0.418 | max: lag  500 r = 0.438
ion          first peak: lag  250 =   10.00 ms r = 0.992 | max: lag  250 r = 0.992
lambda       first peak: lag  251 = 1255.00 ms r = 0.981 | max: lag  500 r = 0.982
```

Expected lags at $f_s = 25\,\text{kHz}$: $180° = 10\,\text{ms} = 250$ samples, revolution $= 500$, engine cycle $= 1000$. For lambda at $200\,\text{Hz}$: $1/0.8\,\text{Hz} = 1.25\,\text{s} = 250$ samples.

c) Interpretation.

- *Cam*: exactly periodic, $r = 1.000$ at $40\,\text{ms}$ = one engine cycle.
- *Crank*: the true period is one revolution ($20\,\text{ms}$, $r = 0.988$). It is not exactly 1 because of the speed ripple and because the period is 500.03 samples, not an integer. Why is $r(250)$ almost as high? Shift the wheel by 180°: all teeth fall onto teeth, except that the two missing teeth (58, 59 at 348°, 354°) meet the real teeth 28, 29 at 168°, 174°, and vice versa.

So x and its shifted copy y differ on $2 \cdot 2 \cdot 3° = 12°$ of 360°. With $\overline{(x-y)^2} = 12/360 = 0.033$ and $\overline{xy} - \bar{x}^2 = \sigma^2 - \frac{1}{2}\overline{(x-y)^2}$, where $\sigma^2 = d(1-d) = 0.250$:

$$r(250) \approx 1 - \frac{0.033/2}{0.250} = 0.933 \quad \text{(measured 0.929)}.$$

The gap, which carries the position information, is only a small part of the signal energy. With the 90 % rule the detector therefore reports 180°, a reminder that a period detector must be designed for the feature of interest. The peaks are very narrow: one sample less (0.72°) shifts every tooth edge and gives $r(249) = 0.48$. (Small peaks at multiples of 8.33 samples come from the teeth.)

- *Crank during the ramp*: the tooth period $6°/\omega$ shrinks from 0.5 ms (2000 rpm) to 0.25 ms (4000 rpm), i.e. from 12.5 to 6.25 samples (mean ≈ 8.3). No long-lag peak survives, and only the tooth-scale peak at lag 8 ($r = 0.35$) remains. In time the signal is not periodic. In the angle domain it would be (Sheet 4).
- *Knock*: $r \approx 0.42$ at ≈ 10 ms (180°). Only the deterministic parts (combustion vibration, valve impacts at 8 kHz) repeat. Noise and the random knock bursts (random onset and phase) do not correlate. So the signal is quasi-periodic, i.e. cyclostationary. The lag 244 instead of 250 is a side effect of the 8 kHz valve bursts: their period is only 3.1 samples, so $r[m]$ jumps between 0.30 and 0.44 from lag to lag around $m = 250$. The first local maximum above the threshold is then at 244.
- *Ion current*: $r = 0.992$ at 180°. In the model all cylinders produce the same deterministic ion curve. On a real engine cycle-to-cycle variations make each event an individual (aperiodic) transient.
- *Lambda*: period 1.25 s = 1/0.8 Hz, $r = 0.98$. The switching sensor turns the small λ oscillation into an almost rectangular signal. Noise shifts the switching instants slightly, hence 251 instead of 250.

Biased estimator. With $1/N$ instead of $1/(N-m)$, every $r[m]$ is multiplied by $(N-m)/N$. At lag 1000 of a 4000-sample record this is a factor of $3000/4000 = 0.75$, so the cam signal would show $r = 0.75$ although it is perfectly periodic. The factor also decreases with m, so shorter lags would always be favoured: the crank peak at lag 500 would drop to $0.988 \cdot 0.875 = 0.86$.

Digital Signal Processing
Systems
Script: Chapter 2 (Systems)

Exercise Sheet 2
with solutions

Learning goals

- Use the operator view $y = S\{x\}$ and decide for a given system whether it is static or dynamic (memory), causal, linear and time-invariant.
- Understand how a two-point lambda controller with integrator works, why it produces a limit cycle, and predict its period from the loop dead time.
- *Bonus (optional):* design numerical test harnesses that *reveal* a property and know their pitfalls (impulse-only tests, small amplitudes, boundary effects); model engine subsystems as dynamic systems: inertia (first-order lag), wall film, transport delay, switching sensor.

Definitions used on this sheet

For a system $y = S\{x\}$ (continuous or discrete time):

- *memoryless (static)*: $y(t_0)$ depends only on $x(t_0)$;
- *causal*: $y(t_0)$ depends only on $x(t)$, $t \le t_0$; equivalently, if $x_1(t) = x_2(t)$ for $t < t_0$, then $y_1(t) = y_2(t)$ for $t < t_0$;
- *linear*: $S\{ax_1 + bx_2\} = aS\{x_1\} + bS\{x_2\}$ for all inputs and constants (superposition = additivity + homogeneity);
- *time-invariant*: $y(t) = S\{x(t)\} \Rightarrow y(t - t_0) = S\{x(t - t_0)\}$ for every shift t_0 (discrete time: integer shifts n_0).

Linearity and time invariance are previewed here; Chapter 3 shows why their combination (LTI) is so powerful.

Exercise 2.1 Properties by hand

PAPER

a) Decide for each system whether it is memoryless, causal, linear and time-invariant (table with yes/no):

S1: $y(t) = 2x(t)$	S2: $y(t) = x^2(t)$	S3: $y[n] = x[n+1] - x[n]$
S4: $y[n] = \frac{1}{3}(x[n] + x[n-1] + x[n-2])$	S5: $y(t) = x(t - T_d)$, $T_d > 0$	S6: $y(t) = \lvert x(t)\rvert$
S7: $y(t) = \int_{-\infty}^{t} x(\tau)\,\mathrm{d}\tau$	S8: $y[n] = n\,x[n]$	S9: $y[n] = x[n] + 0.5$
S10: $y[n] = x[2n]$	S11: $n_k = 1/T_k$ (Sheet 1)	S12: $U = 0.5 + 0.4\tanh\frac{1-\lambda}{0.005}$

b) Prove your answers formally for S6 (linearity), S8 (time invariance) and S10 (causality and time invariance). For S9, show that the system is not linear, but that the *difference* of two outputs depends linearly on the difference of the inputs (*incrementally linear*).

c) An engineer computes the mean speed of one crank revolution as the arithmetic mean of the 58 per-tooth speeds $1/T_k$, treating the gap period like a normal tooth period. Compute the relative error at constant speed. After correcting the gap ($3/T_{\text{gap}}$), is the arithmetic mean of the per-tooth speeds equal to $60/T_{\text{rev}}$? Which property of S11 is responsible, and what is the error for a speed fluctuation of $\pm 1\,\%$ (sinusoidal)?

Solution

a) Property table. Treat every system with the same four questions, always in this order:

Step 1: *Memory.* Which input values does $y(t_0)$ (or $y[n_0]$) need? If it needs only $x(t_0)$, the system is memoryless.

Step 2: *Causality.* If it needs other values: are any of them in the *future*? Then the system is non-causal.

Step 3: *Linearity.* Quick check first: a linear system maps $x = 0$ to $y = 0$. If this holds, test homogeneity ($S\{ax\} = aS\{x\}$) and additivity ($S\{x_1 + x_2\} = S\{x_1\} + S\{x_2\}$). Any nonlinear operation *on the values* of x (square, absolute value, tanh, $1/x$, constant offset) destroys linearity.

Step 4: *Time invariance.* Feed the shifted input $x(t - t_0)$ and compare with the shifted output $y(t - t_0)$. Warning signs for time variance: the time variable appears *outside* the argument of x (like the factor n in S8), or the time axis itself is transformed (like $2n$ in S10).

This gives (M = memoryless, C = causal, L = linear, TI = time-invariant):

system	M	C	L	TI	remark
S1 $2x$	yes	yes	yes	yes	amplifier
S2 x^2	yes	yes	no	yes	$S\{2x\} = 4S\{x\}$
S3 $x[n+1] - x[n]$	no	no	yes	yes	forward difference uses the future
S4 3-point MA	no	yes	yes	yes	memory of 2 samples
S5 delay	no	yes	yes	yes	transport delay (lambda!)
S6 $\lvert x\rvert$	yes	yes	no	yes	rectifier
S7 integrator	no	yes	yes*	yes	*initially at rest
S8 $n\,x[n]$	yes	yes	yes	no	gain changes with time
S9 $x + 0.5$	yes	yes	no	yes	affine, $S\{0\} \neq 0$
S10 $x[2n]$	no	no	yes	no	down-sampler
S11 $1/T$	yes	yes	no	yes	speed from tooth period
S12 tanh	yes	yes	no	yes	switching lambda sensor

Comments on the less obvious entries:

- *S3, S4, S5* have memory: they need x at times other than n. Only S3 needs a *future* value ($x[n+1]$) and is therefore non-causal.
- *S7*: $y(t) = \int_{-\infty}^{t} x\,\mathrm{d}\tau$ needs the whole past. It is linear only if the integrator starts at rest. With an initial value $y_0 \neq 0$ it behaves like S9 (an offset that does not scale with x).
- *S11*: each speed value is computed from one period T_k only, so it is memoryless *in the tooth index* k, but $1/T$ is nonlinear.
- *S12*: the sensor characteristic is static (the sensor lag is a separate block), but extremely nonlinear: it saturates at 0.1 and 0.9 V.

b) Proofs. To show that a property *fails*, one concrete counterexample is enough. To show that it *holds*, you need a proof for all inputs and all constants.
S6, linearity.

Step 1: Homogeneity requires $S\{ax\} = aS\{x\}$ for every a.

Step 2: Choose $x_1(t) = 1$ and $a = -1$: $S\{-x_1\} = |-1| = 1$, but $-S\{x_1\} = -1$. Homogeneity fails ⇒ S6 is not linear.

Step 3: Additivity fails as well: $x_1 = 1$, $x_2 = -1$ gives $S\{x_1 + x_2\} = 0 \neq S\{x_1\} + S\{x_2\} = 2$.

Step 4: For comparison, TI holds: $x_2(t) = x(t - t_0)$ gives $y_2(t) = |x(t - t_0)| = y(t - t_0)$.

How a time-invariance proof works. The shifted input $x_2[n] = x[n - n_0]$ is not derived – you *choose* it, because the test requires it. Time invariance means: the system behaves today as it did yesterday; the same input applied n_0 samples later gives the same output, n_0 samples later. So compare two experiments:

(i) original: input $x[n]$, output $y[n] = S\{x\}[n]$;

(ii) delayed: the same signal fed in n_0 samples later. What x had at time m, x_2 must have at time $m + n_0$. With $n = m + n_0$, i.e. $m = n - n_0$: $x_2[n] = x[n - n_0]$ (the minus sign means a delay, a shift to the right).

The system is time-invariant if the response to the delayed input equals the delayed original response, $S\{x_2\}[n] = y[n - n_0]$, for every x and every n_0.
S8, time invariance.

Step 1: Shifted input: $x_2[n] = x[n - n_0]$.

Step 2: Response to it: $y_2[n] = n\,x_2[n] = n\,x[n - n_0]$.

Step 3: Shifted output: $y[n - n_0] = (n - n_0)\,x[n - n_0]$.

Step 4: The two differ by $n_0\,x[n - n_0] \neq 0$ for $n_0 \neq 0 \Rightarrow$ time-variant. The gain n depends on the absolute time. The signal x is shifted, the factor n is not.

Step 5: *Example.* Impulse at $n = 2$: $x[2] = 1$, so $y[2] = 2 \cdot 1 = 2$. Feed it $n_0 = 3$ samples later: $x_2[5] = 1$, so $y_2[5] = 5 \cdot 1 = 5$. The delayed original response is $y[5 - 3] = y[2] = 2 \neq 5$: the same impulse is amplified more strongly when it arrives later.

Step 6: S8 is nevertheless linear: $n\,(ax_1[n] + bx_2[n]) = a\,nx_1[n] + b\,nx_2[n]$.

S10, causality and time invariance.

Step 1: Causality: $y[1] = x[2]$, $y[5] = x[10]$, ...: for $n > 0$ the output needs a *future* input $\Rightarrow$ non-causal. For the same reason the output depends on $x[2n] \neq x[n] \Rightarrow$ memory.

Step 2: Shift by one: $x_2[n] = x[n - 1]$ gives $y_2[n] = x_2[2n] = x[2n - 1]$.

Step 3: Shifted output: $y[n - 1] = x[2(n - 1)] = x[2n - 2]$.

Step 4: $x[2n - 1] \neq x[2n - 2]$ in general $\Rightarrow$ time-variant.

Step 5: Even shifts $n_0 = 2m$ work: $y_2[n] = x[2n - 2m] = y[n - m]$, i.e. the output is shifted by $n_0/2$. Such a system is called *periodically time-variant.*

S9, incremental linearity.

Step 1: Quick check: $S\{0\} = 0.5 \neq 0 \Rightarrow$ not linear. (Homogeneity: $S\{ax\} = ax + 0.5 \neq a(x + 0.5)$.)

Step 2: Take two inputs: $y_1 - y_2 = (x_1 + 0.5) - (x_2 + 0.5) = x_1 - x_2$.

Step 3: The *difference* of the outputs is a linear function (here the identity) of the difference of the inputs: S9 is incrementally linear.

This is how all "linearised" engine models work: small deviations from an operating point are transferred linearly, although the system itself is affine or nonlinear.

c) Mean speed from per-tooth speeds.

Step 1: *Per-tooth speed.* A normal tooth spans $6° = \frac{1}{60}$ revolution. With the period T_k in seconds: $n_k = \frac{1/60 \text{ rev}}{T_k} \cdot 60\,\text{s/min} = \frac{1}{T_k}$ in rpm.

Step 2: *Gap.* The gap spans 18°, i.e. three teeth: $T_{\text{gap}} = 3T$ at constant speed $n = 1/T$. Treated like a normal tooth, it contributes $1/T_{\text{gap}} = n/3$.

Step 3: *Mean over the 58 periods* (57 normal teeth + gap):

$$\bar{n} = \frac{57\,n + n/3}{58} = 0.98851\,n, \qquad \text{error } -1.15\,\%.$$

Step 4: *Gap corrected* ($3/T_{\text{gap}}$): all 58 values equal n, so at constant speed the mean is exact.

Step 5: *Speed fluctuation.* The physically correct mean speed is "angle travelled / time" $= 60/T_{\text{rev}}$. Model the fluctuation as $n(\theta) = n_0(1 + \varepsilon \sin 2\theta)$, $\varepsilon = 0.01$. The per-tooth speeds sample $n(\theta)$ at equally spaced *angles*; the sine averages to zero over a revolution, so their mean is n_0.

Step 6: *Revolution time.* The time for one revolution is the integral of $\mathrm{d}\theta/\omega$, i.e. the mean of $1/n(\theta)$ over the angle:

$$T_{\text{rev}} = \frac{60}{n_0}\,\overline{\frac{1}{1+\varepsilon\sin 2\theta}} \approx \frac{60}{n_0}\,\overline{1-\varepsilon\sin 2\theta+\varepsilon^2\sin^2 2\theta} = \frac{60}{n_0}\Big(1+\frac{\varepsilon^2}{2}\Big),$$

using $\overline{\sin} = 0$ and $\overline{\sin^2} = \frac{1}{2}$. Hence $60/T_{\text{rev}} \approx n_0(1 - \varepsilon^2/2)$.

Step 7: *Numbers.* The arithmetic mean of the per-tooth speeds is too high by $\varepsilon^2/2 = 5 \cdot 10^{-5}$, i.e. 0.15 rpm at 3000 rpm – exactly the offset seen on Sheet 1.

Step 8: *Responsible property.* S11 ($1/T$) is *nonlinear* and convex. For a convex function the mean of the function values is larger than the function of the mean (Jensen's inequality): $\overline{1/T} \geq 1/\overline{T}$. Averaging and the nonlinearity do not commute.

Correct procedure: average in the period domain (sum of the T_k) and invert *once*: $60/T_{\text{rev}}$.

Exercise 2.2 Closed lambda loop

WOKWI

Project `code/wokwi/sheet02_lambda/`: Arduino Uno and the custom chip `lambda-probe`. Pin 9 sends the fuel command as PWM (`analogWrite`, 490 Hz, duty $= q/2$), A0 reads the sensor voltage `O2`, A1 the ground truth `LAM` $= \lambda/2$. The chip's sliders set the unmetered air (slider − 10 in %, default +5 %) and the transport delay (default 200 ms).

How a two-point controller works. The goal is the stoichiometric mixture $\lambda = 1$. The ECU does not know the fuel quantity q^* that achieves it, because part of the air is not measured (leaks, sensor and injector tolerances, ageing): with unmetered air a, $\lambda = (1 + a)/q$ and $q^* = 1 + a$. The switching sensor (S12 of Exercise 2.1) delivers only *one bit* of information: $U > 0.45$ V means "rich" ($\lambda < 1$, too much fuel), $U < 0.45$ V means "lean" ($\lambda > 1$, too little fuel) – but not *how* rich or lean. The controller therefore uses the simplest possible rule. In every controller period $T_s = 10$ ms:

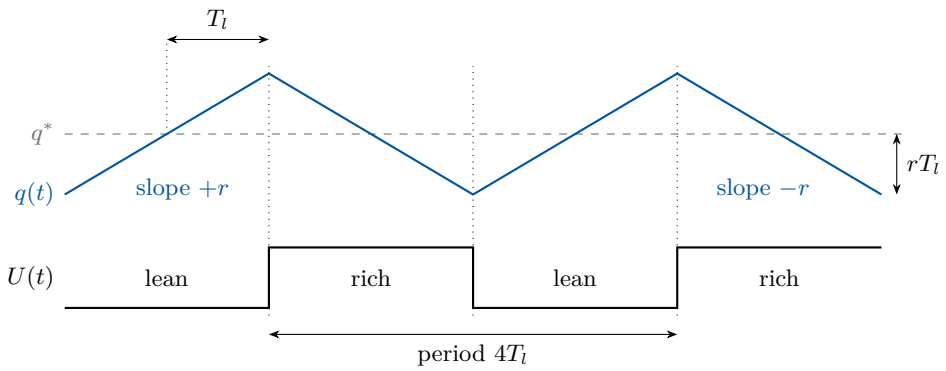
- sensor rich ⇒ decrease q by rT_s (ramp down with slope $-r$),
- sensor lean ⇒ increase q by rT_s (ramp up with slope $+r$),
- optionally, at the moment the sensor *switches*, an additional jump of q by P towards the other side ($-P$ on lean→rich, $+P$ on rich→lean).

"Two-point" means that the controller output (here the slope $\dot{q}$) takes only two values, $\pm r$, like a relay or a thermostat. "With integrator" means that q is the integral of this output: as long as the sensor reports the same state, q keeps moving in the same direction, so it always moves towards q^*, whatever the value of a. On average the controller therefore finds q^* by itself (integral action).

Why it oscillates. The sensor sees a change of q only after the transport delay T_d and the two lags (wall film $\tau_f = 20$ ms, sensor $\tau_s = 30$ ms), i.e. after an effective dead time $T_l \approx T_d + \tau_f + \tau_s$. When q crosses q^*, the sensor still reports the old state for T_l, and q keeps ramping in the wrong direction: it overshoots by rT_l. After the switching, q needs another T_l to return to q^*, and the sensor sees this crossing only T_l later. Each half period therefore lasts $2T_l$:

$$T_{\text{period}} \approx 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

The oscillation neither grows nor decays – a *limit cycle*. The jump P removes part of the overshoot immediately after each switching, so q reaches q^* earlier (by P/r); the period shortens to $4T_l - 2P/r$.



Limit cycle without jump ($P = 0$): U is the sign of $q - q^*$, delayed by T_l.

a) Implement the two-point controller with integrator ($r = 0.2$/s) in `loop()`. Start the simulation and observe the limit cycle in the Serial Monitor (`u_o2`, `lambda`, `q`). Copy 5 s of data into a spreadsheet (or into JSLinux, see the bonus section) and plot λ and U; compare with Fig. 1.3 of the script and with the sketch above. Predict period and λ range for $T_d = 200$ ms and check them.

b) Measure the oscillation period (column `period_ms`) for transport delays of 100, 200, 400 and 800 ms and plot period over T_d. Determine slope and intercept and interpret them.

c) Move the air slider from +5 % to −10 % and to +10 %. Describe the transient and the new mean value of q.

d) Add the jump `PJUMP` = 0.03, then reduce `RATE` to 0.1 with `PJUMP` = 0.025. Compare period and λ amplitude. Why can no two-point controller make the oscillation disappear, and what would a continuous (PI) lambda controller need?

Solution

a) Controller and limit cycle. Translate the rule of the box above line by line into `loop()`:

Step 1: *Read the sensor* and convert it to volts: $u = \text{ADC} \cdot 5/1023$ (line 38 of the template).

Step 2: *Decide rich or lean*: `rich = u > U_THR` with $U_{\text{THR}} = 0.45$ V.

Step 3: *Jump at a switching* (only if `PJUMP` $\neq 0$): if `rich` differs from the value of the last period, move q by P towards the other side. The lean→rich switching is also the moment to measure the period with `millis()`.

Step 4: *Integrator ramp*: change q by $\mp rT_s = \mp 0.2 \cdot 0.01 = \mp 0.002$ per period.

Step 5: *Limit* q to $0.5 \ldots 1.5$, so that the duty cycle stays in $0.25 \ldots 0.75$.

Step 6: *Output*: duty $= q/2$, i.e. `analogWrite` value $255 \cdot q/2 = 127.5\,q$ (rounded).

Reference solution (`code/solutions/wokwi/sheet02_lambda/sketch.ino`):

```
41    /* ---- two-point controller with integrator ---------------------- */
42    bool rich = u > U_THR;
43    if (rich != rich_old) {
44        q += rich ? -PJUMP : PJUMP;                    /* P jump */
45        if (rich) {                                    /* lean -> rich */
46            unsigned long now = millis();
47            if (t_switch) period_ms = now - t_switch;
48            t_switch = now;
49        }
50    }
51    q += (rich ? -RATE : RATE) * (TS_US * 1e-6f);      /* I ramp */
52    if (q < 0.5f) q = 0.5f;
53    if (q > 1.5f) q = 1.5f;
54    rich_old = rich;
```

```
55
56      /* duty = q / 2, 8-bit PWM */
57      analogWrite(INJ_PIN, (int)(127.5f * q + 0.5f));
```

What you should see:

- `u_o2` switches between ≈ 0.1 V and 0.9 V (ADC values ≈20 and 184), almost rectangular as in Fig. 1.3.
- q is a triangle around $q^* = 1.05$. Because one PWM step is $2/255 = 0.0078$ and the ramp adds only 0.002 per period, q changes in small stairs (one step about every 40 ms).
- λ is a triangle as well (ramps of q, slightly rounded by the wall film). The ground truth via A1 has a resolution of only $2 \cdot 4.9\,\text{mV} \approx 0.01$ in λ, so its extreme values scatter by ± 0.01.
- The first period after the start is longer: q starts at 1.0 and must first ramp up to $q^* = 1.05$.

Prediction for $T_d = 200$ ms. Effective dead time $T_l = T_d + \tau_f + \tau_s = 200 + 20 + 30 = 250$ ms. Period $4T_l = 1.0$ s (the offline simulation of Bonus Exercise 2.B2 c) with the same model as the chip gives 0.98 s). Overshoot of q: $rT_l = 0.2 \cdot 0.25 = 0.05$, so $\lambda \approx 1 \mp 0.05/1.05 = 1 \mp 0.048$, i.e. $0.952 \ldots 1.048$. Measured in the model: $0.958 \ldots 1.046$.

b) Period over transport delay.

Step 1: *Measure.* Expected values (the reference sketch run on the PC against a replica of the chip model):

T_d / ms	period	$4(T_d + 50\,\text{ms})$	λ (from A1)
100	0.58 s	0.60 s	0.978…1.026
200	1.00 s	1.00 s	0.958…1.046
400	1.80 s	1.80 s	0.919…1.095
800	3.40 s	3.40 s	0.860…1.193

In Wokwi the values may deviate by one controller period (10 ms) per switching. The offline model of Bonus Exercise 2.B2 gives up to 20 ms less, because there the controller sees the sensor value of the current millisecond (no extra sampling delay).

Step 2: *Fit a straight line* $T_{\text{period}} = m\,T_d + c$ (least squares over the four points): $m = 4.02$, $c = 0.19$ s.

Step 3: *Slope.* $m \approx 4$: per period the sensor switches twice, and each half period contains the dead time twice (the overshoot builds up during T_l, and the return to q^* is seen only T_l later). So every millisecond of transport delay adds 4 ms to the period.

Step 4: *Intercept.* $c/4 \approx 47$ ms is the part of T_l that is not transport delay: the two lags, $\tau_f + \tau_s = 50$ ms (a ramp passing a first-order lag τ is delayed by τ). The small difference comes from the 8-bit PWM stairs of q and the 10 ms controller period.

Step 5: λ *amplitude* grows with T_l as predicted: $\pm rT_l/q^* = \pm 0.029,\ 0.048,\ 0.086,\ 0.162$ for the four delays. At large T_d the range becomes asymmetric ($0.860 \ldots 1.193$), because $\lambda = q^*/q$ is a nonlinear function of q.

c) Change of the unmetered air.

Step 1: *New target.* At −10 % air the stoichiometric quantity drops to $q^* = 0.90$. At the moment of the change $q \approx 1.05$, so the mixture is suddenly rich: $\lambda = 0.90/1.05 = 0.857$.

Step 2: *Dead time.* The sensor reports rich only after the dead time (simulation: 0.21 s after the step).

Step 3: *Ramp.* The integrator now ramps q down with 0.2 s. It needs $(1.05 - 0.90)/0.2 = 0.75$ s to reach q^*.

Step 4: *Overshoot.* The sensor sees the crossing only T_l later, so lean is reported again after about $0.21 + 0.75 + 0.25 \approx 1.2\,\mathrm{s}$ (simulation `air_step()` in the reference solution of Bonus 2.B2: 1.25 s).

Step 5: *New limit cycle* around the mean $q = 0.900$. At +10 % the mixture becomes lean, q ramps up and settles around mean $q = 1.098$. The overshoot of q (rT_l) is the same; the λ amplitude rT_l/q^* changes only slightly with q^*.

The integrator removes any constant disturbance: the mean of q always ends at $q^* = 1 + a$, without knowing a. This is the purpose of the lambda loop (adaptation of fuel metering errors, air leaks, ageing).

d) Jump P.

Step 1: $P = 0.03$, $r = 0.2/s$. After each switching the jump undoes P of the overshoot $rT_l = 0.05$; q^* is reached $P/r = 0.15\,\mathrm{s}$ earlier in each half period. Prediction $4T_l - 2P/r = 1.0 - 0.3 = 0.70\,\mathrm{s}$; model: 0.70 s. The peak deviation is still rT_l (it builds up *before* the switching), so λ stays at $0.958 \ldots 1.045$: faster, but not smaller.

Step 2: $P = 0.025$, $r = 0.1/s$. Halving r halves the overshoot: $rT_l = 0.025$, λ $0.978 \ldots 1.027$ (model). Without jump the period would stay $4T_l = 1.0\,\mathrm{s}$ (it does not depend on r); the jump shortens it: prediction $1.0 - 2 \cdot 0.025/0.1 = 0.5\,\mathrm{s}$, model 0.58 s.

Step 3: *Why the deviation in step 2?* Here $P \approx rT_l$: the jump lands almost exactly on q^*. The rule "a lag delays a ramp by τ" holds for ramps, not for jumps: after a jump the lagged signal approaches its new value only exponentially, so it reaches q^* later than predicted. (It is not the 8-bit PWM: without quantisation the model gives 0.60 s as well.) Result: half the amplitude at a higher frequency than without jump.

Step 4: *Why the oscillation cannot disappear.* The sensor delivers only the *sign* of $\lambda - 1$ (relay characteristic). The controller must keep moving q to find out on which side of q^* it is. Relay + integrator + dead time always yield a limit cycle of period $\approx 4T_l$; r and P only change its amplitude and period.

Step 5: *What a PI controller would need.* A continuous controller needs a *proportional* measure of $\lambda - 1$, i.e. a wide-band (linear) lambda sensor. With the switching sensor, the oxygen storage of the catalytic converter tolerates the small, fast oscillation – one reason why period and amplitude must be kept small, i.e. why T_d matters.

Deliverables

2.1: property table, proofs. 2.2: sketch, plot of λ and U over 5 s, period over T_d (table + plot), results of c) and d). Bonus 2.B1 (optional): test program, property table with numerical evidence, $h[n]$ of the LTI systems. Bonus 2.B2 (optional): program, values of τ, response times, prediction formula and comparison table.

Bonus: JSLinux (optional)

For interested students: the following exercises identify unknown systems by experiment and simulate the lambda path offline in C. Bonus Exercise 2.B2 c) derives the limit cycle of Exercise 2.2 in detail.

Setup

The following exercises run in JSLinux, a Linux system in the browser. If you have not done Bonus Exercise 0.B1, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine

and check the compiler with `gcc -version` (or `tcc -v`).

2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet02/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o blackbox blackbox.c -lm
./blackbox
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Exercise 2.B1 Black-box system identification JSLINUX

The header `code/jslinux/sheet02/mystery.h` contains five unknown systems

$$\texttt{void Sk(const double *x, double *y, int N);} \quad k = 1, \ldots, 5$$

(input $x[0..N-1]$, $x[n] = 0$ outside, output $y[0..N-1]$). One of them is LTI; the others violate exactly one or two of the properties of Exercise 2.1. **Do not read the header** – find the properties by experiments, as you would with a supplier's ECU function delivered as object code. The program `code/jslinux/sheet02/blackbox.c` plots the responses to unit impulses at $n_0 = 30$ and $n_0 = 31$ and contains a (weak) homogeneity test with scaled impulses as an example.

a) Superposition test: compare $S\{ax_1+bx_2\}$ with $aS\{x_1\}+bS\{x_2\}$ for random signals and random weights, at amplitude 1 and 0.01. Why is the impulse test of the template not sufficient, and why two amplitudes?

b) Shift test: compare $S\{x[n-k]\}$ with $y[n-k]$ for $k = 1 \ldots 4$. Why are the first and last samples of the test signal set to zero?

c) Causality test: two inputs that agree for $n < n_0$ and differ afterwards.

d) Memory test: two inputs that differ only at n_0. Why must the test be repeated for several n_0?

e) Report a property table with the numerical evidence (largest deviation of each test). For the LTI systems determine $h[n]$ from the plots and give the difference equation. Why does an "impulse response" not characterise the other systems?

Solution

How to build a property test. All four tests follow the same recipe:

Step 1: Take the definition of the property (Exercise 2.1) and read it as an experiment: "if the inputs are related like this, the outputs must be related like that".

Step 2: Generate inputs that satisfy the *if* part – with random signals, not with special ones.

Step 3: Run the system on all inputs and compute the largest deviation from the *then* part.

Step 4: Repeat with many random inputs (`NTRY` = 20) and keep the largest deviation. Deviations below a tolerance (10^{-9}) are rounding errors.

Example, the superposition test of the reference solution (`code/solutions/jslinux/sheet02/blackbox.c`):

```
/* ---- (a) superposition: S{a x1 + b x2} = a S{x1} + b S{x2} ? ---------
   returns the largest deviation relative to the output amplitude       */
static double test_linear(system_t S, double amp)
{
```

```
56      double x1[N], x2[N], x3[N], y1[N], y2[N], y3[N], d[N], e = 0.0, r;
57      int t, n;
58      for (t = 0; t < NTRY; t++) {
59          double a = 2.0 * urand(), b = 2.0 * urand();
60          rand_signal(x1, amp, 0);
61          rand_signal(x2, amp, 0);
62          for (n = 0; n < N; n++) x3[n] = a * x1[n] + b * x2[n];
63          S(x1, y1, N); S(x2, y2, N); S(x3, y3, N);
64          for (n = 0; n < N; n++) d[n] = y3[n] - (a * y1[n] + b * y2[n]);
65          r = maxabs(d, N) / (maxabs(y3, N) + 1e-300);
66          if (r > e) e = r;
67      }
68      return e;
69  }
```

Line 62 builds $x_3 = ax_1 + bx_2$, line 64 the deviation $S\{x_3\} - (aS\{x_1\} + bS\{x_2\})$, line 65 divides by the output amplitude so that the result is comparable for $A = 1$ and $A = 0.01$. The shift test:

```
82  /* ---- (b) time invariance: S{x[n-k]} = y[n-k] ? ------------------- */
83  static double test_shift(system_t S)
84  {
85      double x[N], xs[N], y[N], ys[N], e = 0.0, d;
86      int t, k, n;
87      for (t = 0; t < NTRY; t++)
88          for (k = 1; k <= 4; k++) {
89              rand_signal(x, 1.0, 8);
90              for (n = 0; n < N; n++) xs[n] = n >= k ? x[n - k] : 0.0;
91              S(x, y, N); S(xs, ys, N);
92              for (n = k; n < N; n++) {
93                  d = fabs(ys[n] - y[n - k]);
94                  if (d > e) e = d;
95              }
96          }
97      return e;
98  }
```

Output of the program:

```
 sys  linear(A=1) linear(A=0.01) imp-scale   shift      causal     memory
 S1    0.00e+00   0.00e+00       0.00e+00   1.96e+00   0.00e+00   1.00e+00
 S2    3.78e-16   3.49e-16       0.00e+00   0.00e+00   0.00e+00   2.50e-01
 S3    1.55e+00   3.58e-04       1.29e+00   0.00e+00   0.00e+00   0.00e+00
 S4    0.00e+00   2.47e-16       0.00e+00   0.00e+00   4.61e-01   2.50e-01
 S5    8.43e-01   8.48e-03       0.00e+00   0.00e+00   0.00e+00   6.21e-01

 sys  linear  time-inv  causal  memoryless
 S1   yes     NO       yes     NO
 S2   yes     yes      yes     NO
 S3   NO      yes      yes     yes
 S4   yes     yes      NO      NO
 S5   NO      yes      yes     NO
```

For checking, the systems are (from `code/solutions/jslinux/sheet02/mystery_gen.c`):

	system	properties
S1	$y[n] = x[n - (n \bmod 3)]$ (sample & hold every 3 samples)	linear, time-variant, causal, memory
S2	$y[n] = 0.75\,y[n-1] + 0.25\,x[n-1]$	LTI, causal, memory (IIR)
S3	$y[n] = \tanh(x[n])$	nonlinear, memoryless, TI, causal
S4	$y[n] = 0.25x[n-1] + 0.5x[n] + 0.25x[n+1]$	LTI, *non-causal*, memory
S5	$y[n] = 0.625x[n] + 0.375x[n-1] + 0.25\,x[n]x[n-1]$	nonlinear, TI, causal, memory

a) Superposition.

Step 1: *Impulse test of the template* ($S\{3\delta\}$ vs. $3S\{\delta\}$, column `imp-scale`): detects only S3.

Step 2: *Why S5 passes it.* For a single impulse $x[n]x[n-1] = 0$ at every n: an impulse never excites products of *different* samples. The nonlinear term of S5 is invisible.

Step 3: *Random signals with random weights* excite all such terms: S5 now shows a deviation of 0.84.

Step 4: *Why two amplitudes.* S3 ($\tanh x \approx x - x^3/3$) is almost linear for small inputs: relative deviation $3.6 \cdot 10^{-4}$ at $A = 0.01$ vs. 1.55 at $A = 1$. Likewise S5: $8.5 \cdot 10^{-3}$ vs. 0.84 (the quadratic term is $\propto A^2$, the linear part $\propto A$, so their ratio is $\propto A$).

Step 5: *Conclusion.* Every nonlinear system looks linear for small enough signals (small-signal linearisation). A test at small amplitude alone with a coarse tolerance could miss the nonlinearity; the large amplitude reveals it.

b) Shift test.

Step 1: Build $x_s[n] = x[n-k]$ (line 90) and compare $S\{x_s\}[n]$ with $y[n-k]$ for $n \geq k$.

Step 2: *Why zero guard samples.* The arrays have finite length N, with $x[n] = 0$ outside. Shifting a signal that is non-zero up to the end pushes samples out of the array, and zeros are pushed in at the start. A non-causal LTI system such as S4 (reads $x[n+1]$) would see this boundary difference and be wrongly classified as time-variant. With 8 zeros at both ends (line 89), shifts up to 4 lose nothing.

Step 3: *Result.* Only S1 fails (deviation 1.96): shifting the input by one sample changes *which* samples are held.

c) Causality.

Step 1: Two inputs that are equal for $n < n_0$ and different (random) from n_0 on.

Step 2: A causal system must give equal outputs for $n < n_0$.

Step 3: Only S4 reacts before the inputs differ: deviation 0.46 at $n = n_0 - 1$, caused by the weight 0.25 of $x[n+1]$.

d) Memory.

Step 1: Two inputs that differ only at n_0 (by 1).

Step 2: A memoryless system may change its output only at n_0.

Step 3: *Why several n_0.* For S1 a change at n_0 with $n_0 \bmod 3 \neq 0$ has *no* effect at all (the sample is never read); with $n_0 \bmod 3 = 0$ it affects $n_0 \ldots n_0 + 2$. A single unlucky n_0 would report "memoryless". The program therefore uses $n_0 = 20, 21, 22$.

Step 4: *General lesson.* A test can only falsify a property. A passed test is evidence, not proof – use many random trials.

e) Impulse responses of the LTI systems.

Step 1: *S2.* The plot for $n_0 = 30$ shows 0 at $n = 30$ and then 0.25, 0.1875, 0.141, ... from $n = 31$ on. Each value is 0.75× the previous one: $h[n] = 0.25 \cdot 0.75^{n-1}$ for $n \geq 1$, $h[n] = 0$ for $n \leq 0$.

Step 2: A geometric sequence comes from a recursion $y[n] = 0.75\,y[n-1] + \ldots$; the first value 0.25 one sample late gives the input term $0.25\,x[n-1]$: $y[n] = 0.75y[n-1] + 0.25x[n-1]$, an exponential smoother with one sample delay.

Step 3: *S4.* The plot for $n_0 = 30$ shows 0.25, 0.5, 0.25 at $n = 29, 30, 31$: $h[-1] = 0.25$, $h[0] = 0.5$, $h[1] = 0.25$, i.e. $y[n] = 0.25x[n+1] + 0.5x[n] + 0.25x[n-1]$, a centred, non-causal smoother.

Step 4: *The other systems.* Their response to an impulse depends on *where* the impulse is (S1: three ones for $n_0 = 30$, nothing for $n_0 = 31$) or on *how large* it is and which other samples are present (S3, S5). The formula $y = h * x$ (Chapter 3) requires linearity *and* time invariance; otherwise one impulse response does not describe the system.

Exercise 2.B2 The engine as a dynamic system

JSLINUX

Program: `code/jslinux/sheet02/engine_dyn.c`.

a) *Crankshaft inertia.* With fuel quantity q (1 = nominal), the engine speed obeys $\tau\,\dot{n} = Kq - n$, $\tau = 0.4\,\text{s}$, $K = 2000\,\text{rpm}$. Simulate a fuel step $1 \to 1.5$ at $t = 0.5\,\text{s}$ with the Euler method ($T = 1\,\text{ms}$) and with the exact discretisation $n[k] = a\,n[k-1] + (1-a)Kq[k-1]$, $a = \mathrm{e}^{-T/\tau}$ (derive it!). Determine τ from the 63 % point and compare both methods with the analytic solution at $t = 0.6\,\text{s}$. Is the engine (fuel → speed) static or dynamic?

b) *Lambda path.* Implement `lp_step()` (one step = 1 ms) – it is the model inside the Wokwi chip `lambda-probe`: $q \to$ wall film (first-order lag, 20 ms) $\to \lambda = (1 + a_{\text{air}})/q_f \to$ transport delay T_d $\to$ sensor lag (30 ms) $\to U = 0.5 + 0.4\tanh((1-\lambda)/0.005)$. Classify each block (static/dynamic, linear, TI). Simulate fuel steps $1.00 \to 1.10$ and $1.00 \to 1.20$ ($a_{\text{air}} = +5\,\%$, $T_d = 200\,\text{ms}$) and measure the time until U crosses 0.45 V. Which observations prove that the chain is nonlinear? Under which operating condition is it not time-invariant on a real engine?

c) *Closed loop – offline model of Exercise 2.2.* A two-point controller with integrator changes q with slope $-r$ while the sensor reports rich ($U > 0.45\,\text{V}$) and $+r$ while lean (period 10 ms, 8-bit PWM). Derive the period and the λ amplitude of the resulting oscillation as functions of T_d, r and the lag time constants (hint: a ramp passing a first-order lag τ is delayed by τ). Implement the controller in `closed_loop()`, compare with your prediction for $T_d = 50 \ldots 800\,\text{ms}$, and explain what happens to the mean of q when the unmetered air changes. Extend the prediction to a controller with an additional jump $\pm P$ at every switching of the sensor.

Solution

a) Crankshaft inertia.

Step 1: *Euler.* Replace $\dot{n}$ by $(n[k] - n[k-1])/T$ and evaluate the right-hand side at $k-1$: $n[k] = n[k-1] + \frac{T}{\tau}(Kq[k-1] - n[k-1]) = (1 - \frac{T}{\tau})n[k-1] + \frac{T}{\tau}Kq[k-1]$.

Step 2: *Exact discretisation.* Assume q is constant over one step, $q(t) = q[k-1]$ for $t \in [(k-1)T, kT)$. Then the ODE has the constant input $Kq[k-1]$, and its solution relaxes exponentially from the start value towards $Kq[k-1]$:

$$n(t) = Kq[k-1] + \left(n((k-1)T) - Kq[k-1]\right)\mathrm{e}^{-(t-(k-1)T)/\tau}.$$

Step 3: Evaluate at $t = kT$ and set $a = \mathrm{e}^{-T/\tau}$:

$$n[k] = a\,n[k-1] + (1-a)\,Kq[k-1].$$

With $T = 1\,\text{ms}$, $\tau = 0.4\,\text{s}$: $a = 0.997503$. Euler has $a = 1 - T/\tau = 0.9975$, the first-order Taylor approximation of $\mathrm{e}^{-T/\tau}$.

Step 4: *Simulate and measure τ.* The speed rises from $K \cdot 1 = 2000\,\text{rpm}$ to $K \cdot 1.5 = 3000\,\text{rpm}$. The 63 % point $2000 + 0.632 \cdot 1000 = 2632\,\text{rpm}$ is reached at $t = 0.900\,\text{s}$, i.e. 0.400 s after the step $\Rightarrow \tau = 0.400\,\text{s}$.

Step 5: *Compare at $t = 0.6\,\text{s}$.* Analytic: $2000 + 1000(1 - \mathrm{e}^{-0.1/0.4}) = 2221.20\,\text{rpm}$. Exact discretisation: 2221.20 rpm (identical, as derived). Euler: 2221.44 rpm, off by 0.24 rpm – negligible at $T/\tau = 1/400$.

Step 6: *Static or dynamic?* The speed after the step depends on how long ago the fuel changed, i.e. on past inputs ⇒ dynamic system (with memory), LTI in this model. The difference equation is the exponential smoother of Sheet 3.

Program output:

```
(a) fuel step 1.0 -> 1.5 at t = 0.5 s: n = 2000 -> 2998.1 rpm (t = 3 s)
    63% point reached at t = 0.900 s -> tau = 0.400 s
    n(0.6 s): Euler 2221.44, exact 2221.20, analytic 2221.20 rpm
    difference equation: n[k] = 0.997503 n[k-1] + 0.002497 * K q[k-1]
```

b) Lambda path. `lp_step()` executes the five blocks in signal order, one millisecond per call:

```
80 static double lp_step(lam_path_t *p, double q, double *lam_true)
81 {
82     double lam, lam_d;
83     p->qf += (q - p->qf) / 20.0;                        /* wall film, 20 ms  */
84     lam = (1.0 + p->air) / (p->qf > 0.05 ? p->qf : 0.05);   /* static, NL */
85     if (lam > 2.0) lam = 2.0;
86     if (lam < 0.5) lam = 0.5;
87     p->buf[p->w] = lam;                                  /* transport delay */
88     lam_d = p->buf[(p->w - p->delay + MAXD) % MAXD];
89     p->w = (p->w + 1) % MAXD;
90     p->ls += (lam_d - p->ls) / 30.0;                     /* sensor lag, 30 ms */
91     *lam_true = lam;
92     return es_lambda_voltage(p->ls);                     /* static, NL     */
93 }
```

Step 1: *Wall film* (line 83): first-order lag with $\tau = 20\,\text{ms}$, written as $q_f \mathrel{+}= (q - q_f)/20$ (Euler with $T/\tau = 1/20$) – dynamic, linear, TI.

Step 2: *Mixture* (line 84): $\lambda = (1 + a)/q_f$ – static, *nonlinear* (hyperbola).

Step 3: *Transport delay* (lines 87–89): ring buffer, read the value written T_d steps ago – dynamic (memory), linear, TI.

Step 4: *Sensor lag* (line 90): first-order lag, $\tau = 30\,\text{ms}$ – dynamic, linear, TI.

Step 5: *Switching characteristic* (line 92): static, strongly nonlinear (slope 80 V per unit λ at $\lambda = 1$, saturating at 0.1/0.9 V).

Response to the fuel step $1.00 \to 1.10$ ($a = 0.05$, $T_d = 200\,\text{ms}$), decomposed block by block:

Step 6: λ jumps from $1.05/1.00 = 1.05$ towards $1.05/1.10 = 0.9545$. It crosses 1 when $q_f = 1.05$, i.e. when the wall film has made half of its step: $1 - \mathrm{e}^{-t/20\,\text{ms}} = 0.5 \Rightarrow t = 20\,\text{ms} \cdot \ln 2 = 13.9\,\text{ms}$ (simulation: 13 ms).

Step 7: The transport delay adds 200 ms.

Step 8: The sensor lag must bring λ_s below the 0.45-V point, $0.5 + 0.4 \tanh \frac{1-\lambda_s}{0.005} = 0.45 \Rightarrow \lambda_s = 1 + 0.005\,\text{artanh}(0.125) = 1.0006$: another $\approx 27\,\text{ms}$. Total: 240 ms (simulation).

Step 9: *Double step* $1.00 \to 1.20$: λ crosses 1 when q_f has made only a quarter of its step: $20\,\text{ms} \cdot \ln \frac{4}{3} = 5.8\,\text{ms}$; and since the delayed λ now lies further below 1, the sensor lag also crosses earlier. Total: 223 ms.

```
(b) fuel step 1.00 -> 1.10 at t = 100 ms (air +5 %, Td = 200 ms)
    lambda 1.0500 -> 0.9545, crosses 1 after 13 ms (wall film)
    sensor crosses 0.45 V after 240 ms
    with step 1.00 -> 1.20 (double size): crosses after 223 ms, final U = 0.900 V (step 1.10:
    0.900 V)
```

Proofs of nonlinearity. (i) Doubling the input step does not double the output: the final value is 0.9 V in both cases. (ii) The response *time* depends on the step size (240 vs. 223 ms). An LTI system would only scale its step response. The sensor tells only "rich" or "lean", not *how* rich.

Time variance on a real engine. T_d is the travel time of the gas to the sensor, roughly $\propto$ 1/(mass flow). At changing speed or load T_d changes, so the system is time-variant.

c) Closed loop.

Step 1: *Lump the dynamics.* A ramp passing a first-order lag τ is delayed by τ. For ramps the chain therefore acts like one dead time $T_l = T_d + \tau_f + \tau_s = T_d + 50\,\text{ms}$. Let $q^* = 1 + a$.

Step 2: *Overshoot.* When q ramps up through q^*, the sensor still reports lean for T_l. At the switching $q = q^* + rT_l$.

Step 3: *Return.* The controller ramps down; q is back at q^* after T_l, and the sensor sees this another T_l later. At that switching $q = q^* - rT_l$.

Step 4: *Period and amplitude.* Each half period lasts $2T_l$:

$$T_{\text{period}} = 4\,T_l, \qquad q = q^* \pm rT_l, \qquad \lambda \approx 1 \mp \frac{rT_l}{q^*}.$$

Step 5: *With jump $P < rT_l$.* After the switching q is only $rT_l - P$ away from q^* and reaches it after $T_l - P/r$. The half period shortens by P/r: $T_{\text{period}} = 4T_l - 2P/r$. The peak deviation stays rT_l (it builds up before the switching).

Implementation (`closed_loop()`, controller every 10 steps):

```
if (k % 10 == 0) {                              /* controller, 10 ms */
    int rich = u > 0.45;
    if (rich != rich_old) q += rich ? -pjump : pjump;
    q += (rich ? -rate : rate) * 0.010;
    qpwm = 2.0 * floor(255.0 * q / 2.0 + 0.5) / 255.0;   /* 8-bit PWM */
    rich_old = rich;
}
u = lp_step(&p, qpwm, &lam);
```

Simulation ($r = 0.2/\text{s}$, $+5\,\%$ air, $T_l = T_d + 50\,\text{ms}$):

T_d / ms	period (sim.)	$4T_l$	λ range (sim.)
50	0.38 s	0.40 s	0.985…1.017
100	0.58 s	0.60 s	0.977…1.028
200	0.98 s	1.00 s	0.958…1.046
400	1.78 s	1.80 s	0.924…1.091
800	3.38 s	3.40 s	0.864…1.193

Step 6: *Compare.* The prediction fits within 2–5 %. The simulated periods are 20 ms shorter. This comes from the 8-bit PWM: q changes in stairs of $2/255 = 0.0078$, not as a smooth ramp. With unquantised q the same model gives exactly $4T_l$ (0.60, 1.00, 1.80, 3.40 s). The offset of the sensor threshold ($\lambda_s = 1.0006$) does not matter: it makes one switching earlier and the next one later by the same amount.

Step 7: *Unmetered air* $0/+5/+10\,\%$: mean $q = 0.9989$ / 1.0495 / $1.0986 \approx 1 + a$, period unchanged (0.98 s). The integrator drives the *mean* mixture to $\lambda = 1$ whatever the disturbance (integral action).

Step 8: *With jump.* $P = 0.03$, $r = 0.2/\text{s}$: period 0.70 s (prediction $1.0 - 0.3 = 0.70\,\text{s}$), λ 0.958…1.045. $P = 0.025$, $r = 0.1/\text{s}$: period 0.58 s, λ 0.978…1.027 (prediction 0.5 s, $\pm 2.4\,\%$). Here $P \approx rT_l$, the jump lands almost on q^*, and the lagged signal approaches q^* only exponentially – the ramp rule of Step 1 no longer applies, so the period is longer than predicted. The jump allows a smaller ramp, i.e. a smaller amplitude at the same or a higher frequency.

Digital Signal Processing — Exercise Sheet 3

LTI Systems

Script: Chapter 3 (Linear Time-Invariant Systems) — **with solutions**

Learning goals

- Compute convolutions by hand (graphically and in tables) and in C; verify the algebraic properties of convolution numerically.
- Derive impulse and step responses of the two workhorse smoothers – moving average (FIR) and exponential smoother (IIR) – from their difference equations.
- Quantify the noise-versus-lag trade-off of a smoother (noise gain, mean delay, ramp lag) and relate it to the step response.
- Apply LTI filters to the per-tooth engine speed in real time and understand why the missing-tooth gap must be handled *before* the filter (time invariance in the tooth domain).

Setup: C programs on the simulated ESP32

Exercises 3.2 and 3.3 a) are plain C programs without any circuit: they compute, print tables and draw ASCII plots. They run in Wokwi on an ESP32 that is used as a small C computer – no second environment is needed.

1. Open the project folder `sheet03_conv` (Exercise 3.2) or `sheet03_rpmoffline` (Exercise 3.3 a) in `code/wokwi_vscode/` in VS Code (or open the workspace `sheet03.code-workspace` there, which contains all projects of this sheet). The program is in `src/sketch.ino`; the headers it needs (`asciiplot.h`, `engine_signals.h`) are copies in `src/`.
2. Complete the `TODO`s, build (*PlatformIO: Build*) and start the simulator (`F1` → *Wokwi: Start Simulator*). The program runs once in `setup()` and writes everything to the Serial Monitor; the last line is `done.` Copy the results from there.
3. Why the ESP32 and not the Uno: the programs need tens of kilobytes of RAM (the Uno has 2 KB), and on the Uno `double` is only a 32-bit `float` – the rounding errors of Exercise 3.2 b) would be those of single precision. The ESP32 computes `double` in software: correct, but slow, so Exercise 3.3 a) needs a while in the simulator.
4. The first ESP32 build downloads the toolchain, about 1 GB (see the README in `code/wokwi_vscode/`). In the browser the same projects are in `code/wokwi/`: create an ESP32 project and add the header files as extra tabs.

Background

A discrete-time LTI system is completely described by its impulse response $h[n] = S\{\delta[n]\}$; the output for any input is the convolution

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k].$$

Two smoothers are used throughout the course (written for the per-tooth speed $r[k]$, but valid for any sequence):

$$\text{MA (FIR):}\quad y[n] = \frac{1}{N}\sum_{k=0}^{N-1} x[n-k] = y[n-1] + \frac{x[n]-x[n-N]}{N},$$

$$\text{EMA (IIR):}\quad y[n] = a\,y[n-1] + (1-a)\,x[n] = y[n-1] + \alpha\,(x[n]-y[n-1]), \qquad \alpha = 1-a,$$

called moving average and exponential smoother (exponential moving average). Useful characteristics of a smoother with $\sum_n h[n] = 1$: the *noise gain* $\sum_n h^2[n]$ (variance reduction for white noise) and the *mean delay* $D = \sum_n n\,h[n]$.

Exercise 3.1 Convolution and smoothers by hand

PAPER

a) Compute $y = x * h$ for $x[n] = \{\underline{1}, 2, 0, -1\}$ and $h[n] = \{\underline{1}, -1, 0.5\}$ (underlined: $n = 0$) twice: graphically (flip and shift h) and with a table of products $x[k]h[n-k]$. Check the result with $\sum_n y[n] = \sum_n x[n] \cdot \sum_n h[n]$ and explain why this identity holds.

b) Determine impulse response $h[n]$ and step response $s[n]$ of the MA of length N and of the EMA (from the difference equation by iteration and from $s[n] = \sum_{k\le n} h[k]$). After how many samples does each reach 50 % and 90 % of a step for $N = 16$ and $a = 15/17$?

c) Compute noise gain and mean delay D of both smoothers. Show that for a ramp input $x[n] = c\,n$ every smoother with $\sum h = 1$ has the steady-state output $y[n] = c\,(n - D)$, and that $D = \sum_{n\ge 0}(1 - s[n])$, i.e. the ramp lag can be read off the step response. Which a gives the EMA the same noise gain as the MA of length N? Compare the mean delays.

d) Show that $x[n] = \mathrm{e}^{\mathrm{j}\Omega n}$ is an eigenfunction of every LTI system, $y[n] = H(\Omega)\,x[n]$ with $H(\Omega) = \sum_k h[k]\mathrm{e}^{-\mathrm{j}\Omega k}$. Evaluate $|H|$ for MA_{16}, MA_{30} and the EMA with $a = 15/17$ at $\Omega = 2\pi/30$. (On the 60-tooth wheel, sampled once per 6°, this is the 2nd engine order – the speed ripple caused by the combustions.)

Solution

a) Table ($y[n] = \sum_k x[k]h[n-k]$; rows k, columns n):

$x[k]\,h[n-k]$	$n=0$	1	2	3	4	5
$k=0,\ x=1$	1	−1	0.5			
$k=1,\ x=2$		2	−2	1		
$k=2,\ x=0$			0	0	0	
$k=3,\ x=-1$				−1	1	−0.5
$y[n]$	1	1	−1.5	0	1	−0.5

Graphically: $h[n-k]$ is h flipped and moved to position n; $y[n]$ is the sum of the products where flipped h and x overlap. Length $4+3-1 = 6$. Check: $\sum y = 1 = (1+2+0-1)(1-1+0.5) = 2\cdot 0.5$. Reason: $\sum_n \sum_k x[k]h[n-k] = \sum_k x[k] \sum_m h[m]$ (substitute $m = n-k$). The program of Exercise 3.2 prints `y = 1 1 -1.5 0 1 -0.5`.

b) MA: $h[n] = 1/N$ for $0 \le n \le N-1$, else 0; $s[n] = (n+1)/N$ for $0 \le n < N$, $s[n] = 1$ for $n \ge N-1$ (linear ramp, finite). EMA: iterating $y[n] = ay[n-1] + (1-a)\delta[n]$ gives $h[n] = (1-a)a^n u[n]$; $s[n] = (1-a)\sum_{k=0}^{n} a^k = 1 - a^{n+1}$ (exponential, never exactly 1). For $N = 16$: MA reaches 50 % at $n = 7$ (8/16), 90 % at $n = 14$ (15/16; $14/16 < 0.9$). EMA, $a = 15/17$: $a^{n+1} \le 0.5 \Leftrightarrow n+1 \ge \ln 0.5/\ln a = 5.55$, i.e. $n = 5$; 90 %: $n+1 \ge \ln 0.1/\ln a = 18.4$, $n = 18$. The EMA reacts faster initially but needs longer to settle.

c) MA: $\sum h^2 = N \cdot \frac{1}{N^2} = \frac{1}{N}$, $D = \frac{1}{N}\sum_{n=0}^{N-1} n = \frac{N-1}{2}$. EMA: $\sum h^2 = (1-a)^2 \sum a^{2n} = \frac{(1-a)^2}{1-a^2} = \frac{1-a}{1+a}$, $D = (1-a)\sum na^n = \frac{a}{1-a}$. Ramp: $y[n] = \sum_k h[k]\,c(n-k) = c\,n\sum_k h[k] - c\sum_k kh[k] = c(n-D)$ (for n beyond the length of h; for the EMA asymptotically). With $1-s[n] = \sum_{k>n} h[k]$: $\sum_{n\ge 0}(1-s[n]) = \sum_k k\,h[k] = D$. Equal noise: $\frac{1-a}{1+a} = \frac{1}{N} \Rightarrow a = \frac{N-1}{N+1}$, $\alpha = \frac{2}{N+1}$ (the usual "N-

period EMA"). Then $D_{\mathrm{EMA}} = \frac{(N-1)/(N+1)}{2/(N+1)} = \frac{N-1}{2} = D_{\mathrm{MA}}$: *for equal white-noise reduction both smoothers have exactly the same ramp lag.* They differ in the shape of the step response (b) and in how they suppress periodic disturbances (d). For $N = 16$: $a = 15/17$, $D = 7.5$ samples.
d) $y[n] = \sum_k h[k]\mathrm{e}^{\mathrm{j}\Omega(n-k)} = \mathrm{e}^{\mathrm{j}\Omega n}\sum_k h[k]\mathrm{e}^{-\mathrm{j}\Omega k}$. MA: geometric sum, $|H| = \left|\frac{\sin(N\Omega/2)}{N\sin(\Omega/2)}\right|$; EMA: $H = \frac{1-a}{1-a\mathrm{e}^{-\mathrm{j}\Omega}}$. At $\Omega = 2\pi/30$: MA_{16}: $\frac{\sin(16\pi/30)}{16\sin(\pi/30)} = \frac{0.9945}{1.6724} = 0.595$; MA_{30}: $\sin(\pi) = 0 \Rightarrow |H| = 0$ – the MA over one full period removes the ripple *completely* (the sum over a period of a zero-mean periodic signal vanishes); EMA: $\frac{0.1176}{|1-0.8824(0.9781-0.2079\mathrm{j})|} = \frac{0.1176}{0.2289} = 0.514$. These values are confirmed by the simulation in Exercise 3.3 a).

Exercise 3.2 Convolution engine

WOKWI

Program: project `sheet03_conv` (ESP32 without circuit, see Setup).

a) Implement `conv(x, nx, h, nh, y)` (full linear convolution, output length $n_x + n_h - 1$, no out-of-range accesses) and check it with the example of Exercise 3.1 a).

b) Verify commutativity, associativity and distributivity with random sequences of lengths 200, 50 and 30. Why are the deviations not exactly zero? Why might they be exactly zero for "nice" test numbers?

c) Implement the exponential smoother recursively (`exp_smooth()`) and compare it with the convolution by its impulse response truncated to L samples ($a = 0.9$, $L = 10 \ldots 160$). Relate the error to a^L and compare the number of multiplications per output sample. Verify the step response formula of Exercise 3.1 b).

d) Convolve the impulse response of an 8-point MA with itself. Explain the shape and the mean delay of the result and compare the step responses of one MA and of the cascade.

Solution

a) Key part of `code/solutions/wokwi/sheet03_conv/sketch.ino` – the summation limits make sure that both $x[k]$ and $h[n-k]$ exist:

```
static int conv(const double *x, int nx, const double *h, int nh, double *y)
{
    int n, k, ny = nx + nh - 1;
    for (n = 0; n < ny; n++) {
        int k0 = n - nh + 1 > 0 ? n - nh + 1 : 0;     /* h index n-k < nh  */
        int k1 = n < nx - 1 ? n : nx - 1;             /* x index k < nx    */
        double s = 0.0;
        for (k = k0; k <= k1; k++) s += x[k] * h[n - k];
        y[n] = s;
    }
    return ny;
}
```

Output: `y = 1 1 -1.5 0 1 -0.5` (as by hand).
b), c) Output:

```
(b) commutativity   max|x*h - h*x|             = 2.66e-15
    associativity   max|(x*h)*g - x*(h*g)|     = 1.07e-14
    distributivity  max|x*(h+g) - (x*h + x*g)| = 5.33e-15

(c) exponential smoother a = 0.90, 1000 samples
     L   max|recursive - conv|   a^L      mult./sample
    10   3.774e-01           3.487e-01   10 (recursion: 2)
    20   1.316e-01           1.216e-01   20 (recursion: 2)
    40   1.600e-02           1.478e-02   40 (recursion: 2)
    80   2.365e-04           2.185e-04   80 (recursion: 2)
   160   5.166e-08           4.773e-08   160 (recursion: 2)
    step response vs. 1 - a^(n+1): max deviation 1.11e-16
```

The properties hold exactly in real arithmetic; in floating point the sums are evaluated in a different order and rounded differently (deviations of a few 10^{-15}, i.e. some units of the last place relative to values of order 10). With test numbers that are short binary fractions (e.g. multiples of 2^{-15}) all products and sums are exact and the deviation is exactly 0 – a test that proves nothing about rounding. (c): The truncated impulse response misses the tail $\sum_{n\geq L} h[n] = a^L$; for an input of magnitude ≈ 1.1 (step + noise) the error is $\approx 1.1\,a^L$, as observed. The recursion needs 2 multiplications per sample for an *infinite* impulse response, the convolution L – the reason why IIR filters are cheap. The recursive step response matches $1 - a^{n+1}$ to rounding precision.
d) $h_{\mathrm{MA8}} * h_{\mathrm{MA8}} = \frac{1}{64}\{1, 2, \ldots, 7, 8, 7, \ldots, 2, 1\}$ (length 15): the overlap of two rectangles of width 8 grows linearly and shrinks again – a triangle. Sum 1, mean delay $7 = 2 \cdot 3.5$ (mean delays add under convolution, like the means of independent random variables). The step response of the cascade is a smooth S-curve (piecewise parabolic) reaching 1 after 15 samples instead of the linear ramp of one MA reaching 1 after 8 samples: better high-frequency suppression, twice the delay.

Exercise 3.3 Smoothing the RPM estimate

WOKWI

The per-tooth speed $r[k] = 1/T_k$ of Sheet 1 contains the real speed fluctuation (2nd order, $\pm 1\,\%$) and timer quantisation. We smooth it with an MA and an EMA in the *tooth domain* (one sample per tooth). Three ways to treat the gap (period $\approx 3T$) are compared: `GAP_MODE` 0 – use $1/T_{\mathrm{gap}}$ like a normal value; 1 – correct it to $3/T_{\mathrm{gap}}$ (one sample); 2 – insert $3/T_{\mathrm{gap}}$ three times (one sample per 6°).

a) *Offline prediction* (ESP32 without circuit, see Setup; project `sheet03_rpmoffline`). The program simulates the crank edges of the virtual engine with 0.5 µs time stamps. Implement the gap handling, `moving_average()` (running sum) and `exp_smoother()`. Report: at constant 3000 rpm mean, standard deviation and min/max of raw, MA_{16}, MA_{30}, $\mathrm{EMA}_{2/17}$ for all three gap modes; for a step 3000→5000 rpm the 50 %/90 % times in samples for $N = 8, 16, 32$ and $\alpha = 2/(N+1)$; for a ramp of 3000 rpm/s the lag error in rpm, in samples and in ms. Compare with Exercise 3.1.

b) *Real time* (WOKWI, `code/wokwi/sheet03_rpmfilter/`). The ISR pushes tooth periods into a FIFO; `loop()` computes and filters the speed. Implement the same three steps in the sketch. It prints one line per revolution and, whenever the raw speed jumps by more than 150 rpm between two teeth, a capture block of 128 samples (32 before the jump). Start at 3000 rpm and *click* the speed slider at 5000 rpm. Determine the 50 %/90 % points of both filters from the capture and the ramp lag $D = \sum(1 - s[n])$. (Dragging the slider produces a staircase of steps, not a ramp – why is the step response sufficient anyway?)

c) Repeat with `GAP_MODE` 0 and 1. Explain the per-revolution values and the dips in the capture. Why does the gap violate the time invariance of the tooth-domain processing, and why is only mode 2 compatible with the exact ripple cancellation of MA_{30}?

d) Design: the injection timing needs the speed during an acceleration of 5000 rpm/s at 1000 rpm with an error below 0.5 %, while the 2nd-order ripple should be suppressed. Is this possible with an MA in the tooth domain? What do real ECUs do (keyword: *segment time*)?

Solution

a) Key parts of `code/solutions/wokwi/sheet03_rpmoffline/sketch.ino`:

```
/* ---- (a) per-tooth speed, gap handling ---------------- */
v = 1.0 / (T * TICK);                    /* rpm = 1/T[s] */
if (gap && gap_mode != GAP_NAIVE) v *= 3.0;
if (gap && gap_mode == GAP_THREE) rep = 3;
```

```c
/* moving average over N samples, running sum; the first N-1 outputs
   use the first sample as "history" (filter pre-loaded, no start-up)  */
static void moving_average(const float *x, int n, int N, float *y)
{
    double s = N * x[0];
    int k;
    for (k = 0; k < n; k++) {
        s += x[k] - (k >= N ? x[k - N] : x[0]);
        y[k] = s / N;
    }
}

/* exponential smoother y[k] = y[k-1] + alpha (x[k] - y[k-1]),
   pre-loaded with x[0]                                          */
static void exp_smoother(const float *x, int n, double alpha, float *y)
{
    double s = x[0];
    int k;
    for (k = 0; k < n; k++) {
        s += alpha * (x[k] - s);
        y[k] = s;
    }
}
```

Results (constant 3000 rpm, after settling):

```
gap handling: naive gap (2842 samples)
 raw        mean  2966.4  std 260.20  min   996.5  max  3030.3
 MA16       mean  2966.4  std  58.10  min  2859.7  max  3017.8
 MA30       mean  2966.3  std  32.88  min  2932.2  max  3000.0
 EMA 2/17   mean  2966.4  std  57.60  min  2750.7  max  3014.3
gap handling: gap once (2842 samples)
 raw        mean  3000.3  std  21.56  min  2967.4  max  3030.3
 MA16       mean  3000.3  std  12.22  min  2982.0  max  3017.8
 MA30       mean  3000.4  std   1.07  min  2998.7  max  3002.7
 EMA 2/17   mean  3000.3  std  10.75  min  2984.8  max  3015.5
gap handling: gap x3 (2940 samples)
 raw        mean  2999.9  std  21.30  min  2967.4  max  3030.3
 MA16       mean  3000.0  std  12.57  min  2982.0  max  3017.8
 MA30       mean  3000.0  std   0.04  min  2999.9  max  3000.2
 EMA 2/17   mean  3000.0  std  10.87  min  2984.8  max  3015.5
```

With correct gap handling the raw std (21.3 rpm) is the ripple ($30/\sqrt{2}$). The attenuation factors $12.57/21.30 = 0.59$ (MA_{16}) and $10.87/21.30 = 0.51$ (EMA) agree with $|H| = 0.595$ and 0.514 of Exercise 3.1 d); MA_{30} in mode 2 leaves 0.04 rpm (timer quantisation only). Step and ramp (mode 2):

```
 MA8        50% after   4 samples, 90% after   7 samples
 EMA 2/9    50% after   2 samples, 90% after  10 samples
 MA16       50% after   8 samples, 90% after  14 samples
 EMA 2/17   50% after   5 samples, 90% after  18 samples
 MA32       50% after  16 samples, 90% after  28 samples
 EMA 2/33   50% after  11 samples, 90% after  37 samples

 MA16       lag error    6.8 rpm =  7.93 samples =  2.25 ms at 3523 rpm
 EMA 2/17   lag error    6.9 rpm =  7.98 samples =  2.27 ms at 3523 rpm
 MA32       lag error   13.8 rpm = 16.07 samples =  4.56 ms at 3523 rpm
 EMA 2/33   lag error   13.8 rpm = 16.04 samples =  4.55 ms at 3523 rpm
```

The step points match 3.1 b) (MA_{16} is exactly at 50 % at $n = 7$; the ripple decides whether 7 or 8 is reported). The ramp lag is $D + 0.5$: $(N-1)/2$ from the filter plus half a sample because $1/T_k$ is the mean speed over the tooth, i.e. the speed half a tooth *before* the edge. MA and EMA with $\alpha = 2/(N+1)$ have the same lag, as predicted in 3.1 c).

b) Sketch (`code/solutions/wokwi/sheet03_rpmfilter/sketch.ino`), filter and gap handling:

```
void process(float r, bool gap)
{
    int16_t ri = (int16_t)(r + 0.5f);
    if (!filt_init) {                          /* pre-load with first value */
        for (uint8_t i = 0; i < N_MA; i++) ma_buf[i] = ri;
        ma_sum = (int32_t)ri * N_MA;
        ema = r;
        filt_init = true;
    }
    /* moving average: running sum, add newest, subtract oldest */
    ma_sum += ri - ma_buf[ma_idx];
    ma_buf[ma_idx] = ri;
    if (++ma_idx >= N_MA) ma_idx = 0;
    float ma = (float)ma_sum / N_MA;
    /* exponential smoother */
    ema += ALPHA * (r - ema);
```

```
        /* per-tooth speed: rpm = 1 / T[s] = 2e6 / T[ticks] */
        float r = 2.0e6f / T;
        uint8_t rep = 1;
#if GAP_MODE >= 1
        if (gap) r *= 3.0f;                    /* gap spans 18 deg */
#endif
#if GAP_MODE == 2
        if (gap) rep = 3;                      /* one sample per 6 deg */
#endif
        for (uint8_t i = 0; i < rep; i++) process(r, gap);
```

Expected capture (reference sketch run on the PC with simulated crank edges; step 3000→5000 rpm at $k = 0$):

```
k,raw,ma,ema,gap          k,raw,ma,ema,gap
-1,3012,3018,3014,0        7,4950,3994,4246,0
0,4988,3143,3247,0         8,4950,4114,4329,0
1,4988,3267,3451,0        14,4988,4846,4671,0
4,4963,3633,3926,0        15,5013,4971,4711,0
5,4950,3753,4046,0        18,5025,4978,4808,0
```

The MA rises linearly by $2000/16 = 125$ rpm per tooth and reaches the new value after 16 samples; the EMA jumps by $\alpha \cdot 2000 = 235$ rpm first and then approaches exponentially: 50 % after 5, 90 % after 18 samples (MA: 8 and 14). The ramp lag follows from the step response, because for an LTI system the response to a ramp is the running sum of the step response: $D = \sum_{k\geq 0}(1 - s[k])$ evaluated on the capture (baseline = mean of the 32 pre-trigger samples, final value = mean of the last 36) gives 7.65 (MA) and 7.56 (EMA) samples, close to 7.5; the extra half sample of the offline ramp (7.9) is hidden here because the time origin is the first *measured* jump. A staircase is a sum of shifted steps – by superposition its response is the sum of shifted step responses; nothing new is learned from it. Note: the per-revolution line samples the filters always at the same crank angle (at the gap), so the ripple appears there as a constant offset (e.g. MA_{16} 2982, EMA 2986 at 3000 rpm) – stroboscopic sampling of a crank-synchronous signal.

c) Mode 0: the gap value is $n/3$, i.e. a negative pulse of $-2n/3 = -2000$ rpm once per revolution. MA_{16} shows a rectangular dip of $2000/16 = 125$ rpm lasting 16 teeth (min 2860), the EMA a dip of $\alpha \cdot 2000 = 235$ rpm (min 2751) that decays over ≈ 20 teeth; the mean is biased by -1.1 % (compare 2.1 c). The per-revolution line, printed right after the gap, shows the worst value (3000 rpm: MA 2860, EMA 2751; 7000 rpm: 6674, 6418). Mode 1 removes the bias, but the sample covering 18° has the same weight as those covering 6°; MA_{30} then spans 30 samples $\neq 180°$ for half of the windows: residual std 1.07 rpm. Mode 2 makes the sample index proportional to the crank angle (6° per sample, 60 per revolution); MA_{30} spans exactly 180° and cancels the 2nd order completely (std 0.04). Time invariance: in the tooth domain the operator

"tooth period → speed" must be the same for every index k. With the gap it is not: at $k \equiv 57 \pmod{58}$ the measurement covers 18° instead of 6°, i.e. the system changes periodically with k (like S1 or S10 of Sheet 2). A shift of the speed profile by one tooth does not shift the output by one sample. An LTI filter after a time-variant measurement produces artefacts that no filter design can remove; the correction must restore a uniform (angle-equidistant) sequence first.
d) At 1000 rpm one sample takes 1 ms; 5000 rpm/s are 5 rpm per sample. $0.5\,\% = 5$ rpm allow a lag of only 1 sample, i.e. $(N-1)/2 + 0.5 \leq 1$, $N \leq 2$ – no ripple suppression at all ($|H| \approx 1$). Conversely MA_{30} removes the ripple but lags 15.5 samples = 15.5 ms, i.e. 78 rpm (7.8 %). A causal smoother cannot have both. Real ECUs therefore (i) measure the *segment time* (time for $720°/z = 180°$, i.e. exactly the MA_{30} in the period domain) for torque/misfire functions, synchronous to the firing, (ii) use the short tooth time for angle extrapolation between teeth, and (iii) compensate the known lag by prediction ($\hat{n} = n_{\mathrm{filt}} + \dot{n} \cdot D$, an estimate of the acceleration). Note also that the lag in *samples* is constant while the lag in *time* scales with $1/n$: at 6000 rpm the same MA_{30} lags only 2.6 ms.

Deliverables

3.1: convolution table, derivations of h, s, noise gain, D, $|H|$. 3.2: program and output, explanation of the rounding and truncation errors, plot of the triangular impulse response. 3.3: completed `sheet03_rpmoffline` sketch with result tables; sketch; capture of a step for all three gap modes (plot in any tool, e.g. a spreadsheet), 50/90 % points and ramp lag; answer to d).

Digital Signal Processing Exercise Sheet 4

Sampling

Script: Chapter 4 (Sampling and Quantisation) **with solutions**

Learning goals

- Derive and apply the quantisation-noise model and the rule SNR $\approx 6.02N + 1.76$ dB; size an ADC for the knock sensor.
- Configure timer-driven sampling on a microcontroller and verify the sampling rate and the conversion time with the logic analyzer.
- Quantify the effect of sampling-time jitter and compare it with quantisation noise.
- Resample a time-sampled engine signal onto a crank-angle grid using the edges of the 60-2 crank signal, and explain when the time domain and when the angle domain is the natural one.

Setup: C programs on the simulated ESP32

Exercises 4.3 and 4.4 are plain C programs without any circuit, like Exercises 3.2 and 3.3 a): they run in Wokwi on an ESP32 that is used as a small C computer.

1. In VS Code open the workspace `sheet04.code-workspace` in `code/wokwi_vscode/`. It contains the three projects of this sheet: `sheet04_adc` (Exercise 4.2, Arduino Uno), `sheet04_quant` (Exercise 4.3) and `sheet04_angle` (Exercise 4.4, both ESP32). The programs are in `src/sketch.ino`; the headers they need are copies in `src/`.
2. Complete the `TODO`s, build (*PlatformIO: Build*) and start the simulator (`F1` → *Wokwi: Start Simulator*; in the workspace choose the project's `wokwi.toml`). The program runs once in `setup()` and writes everything to the Serial Monitor; the last line is `done.`
3. The ESP32 has about 300 KB of RAM and computes `double` in software. The programs therefore use shorter records than a PC program would, store long sample arrays as `float` or 16-bit integers (on the heap), and need some seconds in the simulator.

Background

An ADC with N bits and full-scale range V_{FS} has the step size (1 LSB) $\Delta = V_{\mathrm{FS}}/2^N$. If the signal is "busy" (it moves over many steps between samples), the quantisation error $e = x_q - x$ is well modelled as white noise, uniformly distributed in $[-\Delta/2, \Delta/2)$. Sampling instants that deviate randomly from the ideal grid, $t_n = nT_s + \delta_n$ with $\mathrm{std}(\delta_n) = \sigma_j$ (*jitter*), produce an additional error $e_n \approx \dot{x}(nT_s)\,\delta_n$ that grows with the signal frequency.

An ECU samples two kinds of signals: *time-based* (fixed T_s, e.g. the knock sensor with 20 kHz to 50 kHz) and *angle-based* (one value per fixed crank-angle increment, triggered by the crank wheel). An angle grid can also be computed afterwards from time samples: the crank edges provide pairs (time, angle), between which the angle is interpolated.

Exercise 4.1 Quantisation and jitter budget PAPER

a) Assume a uniformly distributed quantisation error. Derive its variance and show that a full-scale sine quantised with N bits has SNR $= 6.02N + 1.76$ dB. By how much does the SNR drop if the sine uses only 1/10 of the full-scale amplitude?

b) Compute 1 LSB and the rms quantisation noise for the Arduino Uno (10 bit, 0 V to 5 V) and the ESP32 (12 bit, 0 V to 3.3 V).

c) The conditioned knock-sensor signal spans 0 V to 5 V (offset 2.5 V) for the heaviest knock. The weakest knock that still has to be measured has an amplitude of 10 mV; its quantisation SNR must be at least 30 dB. Which dynamic range (in dB) is required, and how many bits does the ADC need?

d) The knock detector only uses a band of width $B = 1\,\text{kHz}$ around the knock frequency, the ADC samples at $f_s = 50\,\text{kHz}$. Assuming white quantisation noise, by how many dB does a band-pass filter improve the SNR of (c)? How many ADC bits does this "save"?

e) Show that jitter limits the SNR of a sine of frequency f to $\text{SNR}_j = -20\log_{10}(2\pi f\sigma_j)$. (i) Which jitter is allowed for a 6.5 kHz knock signal if jitter shall not degrade a 12-bit ADC? (ii) A sampling loop that polls `micros()` (resolution 4 µs on the Uno) produces a timing error uniformly distributed over 4 µs. Compute σ_j, SNR_j at 6.5 kHz and the *effective number of bits* $\text{ENOB} = (\text{SNR} - 1.76)/6.02$.

Solution

a) For $e \sim \mathcal{U}[-\Delta/2, \Delta/2)$: $\sigma_e^2 = \frac{1}{\Delta}\int_{-\Delta/2}^{\Delta/2} e^2\,\mathrm{d}e = \frac{\Delta^2}{12}$. A full-scale sine has amplitude $A = 2^{N-1}\Delta$ and power $P_s = A^2/2 = 2^{2N}\Delta^2/8$. Hence

$$\text{SNR} = \frac{P_s}{\sigma_e^2} = \frac{3}{2}\,2^{2N} \;\Rightarrow\; 10\log_{10}\text{SNR} = 10\log_{10}1.5 + 20N\log_{10}2 = 1.76 + 6.02N\ \text{dB}.$$

The noise power does not depend on the signal, so an amplitude of 1/10 of full scale lowers the SNR by $20\log_{10}10 = 20\,\text{dB}$ (equivalent to losing 3.3 bits). Every unused part of the input range costs resolution.
b) Uno: $\Delta = 5/1024\,\text{V} = 4.88\,\text{mV}$, $\sigma_e = \Delta/\sqrt{12} = 1.41\,\text{mV}$. ESP32: $\Delta = 3.3/4096\,\text{V} = 0.806\,\text{mV}$, $\sigma_e = 0.233\,\text{mV}$.
c) Dynamic range: $20\log_{10}(2.5\,\text{V}/10\,\text{mV}) = 48\,\text{dB}$. A sine of amplitude a has $\text{SNR} = (a^2/2)/(\Delta^2/12) = 6a^2/\Delta^2$. $\text{SNR} \geq 10^3$ requires $\Delta \leq a\sqrt{6\cdot10^{-3}} = 0.775\,\text{mV}$, i.e. $2^N \geq 5\,\text{V}/0.775\,\text{mV} = 6455$, so $N = 13$ bits (a 12-bit ADC gives $\Delta = 1.22\,\text{mV}$ and only 26 dB). Equivalently: $48 + 30 = 78\,\text{dB}$ of "full-scale SNR" $\Rightarrow N \geq (78 - 1.76)/6.02 = 12.7$.
d) White quantisation noise is spread evenly over $[0, f_s/2]$; a filter of bandwidth B keeps the fraction $2B/f_s = 1/25$. The SNR improves by $10\log_{10}(f_s/2B) = 14\,\text{dB}$, i.e. 14/6.02 ≈ 2.3 bits: with the band-pass a 12-bit ADC reaches $26 + 14 = 40\,\text{dB}$ and even 11 bits (≈ 34 dB) would suffice. (Oversampling and filtering trade sampling rate for resolution: 3 dB, i.e. half a bit, per doubling of f_s.)
e) For $x = A\sin\omega t$: $e_n \approx A\omega\cos(\omega n T_s)\,\delta_n$ with independent δ_n, so $\overline{e^2} = A^2\omega^2\sigma_j^2/2$ and $\text{SNR}_j = (A^2/2)/\overline{e^2} = 1/(\omega\sigma_j)^2$, i.e. $\text{SNR}_j = -20\log_{10}(2\pi f\sigma_j)$. (i) 12 bit: $\text{SNR}_q = 74.0\,\text{dB}$; $2\pi f\sigma_j \leq 10^{-74/20} = 2.0\cdot10^{-4}$ gives $\sigma_j \leq 4.9\,\text{ns}$ at 6.5 kHz – only hardware-triggered conversions achieve this. (ii) Uniform over 4 µs: $\sigma_j = 4\,\mu\text{s}/\sqrt{12} = 1.15\,\mu\text{s}$, $\text{SNR}_j = -20\log_{10}(2\pi\cdot6500\cdot1.15\cdot10^{-6}) = 26.5\,\text{dB}$, $\text{ENOB} = 4.1$ bits. A software-polled sampling loop turns a 10-bit ADC into a 4-bit ADC for knock signals!

Exercise 4.2 ADC in practice: timer-driven sampling

WOKWI

Open the Uno project `code/wokwi/sheet04_adc/` (sketch, diagram and the `engine-sim` chip files). Timer1 triggers an interrupt service routine (ISR) at $f_s = 5\,\text{kHz}$ that samples the `KNOCK` output of the chip (A1) into a RAM buffer of 400 samples; the buffer is printed afterwards (the UART cannot transmit 5 kHz × a CSV line in real time). Pin 8 toggles at every sample (logic analyzer D0), pin 9 is HIGH while the ISR runs (D1), the chip's `TDC` output (pin 2, INT0) marks combustion TDCs.

a) Timer1 runs in CTC mode with prescaler 8. Compute the compare value `OCR1A` for 5 kHz and complete `timer1_start()`. What is the lowest sampling rate possible with this prescaler?

b) Complete the ISR. Record about 0.1 s with the logic analyzer and determine from the VCD

file (i) the sampling period (D0) and (ii) the time the ISR needs (pulse width on D1). Which sampling rate is the theoretical maximum for this structure? Why is the practical limit lower?

c) Between combustions, 140° to 175° after each TDC, the knock signal contains only sensor noise. Complete the computation of the index range of this quiet window (at 3000 rpm) and run the sketch with the noise slider at 50 mV, 10 mV and 0 mV. Compare the printed standard deviation (in LSB) with your expectation (chip output = 2.5 V + 0.6· sensor signal). Would the 12-bit ESP32 ADC improve the knock measurement here?

d) Turn the potentiometer to an arbitrary position and read the code several times. Is the quantisation error of this constant input "noise"? What does this mean for the validity of the model of Exercise 4.1?

Wokwi measures time in *simulated* time; the Uno core is simulated instruction by instruction, but treat the measured durations as indicative.

Solution

a) Timer clock $f_T = 16\,\text{MHz}/8 = 2\,\text{MHz}$; the period is $(\texttt{OCR1A}+1)/f_T$, so $\texttt{OCR1A} = 2\cdot10^6/5000 - 1 = 399$. The 16-bit register allows at most $65536/f_T = 32.8\,\text{ms}$, i.e. $f_{s,\min} = 30.5\,\text{Hz}$ (prescaler 64/256/1024 for slower rates).

```
/* ---------------- Timer1 compare-match interrupt: one sample ---------- */
ISR(TIMER1_COMPA_vect)
{
    PORTB |= _BV(PB1);                  /* pin 9 HIGH: ISR busy            */
    PINB = _BV(PB0);                    /* writing 1 to PINx toggles pin 8 */
    uint16_t v = analogRead(A1);        /* ~112 us on the Uno              */
    if (tdc_seen) { v |= 0x8000; tdc_seen = 0; }
    buf[nbuf++] = v;
    if (nbuf >= NBUF) { TIMSK1 = 0; done = true; }   /* block complete     */
    PORTB &= ~_BV(PB1);                 /* pin 9 LOW                       */
}

/* ---------------- (a) Timer1: CTC mode, prescaler 8 ------------------- */
void timer1_start()
{
    noInterrupts();
    TCCR1A = 0;                         /* no PWM outputs                  */
    TCCR1B = _BV(WGM12) | _BV(CS11);    /* CTC (TOP = OCR1A), clk/8 = 2 MHz */
    OCR1A  = F_CPU / 8 / FS_HZ - 1;     /* 16e6/8/5000 - 1 = 399           */
    TCNT1  = 0;
    TIFR1  = _BV(OCF1A);                /* clear a pending flag            */
    TIMSK1 = _BV(OCIE1A);               /* enable compare-match interrupt  */
    interrupts();
}
```

b) D0 changes level every 200 µs (square wave with period 400 µs). The D1 pulses last about 110 µs: the conversion itself takes 13 ADC clock cycles at 16 MHz/128 = 125 kHz, i.e. 104 µs, plus the overhead of `analogRead()` and the ISR entry/exit. Theoretical maximum: $1/112\,\mu\text{s} \approx 8.9\,\text{kHz}$ – then the CPU does nothing but sampling. In practice other interrupts (Timer0 for `millis()`, INT0, UART) must fit in between and the main program needs CPU time, so about 5 kHz to 7 kHz is a sensible limit with `analogRead()`. Single edges on D0 may be delayed by a few µs when the Timer0 ISR is running at the compare match: this is jitter (it can be removed by disabling Timer0, see Sheet 5).

c) Samples per degree: $f_s/(6\,n) = 5000/18000 = 0.278$, so $i_0 = \lceil 140 \cdot 0.278 \rceil = 39$, $i_1 = \lfloor 175 \cdot 0.278 \rfloor = 48$ (10 samples per TDC, about 70 per block). Expected standard deviation:

noise slider	sensor noise at ADC	expected std	simulation (3990 samples)
50 mV	30 mV	6.15 LSB	6.06 LSB
10 mV	6 mV	1.26 LSB	1.25 LSB
0 mV	0	0	0.46 LSB

("expected" = $\sqrt{(0.6\sigma_{\text{noise}}/\Delta)^2 + 1/12}$; the simulation column was obtained with the same engine model sampled exactly like the sketch. A single block contains only ≈ 70 samples, so the printed value scatters by about $\pm 9\,\%$.) The quantisation noise (0.29 LSB) is negligible against the sensor noise ($\sqrt{6.14^2 + 0.29^2} = 6.15$, i.e. 0.01 dB worse): the measurement is limited by the *sensor*, not by the ADC; a 12-bit ADC would not help. Only for the noise-free signal does quantisation dominate: the offset 2.5 V lies exactly on the transition between codes 511 and 512, so tiny residues of the decaying bursts make the LSB flicker (std ≈ 0.5 LSB in the simulation; in Wokwi the result depends on how the simulator rounds and may be exactly 0).

d) The code is constant, e.g. 3.30 V gives $\lfloor 3.30 \cdot 1024/5 \rfloor = 675$ every time. The error is a fixed, signal dependent offset (up to 1 LSB), not a random variable: the noise model of 4.1 requires a signal that changes over many LSB between samples (or added noise/dither). For slowly varying signals the quantisation error is correlated and appears as distortion, not as white noise.

Exercise 4.3 Quantisation noise and jitter

WOKWI

The ESP32 program `code/wokwi_vscode/sheet04_quant/` generates a sine of $f_0 = 1234.5$ Hz at $f_s = 50$ kHz (10^4 samples; f_0/f_s is not a simple fraction, so the error is not periodic). The full-scale range is $[-1, 1)$.

a) Implement the function `quantise()`, a mid-tread quantiser with N bits: rounding to the nearest step and clipping to the codes $q = -2^{N-1}, \ldots, 2^{N-1} - 1$. Implement `snr_db()`. Measure the SNR for $N = 4 \ldots 16$ at full scale and compare with $6.02N + 1.76$ dB. For which N does the formula fail, and why?

b) Repeat with an amplitude of 0.1 (−20 dBFS).

c) Implement jittered sampling (Gaussian δ_n, no quantisation) and measure the SNR for $f = 100\,\text{Hz} \ldots 20\,\text{kHz}$ and $\sigma_j = 0.1, 1, 4\,\mu\text{s}$. Compare with $-20\log_{10}(2\pi f \sigma_j)$.

d) Combine a 12-bit quantiser with jitter: compute the ENOB for the knock frequency 6.5 kHz. At which signal frequency are jitter noise and quantisation noise equal for $\sigma_j = 1\,\mu\text{s}$?

Solution

```c
static double quantise(double x, int nbits)
{
    double delta = 2.0 / (double)(1L << nbits);
    long q = (long)floor(x / delta + 0.5);
    long qmax = (1L << (nbits - 1)) - 1, qmin = -(1L << (nbits - 1));
    if (q > qmax) q = qmax;
    if (q < qmin) q = qmin;
    return q * delta;
}

/* SNR in dB of x (reference) and its distorted version y */
static double snr_db(const float *x, const float *y, int n)
{
    double ps = 0.0, pe = 0.0;
    int i;
    for (i = 0; i < n; i++) {
        double e = (double)y[i] - x[i];
        ps += (double)x[i] * x[i];
        pe += e * e;
    }
```

```
21      return 10.0 * log10(ps / pe);
22  }
```

Jittered sample: `y[n] = A*sin(2*M_PI*f*(t + sj*grand()) + 0.3);`
a/b) Program output (SNR in dB, 10^4 samples):

N	full scale	$6.02N + 1.76$	-20 dBFS	expected
4	24.74	25.84	5.27	5.84
5	31.21	31.86	10.57	11.86
6	37.46	37.88	18.80	17.88
8	49.77	49.92	29.79	29.92
10	61.89	61.96	42.17	41.96
12	73.95	74.00	53.98	54.00
14	85.99	86.04	66.09	66.04
16	98.14	98.08	78.08	78.08

From 8 bits on, the formula holds within 0.2 dB. For $N \leq 5$ the measured SNR is up to 1.1 dB lower: with few levels the error is no longer uniform and independent of the signal (it is a deterministic, signal-dependent staircase error with harmonics), and the amplitude $1 - \Delta/2$ needed to avoid clipping reduces the signal power. The -20 dBFS column is 20 dB lower (within ± 0.4 dB) for $N \geq 7$; for small N the reduced sine spans only a few codes and the deviations are larger (± 1.3 dB).
c) Measured vs. theory (dB):

f [Hz]	$\sigma_j = 0.1$ µs		1 µs		4 µs	
	meas.	theory	meas.	theory	meas.	theory
100	83.94	84.04	64.03	64.04	52.01	52.00
1000	64.11	64.04	44.06	44.04	32.06	32.00
6500	47.80	47.78	27.82	27.78	15.76	15.74
10500	43.71	43.61	23.63	23.61	11.49	11.57
20000	37.93	38.02	17.98	18.02	6.16	5.97

The SNR drops by 20 dB per decade of frequency and per decade of jitter. The measured values scatter by about ± 0.1 dB because only 10^4 random jitter values enter each estimate. Only at 20 kHz with 4 µs ($2\pi f \sigma_j = 0.5$) does the first-order approximation $e \approx \dot{x}\,\delta$ become inaccurate.
d) 12 bit + jitter at 6.5 kHz: $\sigma_j = 0.1$ µs: 47.7 dB (ENOB 7.6); 1 µs: 27.8 dB (ENOB 4.3); 4 µs: 15.8 dB (ENOB 2.3). At 100 Hz and 0.1 µs the full 12 bits remain (ENOB 11.98). Equality of the two noise sources: $2\pi f \sigma_j = 10^{-74/20} \Rightarrow f = 31.8$ Hz for $\sigma_j = 1$ µs. Conclusion: for kHz signals the sampling clock, not the number of bits, decides the effective resolution; ECUs therefore start knock conversions by hardware timers or sample continuously with a fixed ADC clock.

Exercise 4.4 Time domain vs. crank-angle domain

WOKWI

In the ESP32 program `code/wokwi_vscode/sheet04_angle/` the virtual engine accelerates from 1500 rpm to 4500 rpm within 0.5 s (`e.rpm` is changed during the simulation). The ion-current signal is sampled at $f_s = 50$ kHz; the rising crank edges are time-stamped with a resolution of 2 µs ("input capture" as in a real ECU: the time stamp is the first timer tick at or after the edge), and the cam level is latched at every edge.

a) Assign a crank angle to every edge: detect the gap (edge interval $> 2\times$ the previous one), give the first edge after the first gap the angle 0° or 360° (cam level!), then add 6° per edge, 18° across the gap.

b) Resample the signal onto the grid $\theta_m = \theta_0 + m \cdot 0.5°$: for every θ_m interpolate the time $t(\theta_m)$ linearly between the two neighbouring edges, then the signal value $x(t(\theta_m))$ linearly between two time samples (function `x_at()`). The program compares the grid with the true model

angle: report the maximum and rms angle error and explain its size. Change `DT_CAP` to `20e-6` (edges only known at the sample instants), rebuild and compare.

c) The program correlates every engine cycle with the first one, (i) in the angle domain (1440 grid points per cycle) and (ii) in the time domain (segments with the sample count of the first cycle, each starting exactly at its cycle start). Interpret the table and the two plots.

d) The knock-sensor signal contains crank-synchronous parts (combustion vibration $0 \ldots 60°$ after TDC) and fixed-frequency parts (valve impacts: 8 kHz, $\tau = 0.4$ ms). How do the two look in the angle domain during the ramp? Compute the angular length of 3τ of a valve burst at 1500 rpm and 4500 rpm.

e) Which angle step $\Delta\theta$ would be needed to sample the 10.5 kHz knock mode directly in the angle domain at 600 rpm and at 6000 rpm? Which signals should an ECU therefore sample angle-based, which time-based?

Solution

a/b) Key part of the solution:

```c
static double x_at(double tq)
{
    double u = tq * FS;
    int n = (int)floor(u);
    if (n < 0) n = 0;
    if (n > NS - 2) n = NS - 2;
    return X_LSB * (xs[n] + (xs[n + 1] - xs[n]) * (u - n));
}
```

```c
    /* ------------- a) angle of every crank edge ----------------------- */
    /* gap: edge-to-edge time > 2 x previous one (nominal ratio 3).
       The first edge after the gap is tooth 0 (0 or 360 deg, cam decides). */
    for (k = 2; k < ne; k++) {
        double d1 = te[k] - te[k - 1], d0 = te[k - 1] - te[k - 2];
        if (kref < 0) {
            if (d1 > 2.0 * d0) {                        /* first reference */
                kref = k;
                the[k] = cam_e[k] ? 0.0 : 360.0;
            }
        } else {
            the[k] = the[k - 1] + (d1 > 2.0 * d0 ? 18.0 : 6.0);
            if (d1 > 2.0 * d0 && fmod(the[k], 360.0) != 0.0)
                printf("warning: gap at unexpected angle %.0f\n", the[k]);
        }
    }
    printf("a) first reference edge: #%d at t = %.3f ms, angle %.0f deg, "
           "last edge angle %.0f deg\n", kref, te[kref] * 1e3, the[kref], the[ne - 1]);

    /* ------------- b) resampling onto the angle grid ------------------ */
    {
        int ke = kref;
        double th0 = the[kref], emax = 0.0, erms = 0.0;
        na = 0;
        while (na < MAXA) {
            double th = th0 + na * DTH, tq, err;
            if (th > the[ne - 1]) break;
            /* angle -> time: linear between the two neighbouring edges */
            while (ke < ne - 2 && the[ke + 1] < th) ke++;
            tq = te[ke] + (te[ke + 1] - te[ke]) * (th - the[ke]) / (the[ke + 1] - the[ke]);
            if (tq * FS > NS - 1) break;
            /* time -> value: linear between two signal samples */
            xa[na] = (float)x_at(tq);
```

Output: 1449 crank edges, first reference edge #57 at $t = 37.23\,\text{ms}$ (angle 360°, cam LOW), 17245 grid points. Angle error of the grid: max 0.054°, rms 0.023°. The error is caused by the time stamp resolution: an edge is registered up to Δt_{cap} late, i.e. $\omega\,\Delta t_{\text{cap}} = 27000\,°/\text{s} \cdot 2\,\mu\text{s} = 0.054°$ at 4500 rpm – exactly the maximum found. The linear interpolation between edges (constant speed over 6°) is not the limit: the 1 % second-order speed ripple changes ω by only $\approx 0.2\,\%$ over a tooth. With `DT_CAP 20e-6`: max 0.522°, rms 0.224° (and 0.027°/0.011° for 1 µs): the error scales linearly with the capture resolution. This is why ECUs time-stamp crank edges with a hardware capture unit instead of the ADC sample clock.
c) Output (excerpt):

cycle	rpm (mean)	samples/cycle	corr. time domain	corr. angle domain
1	1912	3139	1.000	1.0000
2	2259	2656	−0.131	0.9986
3	2559	2345	0.046	0.9986
5	3072	1953	−0.062	0.9986
7	3510	1710	−0.193	0.9986
10	4079	1471	0.035	0.9986

In the angle domain every cycle is (up to the 10 mV noise of the model) identical to the first one: correlation 0.998 for all cycles. In the time domain the correlation collapses already in the second cycle, although the segments start exactly at the cycle start and the speed changed by only 18 %: the ion peaks (width a few degrees) at 120°, 300°, … arrive 1.6 ms to 9 ms earlier and are narrower. The time plot shows the last cycle compressed to 47 % (about 2 cycles in the window of the first one), the angle plot shows two congruent curves. Crank-synchronous signals are *stationary in the angle domain* and non-stationary in the time domain during speed changes; angle-based processing (synchronous averaging, order analysis, Sheet 6) requires the resampling.
d) The combustion vibration is defined by the angle and stays congruent. A valve burst is a resonance with fixed frequency and decay time: in the angle domain its oscillation period is ω/f and changes with speed. $3\tau = 1.2\,\text{ms}$ corresponds to $9000\,°/\text{s} \cdot 1.2\,\text{ms} = 10.8°$ at 1500 rpm and 32.4° at 4500 rpm (one 8 kHz period: 1.1° resp. 3.4°). Fixed-frequency resonances are stationary in the time domain, crank-synchronous ones in the angle domain – no single domain makes everything stationary.
e) The equivalent sampling rate is $f_{\text{eq}} = \omega/\Delta\theta$; $f_{\text{eq}} > 2 \cdot 10.5\,\text{kHz}$ requires $\Delta\theta < \omega/21\,\text{kHz}$: 0.17° at 600 rpm ($\omega = 3600\,°/\text{s}$) and 1.7° at 6000 rpm. A fixed angle step that is fine enough at idle massively oversamples at high speed, a coarse one aliases at idle. Hence: knock sensor (fixed resonances) is sampled time-based with fixed f_s and evaluated in a crank-angle *window*; crank-synchronous quantities (cylinder pressure, ion current, engine speed per tooth, misfire detection) are sampled or resampled angle-based.

Deliverables

4.1: derivations and numbers. 4.2: completed sketch, logic-analyzer screenshot with sampling period and ISR duration, quiet-window results for three noise levels. 4.3: SNR tables (quantisation, jitter, ENOB) with short discussion. 4.4: completed program, angle-error values for two capture resolutions, the correlation table and the two plots with your interpretation.

Digital Signal Processing — Exercise Sheet 5

Nyquist

Script: Chapter 5 (Sampling Theorem, Aliasing, Reconstruction) — **with solutions**

Learning goals

- Predict alias frequencies with the folding rule and the folding diagram; determine minimum sampling rates, including band-pass (under-)sampling.
- Observe aliasing of the knock resonances on real (simulated) microcontrollers and verify the predictions quantitatively.
- Compare zero-order hold, linear and sinc reconstruction.
- Design an anti-aliasing filter: trade-off between filter order, sampling rate and passband loss.

Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. Sheets 0–4 ran entirely in Wokwi; if you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet05/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o recon recon.c -lm
./recon
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Background

Sampling with $f_s = 1/T_s$ makes all frequencies $f + k f_s$, $k \in \mathbb{Z}$, indistinguishable. A real tone of frequency f therefore appears at the *alias frequency*

$$f_a = |\, f - k f_s|, \qquad k = \mathrm{round}(f/f_s), \qquad 0 \le f_a \le f_s/2.$$

A band-limited signal ($X(f) = 0$ for $|f| \ge B$) is uniquely determined by its samples if $f_s > 2B$ and can be reconstructed by sinc interpolation $x(t) = \sum_n x[n]\,\mathrm{sinc}\big((t - nT_s)/T_s\big)$, $\mathrm{sinc}(u) = \sin(\pi u)/(\pi u)$. Everything above $f_s/2$ has to be removed *before* sampling by an analog anti-aliasing (AA) filter.

In our virtual engine the knock sensor has resonances at 6.5 kHz and 10.5 kHz ($\tau = 0.8$ ms) and valve impacts at 8 kHz ($\tau = 0.4$ ms). The `engine-sim` chip updates its analog outputs every 20 µs (attribute `fs_khz = 50`) and holds the value in between: the chip itself is a sampled system with zero-order hold.

Exercise 5.1 Aliasing by hand

PAPER

a) Compute the alias frequencies of the components 6.5 kHz, 8 kHz, 10.5 kHz and of a (hypothetical) third knock mode at 15 kHz for $f_s = 5$, 8.33, 16.67, 25 and 50 kHz. (8.33 kHz is close to the maximum rate of the Uno ADC.)

b) Draw the folding diagram (f_a over f, 0 kHz to 16 kHz) for $f_s = 8.33$ kHz and mark the four components. Which two components can no longer be separated after sampling?

c) What is the minimum sampling rate for a knock signal with components up to 15 kHz? A 4th-order Butterworth AA filter with $f_c = 15$ kHz ($|H| = 1/\sqrt{1 + (f/f_c)^8}$) is used. Which f_s is needed so that every frequency that folds into 0 kHz to 15 kHz is attenuated by at least 40 dB?

d) Band-pass sampling: an analog band-pass passes only the first knock mode, 6 kHz to 7 kHz. Which sampling rates are allowed ($2f_H/k \le f_s \le 2f_L/(k-1)$, $k = 1, 2, \dots$)? Show that the Uno at 5 kHz could legally capture this band. Where does the band appear, and is it spectrally inverted?

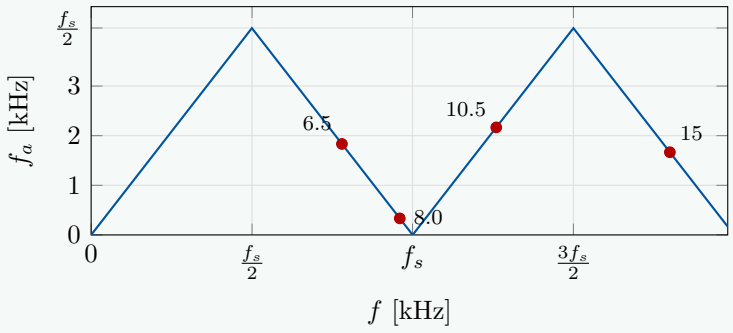
Solution

a) Alias frequencies in kHz (k in brackets):

f	$f_s = 5$	8.33	16.67	25	50
6.5 (knock 1)	1.5 (1)	1.833 (1)	6.5 (0)	6.5	6.5
8.0 (valve)	2.0 (2)	0.333 (1)	8.0 (0)	8.0	8.0
10.5 (knock 2)	0.5 (2)	2.167 (1)	6.167 (1)	10.5	10.5
15.0 (knock 3)	0 (3)	1.667 (2)	1.667 (1)	10.0 (1)	15.0

Notable cases: at 5 kHz the 15 kHz mode becomes DC; at 16.67 kHz the second knock mode lands at 6.17 kHz, right next to the first mode – a 6.5 kHz band-pass would see it; at 25 kHz a 15 kHz mode would mimic 10 kHz.

b) Folding diagram for $f_s = 8.33$ kHz (zig-zag with corners at multiples of $f_s/2$):



The two knock modes end up at 1.83 kHz and 2.17 kHz: only 333 Hz apart, which is less than the frequency resolution of a knock burst of a few ms (Sheet 8). Mode 3 (1.67 kHz) falls into the same region; the valve impact becomes an almost-DC transient.

c) Nyquist: $f_s > 30$ kHz. With an AA filter, a frequency f folds into $[0, 15]$ kHz if $f \ge f_s - 15$ kHz; the weakest attenuation occurs at $f = f_s - 15$. 40 dB requires $(f/f_c)^8 \ge 10^4$, i.e. $f \ge f_c \cdot 10^{0.5} = 47.4$ kHz, so $f_s \ge 62.4$ kHz (in practice 64 kHz). The theoretical minimum of 30 kHz would need an ideal brick-wall filter.

d) $f_L = 6$, $f_H = 7$, $B = 1$ kHz, $k \le \lfloor f_H/B \rfloor = 7$:

k	1	2	3	4	5	6	7
f_s [kHz]	≥ 14	7–12	4.67–6	3.5–4	2.8–3	2.33–2.4	2

$f_s = 5$ kHz lies in the $k = 3$ zone. The band lies in the third Nyquist zone $[5, 7.5]$ kHz (odd zone, not inverted) and appears at $f - f_s$, i.e. at 1 kHz to 2 kHz. Condition: the analog band-pass must suppress the valve (8 kHz→2 kHz) and mode 2 (10.5 kHz→0.5 kHz), which fold into the same 0 kHz to 2.5 kHz range. Band-pass sampling replaces a high f_s by a demanding analog filter.

Exercise 5.2 Aliasing made visible

WOKWI

Two projects record the `KNOCK` output of the engine-sim chip (knock probability 100 %, 3000 rpm) together with the ground-truth outputs `KNK` (HIGH during a knock burst) and `TDC`, and print blocks of CSV lines `n,adc,knk,tdc`: `code/wokwi/sheet05_alias_uno/` (Uno, Timer1 ISR, 512 samples per block) and `code/wokwi/sheet05_alias_esp32/` (ESP32, deadline loop, 2500 samples per block). The JSLinux program `code/jslinux/sheet05/alias_scan.c` cuts out every knock burst (from the rising KNK edge, 4τ) and every valve burst (TDC $+90°$, 5τ), averages the spectra $|X(f)|^2$ of the segments and lists the peaks.

a) Uno: the sampling period is `TS_US = DIV`×20 µs (default `DIV = 10`, 5 kHz). Compute `OCR1A` (prescaler 8) and complete the ISR (store KNK in bit 14, TDC in bit 15). Why does the sketch disable the Timer0 interrupt during acquisition?

b) Copy at least three blocks from the Serial Monitor into `uno.csv`, implement `alias_freq()` and the start index of the valve burst in `alias_scan.c` and run `./alias_scan uno.csv 5000 10`. Compare the spectral peaks with Exercise 5.1. Look at one knock burst in the CSV: how many oscillation periods do you see, and of which frequency? Repeat with `DIV = 6` (8.33 kHz). Check with the logic analyzer that the sample marker (D0) is still regular.

c) ESP32: complete the deadline loop (`TS_US = 40`, 25 kHz) and verify that no sample is late. Analyse with `./alias_scan esp.csv 25000 12`. Then set `TS_US = 60` (16.67 kHz). Why is this particularly dangerous for a knock detector that integrates the energy in a band around 6.5 kHz?

d) Why must the sampling period be a multiple of 20 µs? Consider the Uno at exactly 8 kHz (125 µs): determine the sequence of time offsets between the ADC instants and the last chip update, its effect on the SNR (Exercise 4.1 e) and a value of the chip attribute `fs_khz` in `diagram.json` that fixes the problem.

e) `./alias_scan sim 5000 10` simulates the chip and the ADC (“virtual Wokwi”, also writes `alias_sim.csv`). Compare its peaks with your Wokwi measurement.

The UART transmits about 11.5 characters per ms: printing a CSV line per sample is impossible at kHz rates, therefore samples are buffered in RAM (Uno: 512 samples = 1 KB of its 2 KB). Wokwi is not cycle-accurate for the ESP32 – if `late` is not 0, increase `TS_US` to the next multiple of 20.

Solution

a) `OCR1A` $= 2 \cdot$ `TS_US` $- 1 = 399$ (2 MHz timer clock).

```
ISR(TIMER1_COMPA_vect)
{
    PINB = _BV(PB0);                          /* toggle pin 8: sample marker */
    uint16_t v = analogRead(A0);
    if (PIND & _BV(PD3)) v |= 0x4000;         /* KNK (pin 3)                 */
    if (tdc_seen) { v |= 0x8000; tdc_seen = 0; }
    buf[nbuf++] = v;
    if (nbuf >= NBUF) { TIMSK1 = 0; done = true; }
}

void timer1_start()
{
    noInterrupts();
    TCCR1A = 0;
    TCCR1B = _BV(WGM12) | _BV(CS11);          /* CTC, clk/8 = 2 MHz          */
    OCR1A  = 2 * TS_US - 1;                   /* 2 ticks per microsecond     */
    TCNT1  = 0;
    TIFR1  = _BV(OCF1A);
    TIMSK1 = _BV(OCIE1A);
    TIMSK0 = 0;           /* stop the millis() interrupt: no extra jitter  */
```

```
    interrupts();
}
```

The Timer0 overflow ISR (`millis()`, every 1.024 ms, a few µs long) can delay the Timer1 ISR, i.e. cause sampling jitter (Sheet 4); the Timer1 ISR itself starts with a constant latency.
b) Results of the virtual Wokwi (`sim`, 400 ms, 39 knock and 38 valve bursts); your Wokwi recording should agree within the resolution:

	predicted (5.1)		measured peaks	
f_s [kHz]	knock modes	valve	knock bursts	valve bursts
5 (Uno)	1.5 / 0.5	2.0	1.51, 0.255 (34 %)	1.78, 0.98 (83 %)
8.33 (Uno)	1.833 / 2.167	0.333	1.85	0.367
25 (ESP32)	6.5 / 10.5	8.0	6.50, 10.525 (24 %)	8.025
16.67 (ESP32)	6.5 / 6.167	8.0	6.483	8.183

(all values in kHz; percentages: power relative to the largest peak). At 5 kHz the 6.5 kHz knock appears at 1.5 kHz: a burst of $4\tau = 3.2$ ms (16 samples) shows about 5 periods of 1.5 kHz, i.e. the "wrong" frequency is clearly visible in the time signal. The second peak (alias of 10.5 kHz at 0.5 kHz) is shifted to 255 Hz because the segment of only 3.2 ms resolves no better than $\approx 1/3.2$ ms ≈ 300 Hz and the combustion vibration adds low-frequency content. The valve burst segments (2 ms, 10 samples) have ≈ 500 Hz resolution; the peak at 1.78 kHz instead of 2.0 kHz and the side peak result from the overlap with the mirrored component at $f_s - 2$ kHz $= 3$ kHz. At 8.33 kHz the two knock aliases (333 Hz apart) merge into one peak at 1.85 kHz, and the 8 kHz valve burst looks like a slow 0.33 kHz transient. The Uno at 8.33 kHz works only because the CPU does nothing but sampling during the block ($\approx$ 112 µs of every 120 µs); D0 must show regular 120 µs steps.
c) Deadline loop:

```
    t_next = micros() + TS_US;
    for (i = 0; i < NBUF; i++) {
        while ((int32_t)(micros() - t_next) < 0) { }
        if ((int32_t)(micros() - t_next) > (int32_t)(TS_US / 2)) late++;
        t_next += TS_US;
        digitalWrite(PIN_MARK, mark ^= 1);
        uint16_t v = analogRead(PIN_KNOCK);
        if (digitalRead(PIN_KNK)) v |= 0x4000;
        if (tdc_seen) { v |= 0x8000; tdc_seen = 0; }
        buf[i] = v;
    }
```

At 25 kHz all components lie below $f_s/2 = 12.5$ kHz and appear at their true frequencies (6500 Hz, 10 525 Hz with power ratio $0.24 \approx 0.5^2$ as in the model, valve 8025 Hz). At 16.67 kHz mode 2 aliases to 6.17 kHz and merges with mode 1 into a single peak at 6.48 kHz. A band-pass detector at 6.5 kHz integrates the energy of both modes, and every disturbance between 10 kHz and 11 kHz (e.g. ignition or injector noise) would be counted as knock – aliasing produces false knock detections that no digital filter can remove afterwards.
d) Sampling a held (staircase) signal with a period that is a multiple of 20 µs is equivalent to exact decimation of the chip's 50 kHz sequence. With 125 µs the offset between ADC instant and last chip update cycles through $125n \bmod 20 = 0, 5, 10, 15, 0, \ldots$ µs. The mean (7.5 µs) is a harmless delay, the varying part has $\sigma = 5.6$ µs, i.e. SNR $= -20\log_{10}(2\pi \cdot 6500 \cdot 5.6 \cdot 10^{-6}) \approx 12.8$ dB for mode 1 (8.7 dB for mode 2), concentrated in spurious lines at $\pm f_s/4$ around each component because the offset is periodic. Fix: `"fs_khz": "40"` (update every 25 µs, $125 = 5 \cdot 25$; the chip's own Nyquist limit 20 kHz is still above 10.5 kHz) or `"200"` (5 µs, more simulation load).
e) The simulation produces the peaks of the table in b); the Wokwi recordings differ only by the random knock amplitudes/phases (the chip uses its own random sequence) and the number of bursts.

Exercise 5.3 Reconstruction and anti-aliasing JSLINUX

Reconstruction. `code/jslinux/sheet05/recon.c` represents a "continuous" sine on a grid of $M = 40$ points per sampling interval ($f_s = 25\,\text{kHz}$, 600 samples) and computes the normalised error $\text{NMSE} = \sum e^2 / \sum x^2$ of a reconstruction on this grid.

a) Implement zero-order hold, linear interpolation and sinc interpolation with $\pm K = 16$ terms, once plain (truncated) and once with a Hann taper $w(u) = \frac{1}{2} + \frac{1}{2}\cos(\pi u/(K+1))$. Show analytically that the ZOH of a sine has $\text{NMSE} = 2(1 - \sin(\omega T_s)/(\omega T_s))$ and compare. Discuss the table (method vs. f/f_s).

b) A 15 kHz tone is sampled at 25 kHz and sinc-reconstructed. What comes out? Predict frequency *and* phase.

Anti-aliasing. `code/jslinux/sheet05/antialias.c` simulates the analog world on a 1 MHz grid; the AA filter (cut-off $f_c = 12\,\text{kHz}$) is simulated on this grid, the ADC takes every ($1\,\text{MHz}/f_s$)-th value. Test signals: knock tones 6.5 kHz/10.5 kHz; a disturbance at $f_d = f_s - 6.5\,\text{kHz}$ that aliases exactly onto the knock frequency; white noise (bandwidth 500 kHz).

c) Implement the RC low-pass (exact update for an input held constant over Δt), the 4th-order Butterworth as a cascade of two 2nd-order sections ($Q_k = 1/(2\sin\frac{(2k-1)\pi}{2n})$, given biquad simulator) and the amplitude measurement at a given frequency in the sampled signal. Run the program for $f_s = 25$ and 50 kHz and filter orders 0, 1, 2, 4. Compare the alias amplitudes with $|H(f_d)|$.

d) Which Butterworth order would be required for 40 dB alias suppression in the band 0 kHz to 10.5 kHz with $f_c = 12\,\text{kHz}$ at $f_s = 25\,\text{kHz}$ and at 50 kHz? Formulate a design recommendation for the knock ADC.

Solution

a) Implementations:

```
static void rec_zoh(void)
{
    int i;
    for (i = 0; i < NF; i++) xr[i] = xs[i / M];
}

static void rec_lin(void)
{
    int i;
    for (i = 0; i < NF; i++) {
        int n = i / M;
        double u = (double)(i % M) / M;
        xr[i] = n + 1 < NSMP ? (1.0 - u) * xs[n] + u * xs[n + 1] : xs[n];
    }
}

static void rec_sinc(int windowed)
{
    int i, k;
    for (i = 0; i < NF; i++) {
        double tn = (double)i / M;              /* time in sample periods */
        int n = i / M;
        double s = 0.0;
        for (k = n - K; k <= n + K + 1; k++) {
            double u = tn - k, w = 1.0;
            if (k < 0 || k >= NSMP) continue;
            if (windowed) {
                if (fabs(u) >= K + 1) continue;
                w = 0.5 + 0.5 * cos(M_PI * u / (K + 1));   /* Hann taper */
```

```
            }
            s += xs[k] * sinc(u) * w;
        }
        xr[i] = s;
    }
}
```

ZOH error of $x = \sin(\omega t + \varphi)$ at $t = nT_s + u$: $e = \sin(\alpha + \omega u) - \sin\alpha$; averaged over the phase α: $\overline{e^2} = \frac{1}{2}|e^{j\omega u} - 1|^2 = 1 - \cos\omega u$; averaged over $u \in [0, T_s)$: $1 - \sin(\omega T_s)/(\omega T_s)$; division by the signal power $1/2$ gives the formula. Program output (NMSE in dB):

f [Hz]	f/f_s	ZOH	ZOH theory	linear	sinc	Hann-sinc
500	0.02	−22.98	−22.79	−56.83	−42.94	−97.22
1000	0.04	−16.94	−16.78	−44.79	−44.98	−99.46
2500	0.10	−9.06	−8.89	−28.94	−47.59	−101.24
5000	0.20	−3.28	−3.13	−17.12	−37.44	−84.43
8000	0.32	0.27	0.41	−9.43	−32.66	−70.23
10500	0.42	2.02	2.13	−5.28	−30.83	−57.78
12000	0.48	2.76	2.83	−3.38	−27.58	−11.36

The ZOH agrees with the theory within 0.2 dB (the fine grid contains $u = 0$ with zero error, hence slightly smaller values). The ZOH is a poor reconstruction: even at $f = f_s/25$ the error is only 17 dB below the signal – mainly due to the delay $T_s/2$ and the staircase; above $f \approx f_s/3.5$ the error is larger than the signal. Linear interpolation gains about 12 dB per octave (error $\propto (\omega T_s)^4$) but is useless near $f_s/2$. The truncated sinc has a frequency-independent floor of about $-30 \ldots -48$ dB caused by cutting the slowly decaying $(1/u)$ kernel; at low frequencies it is even worse than linear interpolation. The Hann taper removes this floor (-97 to -101 dB), but its transition band costs accuracy close to $f_s/2$ (-11 dB at $0.48 f_s$): the closer a signal gets to Nyquist, the longer the interpolation kernel has to be.

b) $\sin(2\pi\, 15000\, nT_s + 0.7) = \sin(2\pi(15000 - 25000)nT_s + 0.7) = \sin(-2\pi\, 10000\, nT_s + 0.7)$: the samples are those of a 10 kHz sine with inverted phase, $-\sin(2\pi\, 10\,\text{kHz}\, t - 0.7)$. The program finds a zero-crossing frequency of 10 000 Hz and NMSE = 3.00 dB against the 15 kHz original, but −59.7 dB against this alias. The reconstruction is perfect – of the wrong signal.

c) Key code:

```
static void rc_filter(double *y, const double *x, int n, double fc, double dt)
{
    double a = exp(-2.0 * M_PI * fc * dt), s = 0.0;
    int i;
    for (i = 0; i < n; i++) {
        s = a * s + (1.0 - a) * x[i];
        y[i] = s;
    }
}
```

```
    case 4: /* Q_k = 1 / (2 sin((2k-1) pi / (2n))), k = 1, 2           */
        biquad_filter(tmp, x, n, FC, 1.0 / (2.0 * sin(M_PI / 8.0)), dt);
        biquad_filter(y, tmp, n, FC, 1.0 / (2.0 * sin(3.0 * M_PI / 8.0)), dt);
        break;
```

```
    for (n = 0; n < N; n++) {
        double v = y[(n0 + n) * D], w = 2.0 * M_PI * f * n / fs;
        re += v * cos(w);
        im -= v * sin(w);
    }
    return 2.0 * sqrt(re * re + im * im) / N;
```

Results (f_c = 12 kHz; "noise" = noise PSD in 5 kHz to 11 kHz after sampling relative to the unaliased analog noise PSD):

f_s	order	gain 6.5k	gain 10.5k	alias of f_d	$\lvert H(f_d)\rvert$	noise
25 kHz	0	0.00	0.00	0.0 dB	0.0 dB	+15.7 dB
($f_d = 18.5$ kHz)	1	−1.12	−2.47	−5.3 dB	−5.3 dB	+2.0 dB
	2	−0.36	−2.01	−8.2 dB	−8.2 dB	0.0 dB
	4	−0.03	−1.29	−15.2 dB	−15.2 dB	0.0 dB
50 kHz	0	0.00	0.00	0.0 dB	0.0 dB	+12.7 dB
($f_d = 43.5$ kHz)	1	−1.12	−2.47	−11.5 dB	−11.5 dB	−0.3 dB
	2	−0.36	−2.01	−22.4 dB	−22.4 dB	−1.0 dB
	4	−0.03	−1.29	−44.8 dB	−44.7 dB	−0.2 dB

(gains in dB.) The sampled alias amplitude equals $|H(f_d)|$ exactly: the filter acts before sampling, the sampler only moves the attenuated component to 6.5 kHz. Without filter, all noise of the 500 kHz band folds into $[0, f_s/2]$: the in-band noise rises by $10\log_{10}(500/12.5) = 16$ dB (measured 15.7 dB) at 25 kHz and by 13 dB at 50 kHz. Already a first-order filter removes most of this broadband penalty, but at $f_s = 25$ kHz even the 4th-order filter suppresses the disturbance by only 15 dB while it costs 1.3 dB at 10.5 kHz.

d) The first frequency that folds into 0 kHz to 10.5 kHz is $f_s - 10.5$ kHz. Requirement $(f/f_c)^{2n} \geq 10^4$: $f_s = 25$ kHz: $f/f_c = 14.5/12 = 1.21$, $n \geq 2/\log_{10} 1.21 = 24.3$, i.e. order 25 – impossible. $f_s = 50$ kHz: $f/f_c = 39.5/12 = 3.29$, $n \geq 2/\log_{10} 3.29 = 3.9$, i.e. order 4 (the table confirms 44.8 dB at 43.5 kHz). Recommendation: oversample (e.g. 50 kHz to 100 kHz), use a simple 2nd–4th-order analog AA filter (Sallen-Key), and reduce the rate digitally afterwards (digital low-pass + decimation, Sheet 10), where steep filters are cheap and exactly reproducible.

Deliverables

5.1: alias table, folding diagram, AA and band-pass sampling calculation. 5.2: completed sketches, CSV files of the Uno (two rates) and the ESP32 (two rates), output of `alias_scan` with comparison to 5.1, answers to c) and d). 5.3: completed programs, both tables with discussion, filter-order recommendation.

Digital Signal Processing Exercise Sheet 6

Fourier

Script: Chapter 6 (Fourier Series, Fourier Transform, DTFT) **with solutions**

Learning goals

- Compute Fourier coefficients of engine signals analytically and numerically; understand the line spectrum of the 60-2 crank signal and the effect of the missing teeth.
- Relate the decay time of a resonance to the width of its spectral peak (Lorentzian line shape, bandwidth, quality factor).
- Evaluate the DTFT by direct summation and use its properties (periodicity, symmetry).
- Use *engine orders* to distinguish speed-proportional components from fixed-frequency resonances; track orders during a speed change.

Background

A signal with period P has the Fourier series $x(\theta) = \sum_k c_k\, \mathrm{e}^{\mathrm{j}2\pi k\theta/P}$, $c_k = \frac{1}{P}\int_0^P x(\theta)\mathrm{e}^{-\mathrm{j}2\pi k\theta/P}\,\mathrm{d}\theta$. For crank-angle signals we choose $P = 360°$: then k counts oscillations per crankshaft revolution and is called the *engine order*. At speed n (rpm) order k has the frequency $f = k\,n/60$. Aperiodic signals have a Fourier transform $X(f) = \int x(t)\mathrm{e}^{-\mathrm{j}2\pi ft}\,\mathrm{d}t$; a sampled signal $x[n] = x(nT_s)$ has the DTFT

$$X(\mathrm{e}^{\mathrm{j}\Omega}) = \sum_n x[n]\,\mathrm{e}^{-\mathrm{j}\Omega n}, \qquad \Omega = 2\pi f/f_s,$$

which is 2π-periodic in Ω (f_s-periodic in f) and conjugate symmetric for real $x[n]$. For a finite record of N samples it can be evaluated at any frequency by direct summation; the FFT (Sheet 8) is only a fast way to do this on a regular grid. A sine of amplitude A whose frequency fits an integer number of periods into the record has $|X| = AN/2$, so $A = 2|X|/N$.

Exercise 6.1 Fourier series and transforms of engine signals

PAPER

a) A complete 60-tooth wheel gives a rectangular signal with period 6° and duty cycle 50 %. Compute its Fourier coefficients with respect to $P = 360°$. Which orders occur, what are c_0, $|c_{60}|$, $|c_{120}|$, $|c_{180}|$? Which frequency does order 60 have at 3000 rpm?

b) For the 60-2 wheel (tooth $m = 0, \ldots, 57$ HIGH on $[6m, 6m+3)°$) show that

$$|c_k| = \frac{3}{360}\left|\operatorname{sinc}\frac{k}{120}\right|\left|\frac{\sin(29\pi k/30)}{\sin(\pi k/60)}\right| \quad (k \neq 60m), \qquad |c_{60m}| = \frac{58}{120}\left|\operatorname{sinc}\frac{m}{2}\right|,$$

with $\operatorname{sinc} u = \sin(\pi u)/(\pi u)$. Evaluate c_0, $|c_1|$, $|c_{59}|$, $|c_{60}|$, $|c_{61}|$ and explain the result as a "once-per-revolution modulation". Hint: $x_{60-2} = x_{60} - x_{\mathrm{gap}}$.

c) Compute the Fourier transform of the knock model $x(t) = \mathrm{e}^{-t/\tau}\sin(2\pi f_0 t)$, $t \geq 0$. Show that near f_0 the magnitude is a Lorentzian curve, and derive the -3 dB bandwidth Δf and the quality factor $Q = f_0/\Delta f$. Evaluate for the knock mode ($f_0 = 6.5$ kHz, $\tau = 0.8$ ms) and the valve impact (8 kHz, 0.4 ms). What follows for the bandwidth of a knock band-pass filter?

d) Engine orders: list the frequencies of order 2 (combustions of the 4-cylinder engine), order 60 (crank teeth) and order 0.5 (camshaft) at 1500, 3000 and 6000 rpm, and the order at which the 6.5 kHz knock resonance appears at these speeds.

Solution

a) The signal is periodic with 6°, so only orders $k = 60m$ occur. For a rectangular pulse of width $w = 3°$ in the period $P_0 = 6°$: $c_{60m} = \frac{w}{P_0}\,\mathrm{sinc}(mw/P_0)\,\mathrm{e}^{-\mathrm{j}\pi mw/P_0} = \frac{1}{2}\,\mathrm{sinc}\,\frac{m}{2}\,\mathrm{e}^{-\mathrm{j}\pi m/2}$. $c_0 = 0.5$, $|c_{60}| = 1/\pi = 0.318$, $c_{120} = 0$ (all even multiples vanish for 50 % duty), $|c_{180}| = 1/(3\pi) = 0.106$. At 3000 rpm order 60 is $60 \cdot 50\,\mathrm{Hz} = 3\,\mathrm{kHz}$.
b) Each tooth contributes $\frac{1}{360}\int_{6m}^{6m+3}\mathrm{e}^{-\mathrm{j}2\pi k\theta/360}\,\mathrm{d}\theta = \frac{3}{360}\,\mathrm{sinc}\,\frac{k}{120}\,\mathrm{e}^{-\mathrm{j}\pi k/120}\,\mathrm{e}^{-\mathrm{j}2\pi km/60}$. The sum over $m = 0,\dots,57$ is geometric: $\left|\sum_m \mathrm{e}^{-\mathrm{j}2\pi km/60}\right| = |\sin(58\pi k/60)/\sin(\pi k/60)|$ (and 58 for $k = 60m$), which gives the formula. Values: $c_0 = 58 \cdot 3/360 = 0.4833$; $|c_1| = \frac{3}{360} \cdot 0.99989 \cdot \frac{\sin(\pi/30)}{\sin(\pi/60)} = 0.008333 \cdot 0.99989 \cdot 1.9973 = 0.01664$; $|c_{59}| = 0.01077$; $|c_{60}| = \frac{58}{120} \cdot \frac{2}{\pi} = 0.3077$; $|c_{61}| = 0.01042$. Interpretation: $x_{60-2} = x_{60} - x_{\mathrm{gap}}$, where x_{gap} are the two missing pulses (period 360°). x_{gap} has lines at *all* orders, $|c_k| \approx 2 \cdot 3/360 = 1/60$ at low orders. The gap modulates the tooth signal once per revolution: the tooth line at order 60 loses 2/60 of its amplitude (0.3183 → 0.3077) and is surrounded by sidebands at $60 \pm 1, 60 \pm 2, \dots$, like an amplitude-modulated carrier. The low orders ($k = 1, 2, \dots$) carry the reference-mark information.
c) $x(t) = \frac{1}{2\mathrm{j}}(\mathrm{e}^{(-1/\tau+\mathrm{j}\omega_0)t} - \mathrm{e}^{(-1/\tau-\mathrm{j}\omega_0)t})$, $t \geq 0$:

$$X(f) = \frac{1}{2\mathrm{j}}\left[\frac{1}{1/\tau + \mathrm{j}2\pi(f - f_0)} - \frac{1}{1/\tau + \mathrm{j}2\pi(f + f_0)}\right].$$

For $f \approx f_0 \gg 1/\tau$ the second term is negligible: $|X(f)| \approx \dfrac{\tau/2}{\sqrt{1 + (2\pi\tau(f - f_0))^2}}$ – a Lorentzian with peak $\tau/2$. Half power at $2\pi\tau|f - f_0| = 1$, so $\Delta f = 1/(\pi\tau)$ and $Q = \pi f_0 \tau$. Knock: $\Delta f = 398\,\mathrm{Hz}$, $Q = 16.3$; valve: $\Delta f = 796\,\mathrm{Hz}$, $Q = 10.1$. A knock band-pass much narrower than 400 Hz would not capture more signal energy but would ring longer than the burst itself (its own impulse response lasts $\approx 1/(\pi B)$); a bandwidth of 0.5 kHz to 1 kHz matches the physics. The faster decay of the valve burst makes its peak twice as wide.
d) $f = k\,n/60$:

	1500 rpm	3000 rpm	6000 rpm
order 0.5 (cam)	12.5 Hz	25 Hz	50 Hz
order 2 (combustion)	50 Hz	100 Hz	200 Hz
order 60 (teeth)	1.5 kHz	3 kHz	6 kHz
6.5 kHz resonance	order 260	order 130	order 65

Crank-synchronous components have fixed orders and frequencies proportional to speed; structural resonances have a fixed frequency and a speed-dependent order. At 6000 rpm the tooth frequency even reaches the knock band.

Exercise 6.2 Numerical Fourier series and DTFT

JSLINUX

a) The program `code/jslinux/sheet06/fourier_series.c` samples one revolution of the crank signal `es_crank()` on a grid of $M = 36000$ points (0.01°). Implement the direct summation

$$c_k \approx \frac{1}{M}\sum_{n=0}^{M-1} x[n]\,\mathrm{e}^{-\mathrm{j}2\pi kn/M}, \qquad k = 0,\dots,1200$$

(use a rotating phasor instead of calling `cos` and `sin` for every term) and compare with your results of 6.1 b).

b) Implement the partial sum $S_K(\theta) = c_0 + \sum_{k=1}^{K} 2\,\mathrm{Re}\{c_k \mathrm{e}^{\mathrm{j}2\pi k\theta/360}\}$ and evaluate it around the gap for $K = 60, 180, 600, 1200$. Report maximum, overshoot and rms error. Explain the Gibbs phenomenon: what converges and what does not?

c) `code/jslinux/sheet06/dtft.c`: implement the DTFT by direct summation. For a damped sinusoid ($f_0 = 6.5\,\mathrm{kHz}$, $f_s = 50\,\mathrm{kHz}$, $N = 500$) and $\tau = 0.4/0.8/1.6\,\mathrm{ms}$ the program determines

peak frequency, peak value and $-3\,\mathrm{dB}$ bandwidth. Compare with 6.1 c) (note: $X_{\mathrm{DTFT}} \approx f_s X(f)$), and compare the finite sum with the closed-form DTFT of $a^n \sin(\Omega_0 n) u[n]$.

d) Check numerically: $|X|$ at f, $f + f_s$, $-f$ and $f_s - f$. Explain.

e) Apply the DTFT to a real knock burst of the virtual engine (both modes, noise, combustion vibration). Identify all features of the spectrum.

Solution

a)

```
    for (k = 0; k <= KMAX; k++) {
        /* rotating phasor instead of cos/sin per term (saves time)        */
        double cw = cos(2.0 * M_PI * k / M), sw = sin(2.0 * M_PI * k / M);
        double pr = 1.0, pi = 0.0, re = 0.0, im = 0.0, t;
        for (n = 0; n < M; n++) {
            re += x[n] * pr;               /* x * exp(-j w n)              */
            im -= x[n] * pi;
            t = pr * cw - pi * sw; pi = pr * sw + pi * cw; pr = t;
        }
        c[k].re = re / M; c[k].im = im / M;
```

k	$\lvert c_k\rvert$ numeric	$\lvert c_k\rvert$ analytic	ratio
0	0.483333	0.483333	1.0000
1	0.016642	0.016642	1.0000
2	0.016568	0.016568	1.0000
30	0.000000	0	–
31	0.000780	0.000780	1.0000
59	0.010772	0.010772	1.0000
60	0.307701	0.307700	1.0000
61	0.010419	0.010419	1.0000
120	0.000000	0	–
180	0.102571	0.102567	1.0000

The direct sum is a rectangle-rule integration; with $0.01°$ steps and tooth edges on the grid it is exact to 4–6 digits. The small deviation at $k = 180$ is the aliasing of the sampled pulse train (orders 180 ± 36000). Order 30 vanishes because the 58 tooth phasors $(-1)^m$ cancel; order 120 because of the 50 % duty cycle.

b)

```
static double partial_sum(double theta, int K)
{
    double s = c[0].re, w = 2.0 * M_PI * theta / 360.0;
    int k;
    for (k = 1; k <= K; k++)
        s += 2.0 * (c[k].re * cos(k * w) - c[k].im * sin(k * w));
    return s;
}
```

K	$\max S_K$	overshoot	$\min S_K$	rms error
60	1.1675	16.75 %	−0.3257	0.2346
180	1.1356	13.56 %	−0.1705	0.1413
600	1.0962	9.62 %	−0.0905	0.0833
1200	1.0923	9.23 %	−0.0893	0.0599

With $K = 60$ only the fundamental of the tooth signal and the gap-sidebands are present: the teeth look like a sine. Increasing K makes the ringing narrower (period $\approx 360°/K$) and the

rms error falls ($\propto 1/\sqrt{K}$), but the overshoot next to every edge does not disappear; it tends to 0.0895 of the jump (Gibbs constant; 9.2 % at $K = 1200$, the neighbouring edges only 3° away still contribute). The Fourier series converges in the mean-square sense, not uniformly, at a discontinuity. Consequence: band-limiting a rectangular signal (e.g. an input filter) always produces overshoot – relevant for threshold-based edge detection.
c)

```
static cpx dtft(const double *s, int n, double f)
{
    cpx X = {0.0, 0.0};
    double w = 2.0 * M_PI * f / FS;
    int k;
    for (k = 0; k < n; k++) {
        X.re += s[k] * cos(w * k);
        X.im -= s[k] * sin(w * k);
    }
    return X;
}
```

τ [ms]	peak [Hz]	$\lvert X\rvert$ peak	$f_s\tau/2$	$\Delta f_{-3\mathrm{dB}}$ [Hz]	$1/(\pi\tau)$ [Hz]	Q
0.4	6490	10.00	10	798	796	8.1
0.8	6500	20.00	20	398	398	16.3
1.6	6500	39.92	40	199	199	32.7

Bandwidth and peak agree with the Lorentzian of 6.1 c) (peak of $X(f)$ is $\tau/2$, times f_s for the DTFT). Doubling τ halves the bandwidth and doubles Q (time–bandwidth product $\tau\Delta f = 1/\pi$). Small effects: the peak at 6490 Hz for $\tau = 0.4$ ms (the negative-frequency term pulls it slightly), and $39.92 < 40$ for 1.6 ms because the record of 10 ms cuts the burst after 6.25τ. Direct sum vs. closed form $X = a\sin\Omega_0 \mathrm{e}^{-\mathrm{j}\Omega}/(1 - 2a\cos\Omega_0\mathrm{e}^{-\mathrm{j}\Omega} + a^2\mathrm{e}^{-2\mathrm{j}\Omega})$, $a = \mathrm{e}^{-1/(f_s\tau)}$: maximum difference $7.5\cdot10^{-5}$, consistent with the truncated tail $a^N = 3.7\cdot10^{-6}$.
d) Output: $\lvert X\rvert = 7.6224$ at 6000 Hz, 56 000 Hz, −6000 Hz and 44 000 Hz; $X(6000) = 7.1174 - 2.7283\mathrm{j}$, $X(-6000) = 7.1174 + 2.7283\mathrm{j}$. Periodicity: $\mathrm{e}^{-\mathrm{j}2\pi(f+f_s)n/f_s} = \mathrm{e}^{-\mathrm{j}2\pi fn/f_s}$ for integer n – this is aliasing seen from the frequency domain. Conjugate symmetry $X(-f) = X^*(f)$ holds for real $x[n]$; together: $\lvert X(f_s - f)\rvert = \lvert X(f)\rvert$, so $[0, f_s/2]$ contains all information.
e) Knock burst of amplitude 1.50 V: mode 1 peak at 6470 Hz, $\lvert X\rvert = 28.5$ (expected $1.5\cdot20 = 30$), bandwidth 410 Hz; mode 2 at 10 410 Hz, $\lvert X\rvert = 13.9$ (expected $0.75\cdot20 = 15$), bandwidth 380 Hz. In addition: a broad low-frequency hump (0 Hz to 500 Hz) from the combustion vibration and the noise floor of the white sensor noise. The deviations from the ideal values (peak positions ±90 Hz, heights −5 %) are caused by the noise and the overlap of the two modes and the low-frequency part in a single, random burst.

Exercise 6.3 Engine orders

JSLINUX

`code/jslinux/sheet06/orders.c` records the knock-sensor signal of the virtual engine (knock probability 0.2) at constant 1500, 3000 and 6000 rpm, each over exactly 8 engine cycles (16 revolutions, $f_s = 50$ kHz), removes the mean and evaluates the amplitude spectrum $A(f) = 2\lvert X(f)\rvert/N$ by direct summation on 0 Hz to 1000 Hz (1 Hz steps) and 4 kHz to 12 kHz (10 Hz steps).

a) Implement `amp_spectrum()`. Why is the record chosen as an integer number of revolutions?

b) The combustion vibration of the model is a half-sine $0.2\,\mathrm{V}\cdot\sin(\pi\varphi/60°)$ for $0 \le \varphi < 60°$ after every TDC (period 180°). Compute its Fourier series and the amplitudes of orders 2, 4, 6, 8; enter your formula in `halfsine_order_amp()`.

c) Run the program. Compare the low-frequency lines (frequency and amplitude) at the three speeds with b). For the high band, the program smooths $\lvert X\rvert^2$ over ±150 Hz and reports the

peak in three bands – tabulate frequency and order of the peaks.

d) Explain the fine structure of the high-frequency spectrum: why does the valve resonance appear as a comb of lines with a spacing of order 2 under a Lorentzian envelope, while knock gives a smooth hump?

e) An ECU wants to monitor a crank-synchronous vibration and the knock resonance with fixed band-pass filters. Which of the two filters must be adapted to the engine speed?

Solution

a)

```c
static void amp_spectrum(const double *s, int n, const double *f, double *A, int nf)
{
    int i, k;
    for (i = 0; i < nf; i++) {
        double w = 2.0 * M_PI * f[i] / FS, cw = cos(w), sw = sin(w);
        double pr = 1.0, pi = 0.0, re = 0.0, im = 0.0, t;
        for (k = 0; k < n; k++) {
            re += s[k] * pr;
            im -= s[k] * pi;
            t = pr * cw - pi * sw; pi = pr * sw + pi * cw; pr = t;
        }
        A[i] = 2.0 * sqrt(re * re + im * im) / n;
    }
}
```

With 16 revolutions every order k has $16k$ complete periods in the record: the components are "bin-centred" and $2|X|/N$ is their exact amplitude without leakage into neighbouring frequencies (Sheet 8). The mean is removed because the half-sines have a DC value of 42 mV whose spectral main lobe would otherwise dominate the lowest frequencies.

b) With $w = 60°$, $P = 180°$, $A = 0.2\,\mathrm{V}$ and $\beta = 2\pi m/P$: $\int_0^w \sin(\pi\varphi/w)\mathrm{e}^{-\mathrm{j}\beta\varphi}\,\mathrm{d}\varphi = \frac{\pi/w\,(1+\mathrm{e}^{-\mathrm{j}\beta w})}{(\pi/w)^2-\beta^2}$, hence with $r = 2mw/P = 2m/3$

$$|c_m| = \frac{Aw}{P}\,\frac{2}{\pi}\,\frac{|\cos(\pi r/2)|}{|1-r^2|}, \qquad A_{2m} = 2|c_m|.$$

Harmonic m of the 180° period is engine order $2m$: $A_2 = 76.4\,\mathrm{mV}$, $A_4 = 54.6\,\mathrm{mV}$, $A_6 = 28.3\,\mathrm{mV}$, $A_8 = 6.9\,\mathrm{mV}$ ($c_0 = Aw/P \cdot 2/\pi = 42.4\,\mathrm{mV}$).

c) Low-frequency lines (amplitudes in mV):

	order 2	order 4	order 6	order 8
theory	76.4	54.6	28.3	6.9
1500 rpm (50/100/150/200 Hz)	76.6	54.4	28.5	6.6
3000 rpm (100/200/300/400 Hz)	76.3	53.7	28.4	6.3
6000 rpm (200/400/600/800 Hz)	76.0	53.0	29.4	5.6

The strongest line is always order 2 (50, 100, 200 Hz); the amplitudes are speed-independent and agree with b) within a few percent (deviations: 1 % speed ripple, random knock bursts and noise). High band (peak of the smoothed $|X|^2$):

band	1500 rpm	3000 rpm	6000 rpm
5.5 kHz to 7.5 kHz (knock 1)	6450 Hz / order 258	6460 Hz / 129	6440 Hz / 64
7.5 kHz to 8.5 kHz (valve)	8050 Hz / 322	7950 Hz / 159	7860 Hz / 79
9.5 kHz to 11.5 kHz (knock 2)	10 520 Hz / 421	10 480 Hz / 210	10 580 Hz / 106

The resonances stay at their frequencies (within the ±150 Hz smoothing and the random variation of the few knock events) while their order halves with every doubling of the speed; the low-frequency lines keep their order and double their frequency.

d) The valve impact excites the same damped 8 kHz oscillation with the same phase every 180°: a periodic repetition of one burst. Periodic repetition with period T_{180} samples the burst spectrum (Lorentzian, 796 Hz wide) at multiples of $1/T_{180}$ = order 2 (50, 100 or 200 Hz spacing) – a line spectrum under a fixed envelope. The knock bursts occur randomly (probability 0.2) with random amplitude and phase: no periodicity, hence no lines but a continuous Lorentzian hump at 6.5 kHz/10.5 kHz. (At 6000 rpm the valve envelope is sampled by only a few lines 200 Hz apart, which together with the knock skirt shifts the smoothed maximum to 7.86 kHz.)
e) The filter for the crank-synchronous vibration must follow the speed (order tracking, or processing in the angle domain); the knock band-pass is fixed in frequency (but its measuring window is defined in crank angle).

Exercise 6.4 Order tracking during a run-up

CHALLENGE

`code/jslinux/sheet06/order_track.c`: the engine accelerates from 1500 rpm to 6000 rpm within 1 s; the knock-sensor signal (no knock) is sampled at 50 kHz, crank edges are time-stamped as in Exercise 4.4.

a) The program computes the amplitude spectrum of the time signal on 0 Hz to 500 Hz. Describe it. Why can no order amplitude be read from it?

b) Resample the signal onto a 0.5° grid (code of 4.4) and compute the "order spectrum" $A(O) = 2|\sum_m x_a[m] \mathrm{e}^{-\mathrm{j}2\pi O m \Delta\theta/360°}|/N_a$ for $O = 0 \ldots 10$ over an integer number of revolutions. Compare the order amplitudes with Exercise 6.3.

c) The angle grid has a "sampling rate" of $360°/\Delta\theta = 720$ samples per revolution. Which orders can be represented, and which signal components alias in the angle domain at 1500 rpm? Why does this not disturb the low orders here?

Solution

a) The 2nd order sweeps from 50 to 200 Hz, the 4th from 100 to 400 Hz, ...: the spectrum is a broad smeared plateau with a maximum of only 11.9 mV (instead of 76 mV); overlapping sweeps add up between 100 and 200 Hz. The energy of each order is distributed over the whole frequency range it passes; the DTFT assumes stationarity.
b)

```
    for (k = 2; k < ne; k++) {
        double d1 = te[k] - te[k - 1], d0 = te[k - 1] - te[k - 2];
        if (kref < 0) { if (d1 > 2.0 * d0) { kref = k; the[k] = 0.0; } }
        else the[k] = the[k - 1] + (d1 > 2.0 * d0 ? 18.0 : 6.0);
    }
    {
        int ke = kref, ks = 0;
        while (na < MAXA) {
            double th = na * DTH, tq;
            if (th > the[ne - 1]) break;
            while (ke < ne - 2 && the[ke + 1] < th) ke++;
            tq = te[ke] + (te[ke + 1] - te[ke]) * (th - the[ke]) / (the[ke + 1] - the[ke]);
            if (tq >= ts[NS - 1]) break;
            while (ks < NS - 2 && ts[ks + 1] < tq) ks++;
            xa[na++] = xs[ks] + (xs[ks + 1] - xs[ks]) * (tq - ts[ks]) / (ts[ks + 1] - ts[ks])
    ;
        }
    }
    nrev = (int)(na * DTH / 360.0);              /* whole revolutions only */
    na = (int)(nrev * 360.0 / DTH);
    remove_mean(xa, na);
    for (i = 0; i < NORD; i++) ord[i] = 0.01 * i;
    /* order O = cycles per revolution; angle step DTH -> w = 2 pi O DTH/360 */
    amp_spec(xa, na, ord, DTH / 360.0, ao, NORD);
```

Result: 61 revolutions, 43920 angle samples; order amplitudes $A_2 = 75.5\,\mathrm{mV}$, $A_4 = 53.7\,\mathrm{mV}$, $A_6 = 29.6\,\mathrm{mV}$, $A_8 = 5.6\,\mathrm{mV}$ – sharp lines with the same amplitudes as at constant speed (6.3 c), although the speed quadrupled. Angle-domain resampling turns the non-stationary time signal into a stationary angle signal (*computed order tracking*).
c) Orders up to $O < 360$ are represented. The time signal contains components up to $f_s/2 = 25\,\mathrm{kHz}$, i.e. up to order $25000/25 = 1000$ at 1500 rpm: the valve resonance lies at order 320 (still below 360), but a knock mode at 10.5 kHz would appear at order 420 and alias to $720-420 = 300$, and the broadband sensor noise above order 360 folds back into $[0, 360]$. Aliasing in the angle domain only matters if strong content lies near $720k \pm O$ for the orders O of interest; for $O = 2 \ldots 8$ this is only white noise, which is spread thinly over all orders (the linear interpolation additionally acts as a weak low-pass). For a clean order analysis the time signal must be low-pass filtered (or the grid refined) before resampling – the anti-aliasing rule of Sheet 5 in the angle domain.

Deliverables

6.1: derivations, coefficient values, Lorentzian bandwidths, order table. 6.2: completed programs, coefficient table, Gibbs table with explanation, DTFT table and the checks d)/e). 6.3: completed program, order amplitudes (theory vs. measurement) and resonance table with interpretation. 6.4 (optional): order spectrum vs. time spectrum of the run-up.

Digital Signal Processing Exercise Sheet 7

Laplace Transform

Script: Chapter 7 (Laplace transform, transfer functions, stability) **with solutions**

Learning goals

- Describe engine sensors by transfer functions $H(s)$; relate poles and zeros to decay time, resonance frequency, damping, Q and bandwidth.
- Compute impulse and step responses by partial fractions and decide stability from the pole locations.
- Read and sketch Bode diagrams; judge an RC anti-aliasing filter.
- Handle a transport delay e^{-sT_d} by Padé approximation and understand the consequences of a right-half-plane zero.
- Verify analytic results by numerical ODE integration (Euler, RK4) and understand the stability limit of explicit integrators.
- Map an analog system into the z-domain (forward Euler, impulse invariance, bilinear transform) – a preview of Sheets 9 and 10.

Background

For a causal signal $x(t)$ the (unilateral) Laplace transform is $X(s) = \int_0^\infty x(t)\,\mathrm{e}^{-st}\,\mathrm{d}t$. An LTI system described by a linear ODE with constant coefficients has the *transfer function*

$$H(s) = \frac{Y(s)}{X(s)} = \frac{b_0 s^m + \cdots + b_m}{a_0 s^n + \cdots + a_n} = K\,\frac{\prod_k (s - z_k)}{\prod_k (s - p_k)},$$

the Laplace transform of its impulse response $h(t)$. A causal system is BIBO stable iff all poles satisfy $\mathrm{Re}\, p_k < 0$. On the imaginary axis, $s = \mathrm{j}\omega$, $H(\mathrm{j}\omega)$ is the frequency response.

The knock burst of the virtual engine (`code/common/engine_signals.h`) is

$$x_k(t) = A\,\mathrm{e}^{-t/\tau}\sin(2\pi f_d t)\,\varepsilon(t), \qquad f_d = 6.5\,\mathrm{kHz},\ \tau = 0.8\,\mathrm{ms}$$

(plus a second mode at 10.5 kHz). We interpret it as the impulse response of the mechanical path *combustion chamber* → *engine block* → *piezo sensor*, which is excited by the short pressure shock of knocking combustion. Throughout this sheet we use the normalised second-order model ($\sigma = 1/\tau$, $\omega_d = 2\pi f_d$)

$$H_K(s) = \frac{\omega_0^2}{s^2 + 2\sigma s + \omega_0^2}, \qquad \omega_0^2 = \omega_d^2 + \sigma^2, \qquad \zeta = \sigma/\omega_0. \tag{2}$$

Exercise 7.1 Sensor models in the s-domain

PAPER

a) **Knock resonator.** Show that the impulse response of (2) is $h_K(t) = \frac{\omega_0^2}{\omega_d}\mathrm{e}^{-\sigma t}\sin(\omega_d t)\,\varepsilon(t)$. Compute σ, ω_0 (and f_0), ζ, the quality factor $Q = 1/(2\zeta)$, the poles and the region of convergence. Is the system stable? What would $\sigma < 0$ mean?

b) Compute the **step response** of H_K with partial fractions. Give the overshoot, the time of the first maximum and the 2 % settling time. The DC gain is $H_K(0) = 1$: what does this imply for the slow combustion vibration (a half sine over 60° crank angle) that is also present on the sensor signal?

c) **Bode diagram.** Sketch $|H_K(\mathrm{j}\omega)|$ in dB and the phase over $\log f$ (asymptotes!). Compute the height and frequency of the resonance peak and show that the -3 dB bandwidth is $\Delta\omega \approx 2\sigma$, i.e. $B \approx 1/(\pi\tau)$ in Hz. Interpret this relation between decay time and bandwidth.

d) The second knock mode has $f_d = 10.5\,\text{kHz}$ and the same τ. Mark all four poles of the two-mode model in the s-plane. What distinguishes the poles of the two modes, and what do they have in common?

e) **RC anti-aliasing filter.** The knock signal is sampled at $f_s = 40\,\text{kHz}$ behind an RC low-pass with $R = 1\,\text{k}\Omega$, $C = 10\,\text{nF}$. Give $H_{RC}(s)$, its pole and f_c. Compute the attenuation at 6.5 kHz, 10.5 kHz and at 30 kHz. Onto which frequency does a 30 kHz component alias, and why is that critical? Which f_c would a first-order filter need for 40 dB attenuation at 30 kHz? Compare with a second-order Butterworth filter with the same f_c and conclude. Also give the phase and the group delay of the RC filter at 6.5 kHz.

f) **Lambda sensor with transport delay.** Fuel change → sensor is modelled as $G(s) = \mathrm{e}^{-sT_d}/(1 + sT)$ with $T_d = 200\,\text{ms}$ (gas travel) and $T = 30\,\text{ms}$ (sensor), the values of the Wokwi `lambda-probe` chip.

i. Why is $G(s)$ not a rational function, and why is this a problem for the methods of this chapter?

ii. Derive the Padé(1,1) approximation $\mathrm{e}^{-sT_d} \approx (1 - sT_d/2)/(1 + sT_d/2)$ by matching the Taylor series of both sides as far as possible. Where are poles and zeros of the approximated $G_1(s)$?

iii. Compute the step response of $G_1(s)$ with partial fractions. Determine the minimum (time and value) and explain the *undershoot*. Compare with the exact step response.

iv. Phase error: the exact delay has phase $-\omega T_d$, the Padé term $-2\arctan(\omega T_d/2)$. Up to which frequency is the error below 5°? (Use the series $\arctan x \approx x - x^3/3$.)

Solution

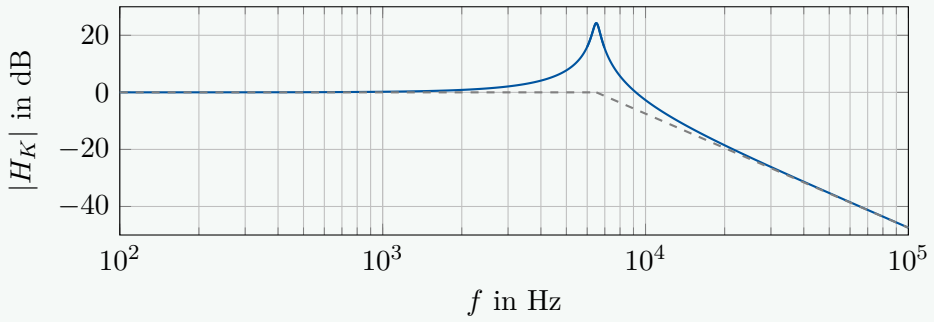
a) With $\mathcal{L}\{\mathrm{e}^{-\sigma t}\sin(\omega_d t)\varepsilon(t)\} = \omega_d/((s+\sigma)^2 + \omega_d^2)$ and $(s+\sigma)^2 + \omega_d^2 = s^2 + 2\sigma s + \sigma^2 + \omega_d^2 = s^2 + 2\sigma s + \omega_0^2$ we get $\mathcal{L}\{h_K\} = \frac{\omega_0^2}{\omega_d} \cdot \frac{\omega_d}{s^2+2\sigma s+\omega_0^2} = H_K(s)$. Numbers: $\sigma = 1/\tau = 1250\,\text{s}^{-1}$, $\omega_d = 40\,840.7\,\text{rad/s}$, $\omega_0 = 40\,859.8\,\text{rad/s}$ ($f_0 = 6503.0\,\text{Hz}$), $\zeta = 0.0306$, $Q = 16.3$. Poles $p_{1,2} = -\sigma \pm \mathrm{j}\omega_d = -1250 \pm \mathrm{j}\,40841\,\text{s}^{-1}$, ROC $\operatorname{Re} s > -1250\,\text{s}^{-1}$. It contains the imaginary axis: the causal system is stable. $\sigma < 0$ means poles in the right half plane: the oscillation grows like $\mathrm{e}^{|\sigma|t}$ (unstable).

b) $\frac{H_K(s)}{s} = \frac{1}{s} - \frac{s+2\sigma}{(s+\sigma)^2+\omega_d^2} = \frac{1}{s} - \frac{s+\sigma}{(s+\sigma)^2+\omega_d^2} - \frac{\sigma}{\omega_d}\frac{\omega_d}{(s+\sigma)^2+\omega_d^2}$, hence

$$y_\varepsilon(t) = \Big[1 - \mathrm{e}^{-\sigma t}\Big(\cos\omega_d t + \frac{\sigma}{\omega_d}\sin\omega_d t\Big)\Big]\varepsilon(t).$$

First maximum at $t_p = \pi/\omega_d = 76.9\,\mu\text{s}$, overshoot $\mathrm{e}^{-\pi\sigma/\omega_d} = \mathrm{e}^{-0.0962} = 0.908$, i.e. **90.8 %**; 2 % settling time $\approx 4/\sigma = 3.2\,\text{ms}$. Because $H_K(0) = 1$ the slow combustion vibration (half sine of 3.3 ms at 3000 rpm, i.e. some 100 Hz) passes unattenuated – the resonator alone is no knock filter; a band-pass (Sheet 10) or at least a high-pass is required.

c) Asymptotes: 0 dB below f_0, −40 dB/decade above f_0 (two poles); phase 0° → −180°, −90° at f_0, the transition takes place within $\approx \pm B/2$ around f_0. Peak: $|H|_{\max} = 1/(2\zeta\sqrt{1-\zeta^2}) = 16.35$ (24.27 dB) at $f_r = f_0\sqrt{1-2\zeta^2} = 6497\,\text{Hz}$. Near the pole p_1, $|H| \propto 1/|\mathrm{j}\omega - p_1| = 1/\sqrt{\sigma^2 + (\omega-\omega_d)^2}$, which drops by $\sqrt{2}$ for $|\omega-\omega_d| = \sigma$: $\Delta\omega = 2\sigma$, $B = \sigma/\pi = 1/(\pi\tau) = 398\,\text{Hz}$. A short burst (small τ) necessarily has a wide spectrum and vice versa (time–bandwidth product $B\tau = 1/\pi$). Numerical check in Exercise 7.2: $B = 398.3\,\text{Hz}$.



d) Mode 2: $p_{3,4} = -1250 \pm \mathrm{j}\,65973\,\mathrm{s}^{-1}$. Same real part (same decay time τ, i.e. same distance from the imaginary axis, same bandwidth 398 Hz), but larger imaginary part (frequency); its damping ratio $\zeta_2 = \sigma/\omega_{0,2} = 0.019$ and $Q_2 = 26$ are therefore different.
e) $H_{RC}(s) = 1/(1 + sRC)$, $RC = 10\,\mu\mathrm{s}$, pole $s = -10^5\,\mathrm{s}^{-1}$, $f_c = 1/(2\pi RC) = 15.9\,\mathrm{kHz}$. $|H|$: $-0.67\,\mathrm{dB}$ at 6.5 kHz, $-1.57\,\mathrm{dB}$ at 10.5 kHz, $-6.58\,\mathrm{dB}$ at 30 kHz. At $f_s = 40\,\mathrm{kHz}$ a 30 kHz component aliases to $40-30 = 10\,\mathrm{kHz}$, right next to the second knock mode, and is attenuated by a factor of only 0.47. For 40 dB: $\sqrt{1 + (30/f_c)^2} = 100 \Rightarrow f_c \approx 300\,\mathrm{Hz}$ – that would also destroy the knock band. A second-order Butterworth with $f_c = 15.9\,\mathrm{kHz}$ gives $-10\log_{10}(1 + (f/f_c)^4)$: $-0.12\,\mathrm{dB}$ at 6.5 kHz and $-11.3\,\mathrm{dB}$ at 30 kHz; still not enough. Conclusion: either a higher-order analog filter or (better) oversampling with a digital decimation filter. Phase at 6.5 kHz: $-\arctan(\omega RC) = -22.2°$; group delay $\tau_g = RC/(1 + (\omega RC)^2) = 8.6\,\mu\mathrm{s}$ ($\approx 0.15°$ crank angle at 3000 rpm – negligible).
f) i. $\mathcal{L}\{x(t - T_d)\} = \mathrm{e}^{-sT_d}X(s)$ is a transcendental function of s; it cannot be written as a ratio of polynomials, so there are no finite sets of poles and zeros, no partial fraction expansion and no finite-dimensional ODE / state-space model (the "state" of a delay is the complete input history of length T_d).
ii. $\mathrm{e}^{-x} = 1 - x + \frac{x^2}{2} - \frac{x^3}{6} + \ldots$ and $\frac{1-ax}{1+ax} = 1 - 2ax + 2a^2x^2 - 2a^3x^3 + \ldots$; matching the linear and the quadratic term gives $a = \frac{1}{2}$ (the error starts with $x^3/12$). $G_1(s) = \frac{1-0.1s}{(1+0.1s)(1+0.03s)}$: poles $s = -10$ and $s = -33.3\,\mathrm{s}^{-1}$, zero $s = +10\,\mathrm{s}^{-1}$ in the right half plane (non-minimum phase, all-pass factor).
iii. $\frac{G_1(s)}{s} = \frac{1-0.1s}{0.003\,s(s+10)(s+100/3)} = \frac{1}{s} - \frac{20/7}{s+10} + \frac{13/7}{s+100/3}$:

$$y(t) = 1 - \tfrac{20}{7}\mathrm{e}^{-10t} + \tfrac{13}{7}\mathrm{e}^{-100t/3} \quad (t \text{ in s}).$$

$y(0) = 0$, $\dot{y}(0) = \frac{200}{7} - \frac{1300}{21} = -33.3\,\mathrm{s}^{-1} < 0$: the response starts in the *wrong direction* (typical of an RHP zero). $\dot{y} = 0$ for $\mathrm{e}^{70t/3} = 13/6$, i.e. $t = 33.1\,\mathrm{ms}$, where $y_{\min} = -0.436$. The exact response is 0 for $t < T_d$ and $1 - \mathrm{e}^{-(t-T_d)/T}$ afterwards; the approximation reaches 0.5 already after 173 ms instead of 221 ms (Exercise 7.3).
iv. $\Delta\varphi = \omega T_d - 2\arctan(\omega T_d/2) \approx (\omega T_d)^3/12$. $\Delta\varphi = 5° = 0.0873$ gives $\omega T_d \approx 1.02$; the exact numerical solution (Exercise 7.3) is $\omega T_d = 1.069$, i.e. $f = 0.85\,\mathrm{Hz}$. Padé(1,1) is fine for controller design at loop frequencies below $\approx 0.5\,\mathrm{Hz}$, useless for the time response around $t = T_d$.

Exercise 7.2 Knock resonator: ODE solvers vs. Laplace JSLINUX

Program `code/jslinux/sheet07/resonator_ode.c` integrates the ODE $\ddot{y} + 2\sigma\dot{y} + \omega_0^2 y = \omega_0^2 u$ of (2) in state-space form $x = (y, \dot{y})^T$.

a) Implement `rhs()`, one step of the forward Euler method and one step of the classical Runge–Kutta method (RK4).

b) Implement the analytic impulse and step responses of Exercise 7.1. The program simulates the impulse response (initial state $y(0) = 0$, $\dot{y}(0) = \omega_0^2$ – why?) and the step response for step sizes $h = 40, 20, \ldots, 0.625\,\mu\mathrm{s}$ and prints the maximum error. Estimate the order of convergence p

from successive errors ($e(h) \propto h^p$).

c) Explain the results of the Euler method: derive the stability condition $|1 + h\lambda| < 1$ for $\dot{x} = \lambda x$ and evaluate it for the poles of H_K. Compare the limit with the sampling period of the engine chip (20 µs). Why does the chip compute the burst from the closed-form solution instead of integrating an ODE?

d) Implement `freq_resp()` and compare peak, resonance frequency and −3 dB bandwidth with your results of Exercise 7.1 c).

Solution

a)/b) Key part of `code/solutions/jslinux/sheet07/resonator_ode.c`:

```c
static void rhs(const double *x, double u, double *dx)
{
    dx[0] = x[1];
    dx[1] = -w0 * w0 * x[0] - 2.0 * sig * x[1] + w0 * w0 * u;
}

static void euler_step(double *x, double u, double h)
{
    double k[2];
    rhs(x, u, k);
    x[0] += h * k[0];
    x[1] += h * k[1];
}

static void rk4_step(double *x, double u, double h)
{
    double k1[2], k2[2], k3[2], k4[2], xt[2];
    int i;
    rhs(x, u, k1);
    for (i = 0; i < 2; i++) xt[i] = x[i] + 0.5 * h * k1[i];
    rhs(xt, u, k2);
    for (i = 0; i < 2; i++) xt[i] = x[i] + 0.5 * h * k2[i];
    rhs(xt, u, k3);
    for (i = 0; i < 2; i++) xt[i] = x[i] + h * k3[i];
    rhs(xt, u, k4);
    for (i = 0; i < 2; i++)
        x[i] += h / 6.0 * (k1[i] + 2.0 * k2[i] + 2.0 * k3[i] + k4[i]);
}
```

The input $\omega_0^2\delta(t)$ integrated over $[0^-, 0^+]$ changes $\dot{y}$ by ω_0^2 while y stays continuous, hence the initial state. Program output (maximum error relative to the peak of the exact response, 0–4 ms):

h [µs]	40	20	10	5	2.5	1.25	0.625
Euler	$3.1 \cdot 10^{27}$	$7.8 \cdot 10^{20}$	$2.9 \cdot 10^{11}$	$1.0 \cdot 10^{5}$	28.7	0.613	0.206
RK4	0.361	$4.6 \cdot 10^{-2}$	$3.0 \cdot 10^{-3}$	$1.8 \cdot 10^{-4}$	$1.1 \cdot 10^{-5}$	$7.2 \cdot 10^{-7}$	$4.5 \cdot 10^{-8}$
RK4 order p	–	2.97	3.96	4.00	4.01	4.00	4.00

RK4 converges with order 4 (error ratio 16 per halving of h) once $\omega_0 h \lesssim 0.4$; at $h = 20$ µs (50 samples/ms) the error is 4.6 %. The step response behaves the same way (RK4: $2.3 \cdot 10^{-2}$ at 20 µs, $9.2 \cdot 10^{-5}$ at 5 µs). The Euler method diverges for $h \geq 2.5$ µs; for $h = 1.25$ and 0.625 µs it is stable but still has 61 % and 21 % error (the observed order 1.57 is still pre-asymptotic, theoretically $p = 1$).

c) Euler applied to $\dot{x} = \lambda x$: $x_{n+1} = (1 + h\lambda)x_n$, stable iff $|1 + h\lambda| < 1$ (a circle of radius 1 around -1 in the $h\lambda$-plane). With $\lambda = -\sigma \pm \mathrm{j}\omega_d$: $|1 + h\lambda|^2 = 1 - 2h\sigma + h^2\omega_0^2 < 1 \iff h < 2\sigma/\omega_0^2 = 1.497$ µs. For lightly damped poles close to the imaginary axis the forward Euler method is unstable for any reasonable step – the chip samples every 20 µs, more than 13 times this limit. For the poles of H_K, RK4 is stable up to $h\omega_0 \approx 2.8$ ($h \approx 69$ µs), but still inaccurate. The chip

therefore evaluates the closed-form solution $\mathrm{e}^{-t/\tau}\sin(\omega_d t)$ which is exact for every step size – this is precisely what the inverse Laplace transform gives us.
d) Peak 24.27 dB at 6497.0 Hz, −3 dB band 6294.8–6693.1 Hz, $B = 398.3$ Hz ($\sigma/\pi = 397.9$ Hz). Selected values: 0.21 dB at 1 kHz, −4.14 dB at 10.5 kHz, −18.55 dB at 20 kHz, −47.4 dB at 100 kHz (one decade above f_0 about −40 dB plus the small offset of the asymptote); phase −89.1° at 6.5 kHz, −20.8° at 6 kHz, −157.5° at 7 kHz.

Exercise 7.3 Lambda sensor: delay and Padé approximation

JSLINUX

Program `code/jslinux/sheet07/lambda_pade.c` compares the exact step response of $G(s) = \mathrm{e}^{-sT_d}/(1+sT)$ with three rational approximations of the delay: a first-order lag $1/(1+sT_d)$, Padé(1,1) and Padé(2,2)

$$\mathrm{e}^{-sT_d} \approx \frac{1 - sT_d/2 + s^2T_d^2/12}{1 + sT_d/2 + s^2T_d^2/12}.$$

a) Any strictly proper $G(s) = N(s)/D(s)$ can be simulated in *controllable canonical form*: $D(p)\,v = u$ and $y = N(p)\,v$, where $p = \mathrm{d}/\mathrm{d}t$ and state $(v, \dot v, \dots)$. Implement `ccf_rhs()` and `ccf_out()` (integration by RK4 is given).

b) Implement the exact system by a delay line (ring buffer of $D = T_d/h$ samples) followed by the exact discretisation of the lag, $y_{k+1} = a\,y_k + (1-a)\,u_{k-D}$, $a = \mathrm{e}^{-h/T}$, and the analytic step response. Enter the Padé coefficients. Compare minimum, time of $y = 0.5$, RMS and maximum error of the three approximations.

c) Compare the phase of the delay part. Up to which frequency is the phase error of Padé(1,1) and Padé(2,2) below 5°?

d) Which approximation would you use (i) to design a lambda controller, (ii) to predict the sensor signal in the time domain?

Solution

a)/b) Core of `code/solutions/jslinux/sheet07/lambda_pade.c`:

```c
static void ccf_rhs(const tf_t *g, const double *x, double u, double *dx)
{
    int i, n = g->n;
    double s = u;
    for (i = 0; i < n - 1; i++) dx[i] = x[i + 1];
    for (i = 0; i < n; i++) s -= g->a[n - i] * x[i];
    dx[n - 1] = s / g->a[0];
}

static double ccf_out(const tf_t *g, const double *x)
{
    int i, m = g->m;
    double y = 0.0;
    for (i = 0; i <= m; i++) y += g->b[m - i] * x[i];   /* b_m x1 + ... */
    return y;
}
```

The delay line reproduces the analytic response to $5.6 \cdot 10^{-15}$. Results (0–1 s, $h = 0.1$ ms):

model	min y	$t_{\min}$ [ms]	t_{50} [ms]	RMS error	max. error
lag $1/(1+sT_d)$	0.000	–	170.9	0.183	0.567
Padé(1,1)	−0.436	33.1	173.2	0.172	0.616
Padé(2,2)	−0.272	88.8	207.7	0.104	0.448
exact	0	–	220.8	–	–

Padé(1,1) confirms the analytic minimum −0.436 at 33.1 ms of Exercise 7.1. Padé(2,2) (poles

$-15 \pm$j8.66, zeros $+15 \pm$j8.66) has a smaller, oscillating undershoot and a better t_{50}. No rational approximation can produce an output that is exactly zero for $0 < t < T_d$.
c) Phase of the delay part in degrees:

f [Hz]	exact	lag	Padé(1,1)	Padé(2,2)
0.1	−7.2	−7.2	−7.2	−7.2
0.5	−36.0	−32.1	−34.9	−36.0
1.0	−72.0	−51.5	−64.3	−71.8
2.0	−144.0	−68.3	−103.0	−138.7

Error $> 5°$ above 0.85 Hz ($\omega T_d = 1.07$) for Padé(1,1) and above 1.97 Hz ($\omega T_d = 2.48$) for Padé(2,2). The lag has the correct low-frequency phase slope but its phase is bounded by $-90°$.
d) (i) Controller design (stability margins, Sheet 2 limit cycle) needs the correct *phase* near the crossover frequency; Padé(2,2) (or (1,1) if the crossover is well below 0.8 Hz) is appropriate. (ii) For time-domain prediction one should keep the delay exact (delay line, Smith predictor); every rational approximation produces a non-physical undershoot.

Exercise 7.4 From s to z: discretisation of analog systems

JSLINUX

A digital filter shall imitate $H_K(s)$ and $H_{RC}(s)$ at $f_s = 40\,\text{kHz}$, $T = 1/f_s$. Three classical mappings are

$$\text{forward Euler: } s = \frac{z-1}{T}, \qquad \text{bilinear: } s = \frac{2}{T}\frac{z-1}{z+1},$$
$$\text{impulse invariance: } h[n] = T\,h(nT) \quad (\text{poles } z_k = \mathrm{e}^{p_k T}).$$

a) **(paper)** Into which region of the z-plane is the left half s-plane mapped by each method, and onto which curve the imaginary axis? Which method can turn a stable analog system into an unstable digital one? Show that the bilinear transform maps ω_a onto $\Omega = 2\arctan(\omega_a T/2)$ (*frequency warping*).

b) Complete the three mapping functions of the program `code/jslinux/sheet07/discretise.c`, i.e. `map_euler`, `map_bilin` and `impinv` (second-order section $H(s) = (b_0 s^2 + b_1 s + b_2)/(a_0 s^2 + a_1 s + a_2)$; impulse invariance via partial fractions with the small complex struct provided).

c) *Prewarping*: with $s = K\frac{z-1}{z+1}$, $K = \omega_r/\tan(\omega_r T/2)$, the frequency ω_r is mapped exactly. Use $\omega_r = \omega_0$ for the resonator and $\omega_r = \omega_c$ for the RC filter.

d) Run the program and interpret the tables: pole radius and angle, peak frequency, gain at DC and close to $f_s/2$. Which method would you choose for the knock resonator, which for the anti-aliasing filter model?

Solution

a) Forward Euler: $z = 1 + sT$; the left half plane becomes the half plane $\operatorname{Re} z < 1$, the jω axis the vertical line $\operatorname{Re} z = 1$. Stable poles outside the unit circle become unstable (exactly the stability condition of Exercise 7.2). Impulse invariance: $z = \mathrm{e}^{sT}$, LHP $\to$ interior of the unit circle (stable stays stable), but the map is not one-to-one: all $s + \mathrm{j}k\,2\pi/T$ map to the same $z \Rightarrow$ aliasing of the frequency response. Bilinear: $z = (1 + sT/2)/(1 - sT/2)$ maps the LHP one-to-one onto the unit disk and the jω axis onto the unit circle; with $z = \mathrm{e}^{\mathrm{j}\Omega}$: $\frac{2}{T}\frac{\mathrm{e}^{\mathrm{j}\Omega}-1}{\mathrm{e}^{\mathrm{j}\Omega}+1} = \mathrm{j}\frac{2}{T}\tan\frac{\Omega}{2} = \mathrm{j}\omega_a$, hence $\Omega = 2\arctan(\omega_a T/2)$; the whole axis $0 \le \omega_a < \infty$ is compressed into $0 \le \Omega < \pi$.
b) Key part of the solution:

```
static void map_euler(const double *c, double T, double *C)
{
    C[0] = c[0] / (T * T);
    C[1] = -2.0 * c[0] / (T * T) + c[1] / T;
    C[2] = c[0] / (T * T) - c[1] / T + c[2];
```

```
}
```

```
static void map_bilin(const double *c, double K, double *C)
{
    C[0] = c[0] * K * K + c[1] * K + c[2];
    C[1] = -2.0 * c[0] * K * K + 2.0 * c[2];
    C[2] = c[0] * K * K - c[1] * K + c[2];
}
```

```
static dig_t impinv(const ana_t *h, double T)
{
    dig_t d;
    if (h->a[0] == 0.0) {            /* first order b2 / (a1 s + a2) */
        double p = -h->a[2] / h->a[1], r = h->b[2] / h->a[1];
        d.B[0] = T * r; d.B[1] = 0; d.B[2] = 0;
        d.A[0] = 1.0; d.A[1] = -exp(p * T); d.A[2] = 0;
    } else {                         /* second order, distinct poles */
        cpx disc = cx(h->a[1] * h->a[1] - 4 * h->a[0] * h->a[2], 0);
        cpx sq = csqrt_(disc);
        cpx p1 = cscale(cadd(cx(-h->a[1], 0), sq), 0.5 / h->a[0]);
        cpx p2 = cscale(csub(cx(-h->a[1], 0), sq), 0.5 / h->a[0]);
        /* residue r_i = N(p_i) / D'(p_i), D'(s) = 2 a0 s + a1 */
        cpx N1 = cadd(cadd(cscale(cmul(p1, p1), h->b[0]), cscale(p1, h->b[1])), cx(h->b[2],
    0));
        cpx N2 = cadd(cadd(cscale(cmul(p2, p2), h->b[0]), cscale(p2, h->b[1])), cx(h->b[2],
    0));
        cpx r1 = cdiv(N1, cadd(cscale(p1, 2 * h->a[0]), cx(h->a[1], 0)));
        cpx r2 = cdiv(N2, cadd(cscale(p2, 2 * h->a[0]), cx(h->a[1], 0)));
        cpx q1 = cexpo(cscale(p1, T)), q2 = cexpo(cscale(p2, T));
        /* T [ r1 (1 - q2 z^-1) + r2 (1 - q1 z^-1) ] / [(1 - q1 z^-1)(1 - q2 z^-1)] */
        d.B[0] = T * cadd(r1, r2).re;
        d.B[1] = -T * cadd(cmul(r1, q2), cmul(r2, q1)).re;
        d.B[2] = 0.0;
        d.A[0] = 1.0;
        d.A[1] = -cadd(q1, q2).re;
        d.A[2] = cmul(q1, q2).re;
    }
    return d;
}
```

c)/d) Program output, knock resonator (exact pole map: $|z| = \mathrm{e}^{-\sigma T} = 0.96923$ at 6500 Hz):

method	$\lvert p\rvert$	$\arg p$ [Hz]	peak [Hz]	$\lvert H\rvert$ at 0	at 6.5 kHz	at 15 kHz
analog	–	–	6497	0.00 dB	24.27 dB	−12.72 dB
forward Euler	1.4075	5167	(4801)	0.00	1.11	−11.90
impulse invariance	0.9692	6500	6498	−0.80	24.27	−9.02
bilinear	0.9755	6011	6008	0.00	13.43	−26.59
bilinear, prewarped	0.9742	6502	6498	0.00	24.27	−24.89

Coefficients: impulse invariance $H(z) = 0.844567\,z^{-1}/(1 - 1.012846z^{-1} + 0.939413z^{-2})$, bilinear $H(z) = 0.201889(1 + z^{-1})^2/(1 - 1.144074z^{-1} + 0.951630z^{-2})$. Forward Euler produces poles at $|z| = 1.41$: **unstable** (its "frequency response" is a formal evaluation without meaning). Impulse invariance keeps the resonance perfectly (pole angle exactly 6500 Hz, peak 24.27 dB) but aliasing raises the response far from the peak (−0.8 dB at DC, −9.0 dB instead of −12.7 dB at 15 kHz). The plain bilinear transform warps the resonance to $f_s/\pi \cdot \arctan(\pi f_0/f_s) = 6.01$ kHz – 500 Hz off, more than the bandwidth, so at 6.5 kHz only 13.4 dB instead of 24.3 dB remain. Prewarping at f_0 moves the pole back to 6502 Hz. The bilinear methods have a double zero at $z = -1$ ($|H| \to -\infty$ at $f_s/2$).
RC low-pass ($f_c = 15.9$ kHz, $\omega_c T = 2.5$):

method	pole z	$\lvert H\rvert$ at 0	6.5 kHz	15.9 kHz	20 kHz
analog	–	0.00 dB	−0.67 dB	−3.01 dB	−4.11 dB
forward Euler	−1.50 (unstable)	0.00	1.13	8.68	13.98
impulse invariance	0.0821	8.70	8.32	7.40	7.27
bilinear	−0.111	0.00	−0.79	−8.32	−80
bilinear, prewarped	−0.501	0.00	−0.15	−3.01	−73

For a filter whose response does not vanish below $f_s/2$, impulse invariance is useless: the DC gain is $\omega_c T/(1-\mathrm{e}^{-\omega_c T}) = 2.72$ (8.7 dB), the sum of all aliased copies. Choice: for the knock resonator (narrow-band, $f_0 \ll f_s/2$) impulse invariance or prewarped bilinear; for the RC model (and any low-/high-pass or band-stop) the prewarped bilinear transform. This is the standard IIR design route of Sheet 10.

Deliverables

Paper solution of 7.1 including the Bode sketch; the error table and the order estimate of 7.2 with the Euler stability limit; the step-response table of 7.3 and your recommendation 7.3 d); the comparison tables of 7.4 with a short justification of your choice of discretisation method.

Digital Signal Processing Exercise Sheet 8

DFT and FFT

Script: Chapter 8 (discrete Fourier transform, FFT algorithms) **with solutions**

Learning goals

- Compute small DFTs by hand and use the DFT properties (periodicity, symmetry for real signals, Parseval).
- Distinguish frequency resolution $\Delta f = f_s/N$ (record length) from the bin spacing obtained by zero padding.
- Explain leakage and choose a window by main-lobe width, side-lobe level, scalloping loss and equivalent noise bandwidth.
- Implement an iterative radix-2 FFT and verify its $\mathcal{O}(N \log N)$ complexity against the $\mathcal{O}(N^2)$ direct DFT.
- Compute a crank-angle-triggered knock spectrum on the ESP32 and judge the memory and operation budget of an 8-bit Uno.

Background

The N-point DFT of $x[n]$, $n = 0, \ldots, N-1$, and its inverse are

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{kn}, \qquad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, W_N^{-kn}, \qquad W_N = \mathrm{e}^{-\mathrm{j}2\pi/N}.$$

$X[k]$ samples the DTFT of the *finite record* at $f_k = k\, f_s/N$. Multiplying by a window $w[n]$ before the DFT convolves the spectrum with the window transform: a sinusoid between two bins leaks into all bins (*leakage*). The radix-2 decimation-in-time FFT splits an N-point DFT into two $N/2$-point DFTs of the even and odd samples,

$$X[k] = G[k] + W_N^k H[k], \qquad X[k + N/2] = G[k] - W_N^k H[k],$$

recursively, giving $\log_2 N$ stages of $N/2$ *butterflies*.

The FFT written on this sheet is used again on Sheets 11 and 12. Its interface is fixed (file `fft.h`):

```
void fft(double *re, double *im, int n, int inverse);
/* in place, n = power of two; inverse = 1 includes the factor 1/n */
```

Exercise 8.1 DFT properties

PAPER

a) Write down the 4×4 DFT matrix and compute the DFT of $x = \{1, 2, 0, -1\}$. Check Parseval's relation $\sum |x[n]|^2 = \frac{1}{N} \sum |X[k]|^2$.

b) Compute the 8-point DFT of $x = \{1, 1, 1, 1, 0, 0, 0, 0\}$ in closed form (geometric series) and give $|X[k]|$. Which symmetry do you observe and why must it hold?

c) **Resolution.** The knock measurement window extends from 10° to 70° after TDC, $f_s = 40\,\mathrm{kHz}$. How many samples does it contain at 3000 rpm and at 6000 rpm, and what is the frequency resolution Δf? Can a knock component at 6.5 kHz be separated from the valve impact at 8 kHz (with a Hann window)? Compare Δf with the natural line width $B = 1/(\pi\tau)$ of the knock burst (Sheet 7).

d) The window of c) at 3000 rpm is zero padded to $N = 256$. What changes in the spectrum, what does not? What *does* improve the resolution?

e) **Windows.** Give main-lobe width (null to null, in bins) and highest side lobe for the rectangular, Hann, Hamming and Blackman windows. The knock component at 6.5 kHz is 60 dB stronger than a valve component at 8 kHz (9.6 bins away for $N = 256$). Which windows can reveal the weak component?

f) **Complexity.** Count complex multiplications of the direct DFT and of the radix-2 FFT for $N = 256$. At 6000 rpm, how many 256-point FFTs per second does an angle-synchronous knock detection for a 4-cylinder engine need, and how many real multiplications per second is that?

Solution

a) $W_4 = -\mathrm{j}$:

$$\begin{pmatrix} X_0 \\ X_1 \\ X_2 \\ X_3 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -\mathrm{j} & -1 & \mathrm{j} \\ 1 & -1 & 1 & -1 \\ 1 & \mathrm{j} & -1 & -\mathrm{j} \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 0 \\ -1 \end{pmatrix} = \begin{pmatrix} 2 \\ 1-3\mathrm{j} \\ 0 \\ 1+3\mathrm{j} \end{pmatrix}.$$

Parseval: $1 + 4 + 0 + 1 = 6$ and $(4 + 10 + 0 + 10)/4 = 6$.

b) $X[k] = \sum_{n=0}^{3} W_8^{kn} = \frac{1-W_8^{4k}}{1-W_8^{k}} = \mathrm{e}^{-\mathrm{j}3\pi k/8} \frac{\sin(\pi k/2)}{\sin(\pi k/8)}$ ($k \neq 0$), $X[0] = 4$. $X = \{4,\ 1-2.414\mathrm{j},\ 0,\ 1-0.414\mathrm{j},\ 0,\ 1+0.414\mathrm{j},\ 0,\ 1+2.414\mathrm{j}\}$, $|X| = \{4, 2.613, 0, 1.082, 0, 1.082, 0, 2.613\}$ (confirmed by `fft_test`). For real x: $X[N-k] = X^*[k]$ (conjugate symmetry), $|X|$ is even around $N/2$; $X[0]$ and $X[N/2]$ are real. Only $N/2 + 1$ bins carry information.

c) At 3000 rpm: 18 000 °/s, 60°≙3.33 ms, $N = 133$ samples, $\Delta f = 1/T_w = 300$ Hz. At 6000 rpm: 1.67 ms, $N = 67$, $\Delta f = 600$ Hz. The distance 1.5 kHz is 5 bins (3000 rpm) resp. 2.5 bins (6000 rpm); with the Hann main lobe (half width 2 bins) both are separable at 3000 rpm, only marginally at 6000 rpm. The natural line width of the burst is 398 Hz, i.e. at 3000 rpm the window already resolves the physical line; at high speed the window, not the physics, limits the resolution.

d) Zero padding to 256 samples evaluates the same DTFT of the 133-sample record on a finer grid (156 Hz spacing): peaks are located more accurately and look smoother, but two lines closer than $\approx 2\Delta f$ remain unresolved and the main lobe keeps its width. Only a longer record (more crank angle, lower speed) or a parametric method (Sheet 11) improves the resolution.

e)

window	main lobe (null–null)	highest side lobe
rectangular	2 bins	−13.3 dB
Hann	4 bins	−31.5 dB
Hamming	4 bins	−42.7 dB
Blackman	6 bins	−58.1 dB

At 9.6 bins distance the leakage of the strong line must stay below ≈ -60 dB. Rectangular (≈ -31 dB there) and Hamming (side lobes nearly flat at $-43 \cdots -50$ dB) fail; Hann (side lobes decay with 18 dB/octave) and Blackman succeed. Exercise 8.3 confirms this numerically.

f) DFT: $N^2 = 65536$ complex multiplications; FFT: $\frac{N}{2}\log_2 N = 1024$ butterflies with one complex multiplication each (64 times fewer; many are trivial, $W^0 = 1$). 6000 rpm: 50 cycles/s × 4 cylinders = 200 windows/s, i.e. $200 \cdot 1024 \cdot 4 \approx 8.2 \cdot 10^5$ real multiplications per second (plus $1.2 \cdot 10^6$ additions) – easy for an ESP32, impossible with a direct DFT on a small ECU ($5.2 \cdot 10^7$ mult/s).

Exercise 8.2 Own FFT

JSLINUX

Files: `code/jslinux/sheet08/fft.h` (template of the FFT header)
`code/jslinux/sheet08/fft_test.c` (test program)

a) Implement the direct DFT `dft()` in `fft_test.c`. Avoid N^2 calls of `sin`/`cos`: tabulate W_N^m, $m = 0..N-1$, and use the index $m = (kn) \bmod N$.

b) Implement `fft()` in `fft.h`: bit-reversal permutation, $\log_2 N$ butterfly stages, $1/N$ scaling for the inverse. Compute the twiddle factors of a stage by the recursion $W_L^{k+1} = W_L^k W_L$ (one `sin`/`cos` pair per stage). The program compares FFT and DFT for random complex data, checks IFFT(FFT x) $= x$, Parseval, and conjugate symmetry. Also check your results of Exercise 8.1 a) and b).

c) Implement `fit_exponent()` (least squares in log–log coordinates). Measure the runtime of DFT and FFT versus N and determine the exponents. Plot $t_{\mathrm{FFT}}/(N \log_2 N)$ – is it constant? Explain deviations for small and for large N.

d) How does the round-off error of the FFT grow with N? Why does the twiddle recursion increase it slightly, and how could one avoid that?

Solution

b) The reference implementation (`code/solutions/jslinux/sheet08/fft.h`):

```c
static void fft(double *re, double *im, int n, int inverse)
{
    int i, j, k, L, half, bit;
    double sgn = inverse ? 1.0 : -1.0;

    if (!fft_ispow2(n) || n < 2) return;

    /* 1) bit-reversal permutation */
    for (i = 1, j = 0; i < n; i++) {
        bit = n >> 1;
        while (j & bit) { j ^= bit; bit >>= 1; }
        j |= bit;
        if (i < j) {
            double t;
            t = re[i]; re[i] = re[j]; re[j] = t;
            t = im[i]; im[i] = im[j]; im[j] = t;
        }
    }

    /* 2) butterflies, stage by stage: L = length of the sub-DFTs */
    for (L = 2; L <= n; L <<= 1) {
        double ang = sgn * 2.0 * M_PI / L;
        double cr = cos(ang), ci = sin(ang);     /* W_L = e^{-+j 2 pi / L} */
        half = L >> 1;
        for (i = 0; i < n; i += L) {             /* every sub-DFT block    */
            double wr = 1.0, wi = 0.0, t;        /* W_L^0                  */
            for (k = 0; k < half; k++) {
                int a = i + k, b = a + half;
                double tr = wr * re[b] - wi * im[b];   /* W b */
                double ti = wr * im[b] + wi * re[b];
                re[b] = re[a] - tr;  im[b] = im[a] - ti;
                re[a] += tr;         im[a] += ti;
                t  = wr * cr - wi * ci;          /* W_L^{k+1} = W_L^k W_L  */
                wi = wr * ci + wi * cr;
                wr = t;
            }
        }
    }

    /* 3) scaling of the inverse transform */
    if (inverse) {
        double s = 1.0 / n;
        for (i = 0; i < n; i++) { re[i] *= s; im[i] *= s; }
    }
}
```

Output: `fft` reproduces $X = \{2, 1-3\mathrm{j}, 0, 1+3\mathrm{j}\}$ and the 8-point result of 8.1 b). Comparison

with the DFT (random complex input):

N	$\max \lvert X_{\mathrm{FFT}} - X_{\mathrm{DFT}} \rvert$	$\max \lvert \mathrm{IFFT}(\mathrm{FFT}x) - x \rvert$	Parseval rel. error
8	$9.9 \cdot 10^{-16}$	$2.5 \cdot 10^{-16}$	$2.0 \cdot 10^{-16}$
128	$2.2 \cdot 10^{-14}$	$2.8 \cdot 10^{-15}$	$1.0 \cdot 10^{-15}$
2048	$1.5 \cdot 10^{-12}$	$3.6 \cdot 10^{-14}$	$4.5 \cdot 10^{-15}$

Real input ($N = 64$): $\max \lvert X[k] - X^*[N-k] \rvert = 7.3 \cdot 10^{-15}$, $\operatorname{Im} X[0] = \operatorname{Im} X[N/2] = 0$.
c) Measured on the reference PC (`gcc -O2`; JSLinux is about 50 times slower):

N	DFT [ms]	FFT [ms]	speed-up	$t_{\mathrm{FFT}}/(N \log_2 N)$ [ns]
16	0.0003	0.00016	2.0	2.46
128	0.0210	0.00132	15.9	1.47
1024	1.413	0.0149	94.6	1.46
2048	6.339	0.0307	206.5	1.36
8192	–	0.178	–	1.67
32768	–	0.863	–	1.76

Fitted exponents: DFT $t \propto N^{2.02}$, FFT $t \propto N^{1.16}$ (a power-law fit to $N \log_2 N$ over $N = 64 \ldots 32768$ gives an exponent slightly above 1, since $\mathrm{d}\ln(N \log N)/\mathrm{d}\ln N = 1 + 1/\ln N$). $t/(N \log_2 N)$ is almost constant (1.4–1.8 ns): for small N the fixed overhead (call, bit reversal, `sin`/`cos` per stage) dominates; for $N \geq 8192$ the data (16 B per complex sample, 128 kB) no longer fit into the L1/L2 cache and memory access becomes the bottleneck. Absolute times differ between machines, the exponents do not.
d) The error grows only slowly (roughly $\propto \log N$ per output for an ideal FFT, compared with $\propto N$ for the naive DFT summation). With the twiddle recursion each W_L^k accumulates k rounding errors, so the error rises to $1.5 \cdot 10^{-12}$ at $N = 2048$ (with exact `sin`/`cos` per k: $3.3 \cdot 10^{-13}$, measured with a first version of the code). Remedies: a precomputed twiddle table (best for repeated FFTs of the same size, as on the ESP32), or re-computing the twiddles exactly every few steps. In single precision (AVR, Exercise 8.4) the recursion error $\approx k \cdot 6 \cdot 10^{-8}$ is still negligible for $N \leq 128$ (inverse round trip error $5.5 \cdot 10^{-6}$).

Exercise 8.3 Windows, leakage and the knock spectrum

JSLINUX

Program `code/jslinux/sheet08/windows.c` uses your `fft.h`. Spectra are scaled as amplitude spectra, $A_k = 2|X[k]|/\sum w[n]$, so that a bin-centred sinusoid of amplitude A appears at $20 \log_{10} A$ dB.

a) Implement the periodic Hann, Hamming and Blackman windows. The program computes coherent gain $\sum w/N$, equivalent noise bandwidth ENBW $= N \sum w^2/(\sum w)^2$, scalloping loss (tone exactly between two bins) and the highest side lobe. Compare with 8.1 e).

b) Leakage: $x[n] = \sin(2\pi\, 6500\, n/f_s) + 10^{-3} \sin(2\pi\, 8000\, n/f_s)$, $N = 256$, $f_s = 40$ kHz. Which windows reveal the weak component? Explain the peak levels with the scalloping loss.

c) Zero padding: two equally strong tones at 6500 Hz and 6900 Hz, records of $N = 64$, 128, 256 samples, each zero padded to 4096 points (Hann). Implement the parabolic interpolation of the peak position $\delta = \frac{1}{2} \frac{A_{k-1} - A_{k+1}}{A_{k-1} - 2A_k + A_{k+1}}$ (dB values). For which record length are the two tones resolved?

d) Knock spectrum: record $N = 256$ samples of `es_knock()` at 40 kHz starting at the TDC of a knocking cylinder (3000 rpm, `knock_prob = 1`), i.e. 0°–115° ATDC, and the window 10°–68° ATDC (128 samples). Identify the peaks. Why is the knock peak higher in the shorter window although it contains less signal energy? Where is the valve impact (8 kHz, 90° ATDC)?

Solution

a) Program output ($N = 256$):

window	coherent gain	ENBW [bins]	scalloping [dB]	highest side lobe [dB]
rectangular	1.000	1.000	−3.89	−13.3
Hann	0.500	1.500	−1.42	−31.5
Hamming	0.540	1.363	−1.75	−42.7
Blackman	0.420	1.727	−1.10	−58.1

Low side lobes are paid for by a wider main lobe and a larger ENBW (more noise per bin, i.e. lower SNR of a single bin).

b) $\Delta f = 156.25$ Hz; 6500 Hz lies at bin 41.6, 8000 Hz at bin 51.2. Levels in dB:

window	peak	bin of 8 kHz	at 9.5 kHz	at 15 kHz
rectangular	−2.47	−31.0	−38.2	−50.7
Hann	−0.91	−58.8	−87.6	−114.4
Hamming	−1.12	−48.4	−55.0	−67.3
Blackman	−0.70	−59.7	−95.1	−121.8

The weak tone (−60 dB) is visible only with Hann and Blackman (the bins around 9.5 kHz are far lower, so the −59 dB at 8 kHz is the tone, not leakage). Rectangular and Hamming bury it under leakage of the strong tone. The peak levels are below 0 dB because the tone is 0.4 bins off the bin centre (scalloping; maximum at 0.5 bins as in a)).

c) Hann, zero padded to 4096: $N = 64$ ($\Delta f = 625$ Hz): two maxima at 6112 Hz and 7288 Hz – *wrong*: they are the merged main lobe with its interference pattern, not the two lines. $N = 128$ ($\Delta f = 312.5$ Hz): a single peak at 6700 Hz (unresolved). $N = 256$ ($\Delta f = 156$ Hz, separation 2.56 bins): 6504.3 Hz and 6895.7 Hz, both 0.2 dB – resolved and accurately located. Zero padding gave a smooth curve in all cases but never created resolution. Key code:

```
static void print_peaks(const double *db, int nb, int nfft, double rel)
{
    int k;
    double thr = db[0];
    for (k = 1; k < nb; k++) if (db[k] > thr) thr = db[k];
    thr -= rel;                                   /* relative threshold */
    for (k = 2; k < nb - 2; k++) {
        if (db[k] > thr && db[k] >= db[k - 1] && db[k] > db[k + 1] &&
            db[k] >= db[k - 2] && db[k] > db[k + 2]) {
            double a = db[k - 1], b = db[k], c = db[k + 1];
            double d = 0.5 * (a - c) / (a - 2 * b + c);   /* offset in bins */
            printf("    peak at %8.1f Hz  (bin %3d%+.2f)  %6.1f dB\n",
                   (k + d) * FS / nfft, k, d, b - 0.25 * (a - c) * d);
        }
    }
}

int main(void)
{
```

d) Seed 7, first knocking cylinder (cyl. 1, onset 15.8° ATDC), peaks within 15 dB of the maximum (Hann): 0°–115°, $N = 256$: 6508 Hz/−19.6 dB, 10 563 Hz/−24.4 dB (and a noise peak at 7349 Hz/−32.3 dB); 10°–68°, $N = 128$: 6513 Hz/−12.9 dB, 10 479 Hz/−17.8 dB. Second cylinder (cyl. 3, onset 19.3°): 6489 Hz/10 491 Hz resp. 6497 Hz/10 520 Hz. The amplitude scaling divides by $\sum w \propto N$: a burst that lasts only $\approx 3\tau = 2.4$ ms appears with its *average* amplitude over the window, so a window matched to the burst gives a higher level (and a better SNR, since the noise outside the burst is excluded). The valve burst starts at 90° ATDC = 5.0 ms, i.e. in the last 22 % of the long window, where the Hann window is small: only a weak 8 kHz contribution (about −34 dB) remains; the short window excludes it completely – another reason for the 10°–70° measurement window.

Exercise 8.4 FFT on the microcontroller

WOKWI

Project `code/wokwi/sheet08_knockfft/` (ESP32 + `engine-sim` chip): `KNOCK` → D34 (ADC1), `TDC` → D18, `KNK` (ground truth) → D19. Add your `fft.h` of Exercise 8.2 to the project.

a) The sketch waits for the rising edge of the `TDC` pulse and then records $N = 256$ samples at $f_s = 40\,\text{kHz}$. Which crank-angle range does the record cover at 1500, 3000 and 6000 rpm? Why is the analysis synchronised to the crank angle and not to time?

b) Implement the capture with a deadline loop (`micros()`, period 25 µs); count missed deadlines in `late` and record whether `KNK` was high during the window.

c) Implement the Hann window, the search for the largest bin above 3 kHz and the parabolic peak interpolation. Why must the search exclude the low-frequency range? Set the knock probability to 50 % and collect about 20 windows: compare peak frequency, peak level and the knock-band energy `e_knock` (6–7 kHz) for windows with and without knock (`knk`). Plot one spectrum in JSLinux from the CSV output.

d) **Uno.** Open `code/wokwi/sheet08_unofft/` (synthetic test signal, same `fft.h`; on the AVR `double` is a 32-bit float). Compile for $N = 64$, 128, 256 and read the memory report. What is the largest feasible N and why? Add the operation count (butterflies, real multiplications and additions). Estimate the execution time from the operation count, assuming roughly 100–150 clock cycles per soft-float operation at 16 MHz, and compare with the time between two TDCs at 3000 rpm. Suggest two ways to fit a 256-point spectrum into the Uno.

Hint: Wokwi is not cycle-accurate: `micros()` differences (`tcap`, `tfft`) are simulated times and only indicative. The Serial output of one window (130 lines) takes about 0.25 s at 115200 baud; TDC events during printing are simply skipped.

Solution

a) $256/f_s = 6.4\,\text{ms}$, i.e. $6 \cdot n \cdot 6.4\,\text{ms}$ degrees: 57.6° at 1500 rpm, 115.2° at 3000 rpm, 230.4° at 6000 rpm (longer than the 180° between two TDCs – the next combustion leaks in; a real system uses an angle-dependent N or f_s). Knock can only occur in a known angle range after TDC; triggering on TDC makes the windows of all combustions comparable and excludes disturbances at known angles (valve closing).
b)/c) Key part of `code/solutions/wokwi/sheet08_knockfft/sketch.ino`:

```
    /* 1) wait for the rising edge of the TDC pulse */
    while (digitalRead(PIN_TDC) == HIGH) { }
    while (digitalRead(PIN_TDC) == LOW) { }

    /* 2) capture N samples with a deadline loop */
    t0 = tn = micros();
    for (n = 0; n < N; n++) {
        if ((long)(micros() - tn) > 0) late++;          /* deadline missed */
        while ((long)(micros() - tn) < 0) { }
        tn += TS_US;
        raw[n] = analogRead(PIN_KNOCK);
        if (digitalRead(PIN_KNK)) knk = 1;
    }
    tcap = micros() - t0;

    /* 3) sensor volts, remove mean, Hann window */
    for (n = 0; n < N; n++) mean += raw[n];
    mean /= N;
    for (n = 0; n < N; n++) {
        double w = 0.5 - 0.5 * cos(2.0 * M_PI * n / N);
        re[n] = (raw[n] - mean) * (VREF / 4095.0) / OFFSET_GAIN * w;
        im[n] = 0.0;
        sw += w;
```

```
    }

    /* 4) FFT and amplitude spectrum in dB (sinusoid of amplitude A -> 20 log A) */
    t0 = micros();
    fft(re, im, N, 0);
    tfft = micros() - t0;
    kp = (int)(F_MIN * N / FS);
    for (k = 0; k <= N / 2; k++) {
        double a = 2.0 * sqrt(re[k] * re[k] + im[k] * im[k]) / sw;
        db[k] = 20.0 * log10(a + 1e-9);
        if (k * FS / N >= F_MIN && db[k] > db[kp]) kp = k;
        /* energy in the knock band 6.0 .. 7.0 kHz */
        if (k * FS / N >= 6000.0 && k * FS / N <= 7000.0) e_knock += a * a;
    }

    /* 5) peak frequency with parabolic interpolation */
    double d = 0.0;
    if (kp < N / 2) {
        double a = db[kp - 1], b = db[kp], c = db[kp + 1];
        d = 0.5 * (a - c) / (a - 2.0 * b + c);
    }
```

The global maximum is the combustion vibration (half sine over 60°, 0.2 V): after the mean removal it still produces a peak near 200 Hz of about −19 dB, as high as the knock peak; hence the search starts at 3 kHz. Expected results (PC emulation of the sketch with the same engine model, 50 % knock probability, 12-bit quantisation; Wokwi gives other random values but the same picture):

	peak frequency	peak level	`e_knock`
knocking (`knk=1`, 8 windows)	6470–6531 Hz	−23.9 to −15.5 dB	−19.7 to −11.0 dB
no knock (8 windows)	random, 7.8–19.8 kHz	−36 to −33 dB	−40.1 to −32.8 dB

The knock band energy separates the two classes by more than 10 dB – the idea of the detector of Sheet 12. The second mode (10.5 kHz) appears about 4–10 dB below the first. `late` should be 0; if not, the simulated `analogRead` is slower than 25 µs (the ESP32 ADC needs about 10–20 µs in reality; a real application uses the I2S/DMA ADC mode).
d) arduino-cli memory report (global variables of 2048 B):

N	arrays $2 \cdot 4N$	global RAM	result
64	512 B	716 B (34 %)	OK
128	1024 B	1228 B (59 %)	OK, 820 B left for stack
256	2048 B	2252 B (109 %)	*data section exceeds available space*

Largest $N = 128$ (about 200 B are taken by the Serial buffers and the core, the stack needs the rest). Operation count for $N = 128$: $\frac{N}{2}\log_2 N = 448$ butterflies, $4 \cdot 448 = 1792$ real multiplications, $6 \cdot 448 = 2688$ additions (plus the twiddle recursion, 4 mult. per butterfly group). With ≈ 120 cycles per float operation: $\approx 4500 \cdot 120 \approx 5.4 \cdot 10^5$ cycles ≈ 34 ms – more than three times the 10 ms between two TDCs at 3000 rpm. Expected strong bins of the test signal ($\Delta f = 312.5$ Hz): bin 21 (6562.5 Hz, 0.92) and its neighbours for 6.5 kHz, bins 33/34 (0.25/0.38) for 10.5 kHz (leakage, no window). Ways out: (1) a real-input FFT (an N-point real FFT via an $N/2$-point complex FFT) halves the memory; (2) 16-bit fixed point (Q15) instead of float: 4 B per complex sample, $N = 256$ needs 1 kB and integer multiplications are much faster than soft-float; (3) do not compute a full spectrum at all – a Goertzel filter or a band-pass (Sheets 9/10) computes the knock-band energy with a few bytes of state.

Deliverables

DFT computations of 8.1; your `fft.h` (it is reused on Sheets 11 and 12!) with the error and runtime tables of 8.2; the window tables and the resolution experiment of 8.3; for 8.4 a CSV spectrum of one knocking and one non-knocking window with your knock/no-knock statistics, and the Uno memory table with the operation-count estimate.

Digital Signal Processing Exercise Sheet 9

Z-Transform

Script: Chapter 9 (z-transform, transfer functions of discrete-time systems) **with solutions**

Learning goals

- Compute z-transforms with their region of convergence (ROC) and invert rational $X(z)$ by partial fractions for causal, anticausal and two-sided sequences.
- Derive $H(z)$, poles and zeros of the RPM smoothers of Sheet 3 and decide stability from the pole positions.
- Read the magnitude response from the pole-zero map (geometric interpretation) and design a two-pole resonator for the knock frequency.
- Build a pole-zero explorer (polynomial roots, ASCII pole-zero map, frequency and impulse response).
- Implement the resonator in Q15 fixed point on the Arduino Uno and observe coefficient quantisation, zero-input limit cycles and overflow oscillations for $r \to 1$.

Background

The (bilateral) z-transform of $x[n]$ is $X(z) = \sum_{n=-\infty}^{\infty} x[n] z^{-n}$, defined in a ring-shaped ROC $r_1 < |z| < r_2$. An LTI system with the difference equation $\sum_k a_k y[n-k] = \sum_k b_k x[n-k]$ has the transfer function

$$H(z) = \frac{b_0 + b_1 z^{-1} + \cdots + b_M z^{-M}}{a_0 + a_1 z^{-1} + \cdots + a_N z^{-N}} = \frac{b_0}{a_0}\, z^{N-M}\, \frac{\prod_k (z - z_k)}{\prod_k (z - p_k)}.$$

A causal system is stable iff all poles lie inside the unit circle (then the ROC $|z| > \max |p_k|$ contains the unit circle), and its frequency response is $H(\mathrm{e}^{\mathrm{j}\Omega})$, $\Omega = 2\pi f / f_s$. Geometrically, $|H(\mathrm{e}^{\mathrm{j}\Omega})| = |\frac{b_0}{a_0}| \prod_k |\mathrm{e}^{\mathrm{j}\Omega} - z_k| / \prod_k |\mathrm{e}^{\mathrm{j}\Omega} - p_k|$: a pole close to the unit circle produces a peak, a zero on it a notch.

The **two-pole resonator** with poles $p_{1,2} = r\mathrm{e}^{\pm \mathrm{j}\theta}$, $\theta = 2\pi f_0 / f_s$,

$$H(z) = \frac{b_0}{1 + a_1 z^{-1} + a_2 z^{-2}}, \qquad a_1 = -2r\cos\theta, \quad a_2 = r^2, \tag{3}$$

is the simplest digital knock filter. Throughout this sheet $f_0 = 6.5\,\mathrm{kHz}$ and $f_s = 40\,\mathrm{kHz}$ ($\theta = 58.5°$).

Exercise 9.1 Analytical z-transforms

PAPER

a) Compute the z-transform and the ROC of $\delta[n-k]$, $\varepsilon[n]$, $a^n\varepsilon[n]$, $-a^n\varepsilon[-n-1]$ and $r^n \cos(\theta n)\,\varepsilon[n]$. Why is the ROC indispensable?

b) Find all sequences with $X(z) = \dfrac{1}{(1 - 0.5z^{-1})(1 - 0.8z^{-1})}$ by partial fractions (one for each possible ROC). Which one is causal, which one has a DTFT? Check $x[0]$ and $x[1]$ of the causal one with the difference equation.

c) **RPM smoothers of Sheet 3.** Derive $H(z)$, poles and zeros of

   i. the exponential smoother $y[n] = a\,y[n-1] + (1-a)\,x[n]$,

   ii. the M-point moving average.

   When are they stable? Give $|H|$ at $\Omega = 0$ and $\Omega = \pi$ for $a = 0.9$ and the notch frequencies of the moving average for $M = 8$ at $f_s = 40\,\mathrm{kHz}$. What happens to the pole at $z = 1$ of $(1 - z^{-M})/(1 - z^{-1})$?

d) **Resonator.** Show that (3) has the poles $r\mathrm{e}^{\pm\mathrm{j}\theta}$. Derive the stability triangle in the (a_1, a_2)-plane ($|a_2| < 1$, $|a_1| < 1 + a_2$) and mark the resonator.

e) Using the geometric interpretation, show that the $-3\,\mathrm{dB}$ bandwidth is $\Delta\Omega \approx 2(1-r)$, i.e. $B \approx (1-r)f_s/\pi$, for r close to 1. Choose r for $B = 400\,\mathrm{Hz}$. Compare with the impulse-invariant image $r = \mathrm{e}^{-T/\tau}$ of the analog knock resonator of Sheet 7 ($\tau = 0.8\,\mathrm{ms}$).

f) Determine b_0 such that $|H(\mathrm{e}^{\mathrm{j}\theta})| = 1$ and evaluate it for $r = 0.97$.

Solution

a) $\delta[n-k] \to z^{-k}$ (all z, except $z = 0$ for $k > 0$ and $z = \infty$ for $k < 0$). $\varepsilon[n] \to \frac{1}{1-z^{-1}}$, $|z| > 1$. $a^n\varepsilon[n] \to \sum_{n\geq 0}(az^{-1})^n = \frac{1}{1-az^{-1}}$, $|z| > |a|$. $-a^n\varepsilon[-n-1] \to -\sum_{m\geq 1}(a^{-1}z)^m = \frac{1}{1-az^{-1}}$, $|z| < |a|$ – the same algebraic expression: only the ROC tells which sequence is meant. $r^n\cos(\theta n)\varepsilon[n] = \frac{r^n}{2}(\mathrm{e}^{\mathrm{j}\theta n} + \mathrm{e}^{-\mathrm{j}\theta n})\varepsilon[n] \to \frac{1-r\cos\theta\, z^{-1}}{1-2r\cos\theta\, z^{-1}+r^2z^{-2}}$, $|z| > r$.
b) $X(z) = \frac{A}{1-0.5z^{-1}} + \frac{B}{1-0.8z^{-1}}$ with $A = \frac{1}{1-0.8/0.5} = -\frac{5}{3}$, $B = \frac{1}{1-0.5/0.8} = \frac{8}{3}$.

- $|z| > 0.8$: $x[n] = (-\frac{5}{3}\,0.5^n + \frac{8}{3}\,0.8^n)\varepsilon[n]$ (causal, ROC contains the unit circle ⇒ DTFT exists).
- $0.5 < |z| < 0.8$: $x[n] = -\frac{5}{3}\,0.5^n\varepsilon[n] - \frac{8}{3}\,0.8^n\varepsilon[-n-1]$ (two-sided, no DTFT).
- $|z| < 0.5$: $x[n] = \frac{5}{3}\,0.5^n\varepsilon[-n-1] - \frac{8}{3}\,0.8^n\varepsilon[-n-1]$ (anticausal, no DTFT).

Check: $y[n] = 1.3y[n-1] - 0.4y[n-2] + \delta[n]$ gives $y[0] = 1$, $y[1] = 1.3$; the formula: $-\frac{5}{3} + \frac{8}{3} = 1$, $-\frac{5}{6} + \frac{32}{15} = 1.3$.
c) i. $H(z) = \frac{1-a}{1-az^{-1}} = \frac{(1-a)z}{z-a}$: zero at 0, pole at a; stable iff $|a| < 1$. $|H(\mathrm{e}^{\mathrm{j}0})| = 1$, $|H(\mathrm{e}^{\mathrm{j}\pi})| = \frac{1-a}{1+a} = 0.0526$ ($-25.6\,\mathrm{dB}$ for $a = 0.9$). ii. $H(z) = \frac{1}{M}\sum_{k=0}^{M-1} z^{-k} = \frac{1}{M}\frac{1-z^{-M}}{1-z^{-1}} = \frac{z^M-1}{M\,z^{M-1}(z-1)}$. The roots of $z^M = 1$ are $\mathrm{e}^{\mathrm{j}2\pi k/M}$; the one at $z = 1$ cancels the pole at $z = 1$, so there remain $M-1$ zeros on the unit circle and $M-1$ poles at $z = 0$: an FIR filter, always stable. For $M = 8$: notches at $kf_s/8 = 5, 10, 15, 20\,\mathrm{kHz}$.
d) $z^2 + a_1z + a_2 = z^2 - 2r\cos\theta\, z + r^2 = (z - r\mathrm{e}^{\mathrm{j}\theta})(z - r\mathrm{e}^{-\mathrm{j}\theta})$. Stability triangle: for complex poles $|p|^2 = a_2 < 1$; for real poles the polynomial $P(z) = z^2 + a_1z + a_2$ must have both roots in $(-1, 1)$, i.e. $P(1) = 1 + a_1 + a_2 > 0$ and $P(-1) = 1 - a_1 + a_2 > 0 \Rightarrow |a_1| < 1 + a_2$, $|a_2| < 1$. The resonator with $r = 0.97$ ($a_1 = -1.0136$, $a_2 = 0.9409$) lies inside, close to the upper edge $a_2 = 1$.
e) For Ω near θ the distance to the far pole is almost constant, $|\mathrm{e}^{\mathrm{j}\theta} - r\mathrm{e}^{-\mathrm{j}\theta}| \approx 2\sin\theta$; the distance to the near pole is $|\mathrm{e}^{\mathrm{j}\Omega} - r\mathrm{e}^{\mathrm{j}\theta}| \approx \sqrt{(1-r)^2 + (\Omega-\theta)^2}$. It grows by $\sqrt{2}$ for $|\Omega - \theta| = 1 - r$: $\Delta\Omega = 2(1-r)$, $B = \frac{\Delta\Omega}{2\pi}f_s = (1-r)f_s/\pi$. $B = 400\,\mathrm{Hz}$: $r = 1 - \pi B/f_s = 0.9686$. Impulse invariance: $r = \mathrm{e}^{-T/\tau} = \mathrm{e}^{-0.03125} = 0.9692$, $B = 392\,\mathrm{Hz}$, close to the analog 398 Hz – both views agree, because $1 - \mathrm{e}^{-T/\tau} \approx T/\tau$ and $B_{\text{analog}} = 1/(\pi\tau)$.
f) $|H(\mathrm{e}^{\mathrm{j}\theta})| = \frac{b_0}{|1-r\mathrm{e}^{\mathrm{j}\theta}\mathrm{e}^{-\mathrm{j}\theta}|\,|1-r\mathrm{e}^{-\mathrm{j}\theta}\mathrm{e}^{-\mathrm{j}\theta}|} = \frac{b_0}{(1-r)\,|1-r\mathrm{e}^{-2\mathrm{j}\theta}|}$, hence $b_0 = (1-r)\sqrt{1 - 2r\cos 2\theta + r^2}$; $r = 0.97$: $b_0 = 0.050393$. Peak gain without normalisation ($b_0 = 1$): $1/0.0504 = 19.8$ (26 dB) – important for the fixed-point scaling in Exercise 9.3.

Exercise 9.2 Pole-zero explorer

JSLINUX

Program `code/jslinux/sheet09/pz_explorer.c` reads the coefficients from the command line,

```
./pz_explorer -fs 40000 -b 0.1 -a 1 -0.9      # exponential smoother
./pz_explorer -fs 40000 -res 6500 0.97        # resonator, gain 1 at f0
```

computes poles and zeros, prints them in polar form (radius, angle in Hz), decides stability, draws the pole-zero map with `ap_pz()` and plots magnitude (dB) and phase over f and the impulse response.

a) Poles and zeros are the roots of $z^LB(z)$ and $z^LA(z)$, $L = \max(M, N)$ (`tf_roots()` is given). Implement the Durand–Kerner iteration `poly_roots()` for a polynomial of degree n with $c_0 =$

1:

$$r_i \leftarrow r_i - \frac{c(r_i)}{\prod_{j\neq i}(r_i - r_j)}, \qquad r_i^{(0)} = (0.4 + 0.9\mathrm{j})^{i+1}.$$

Use the small complex struct of the template (no `complex.h`).

b) Implement the geometric evaluation `freq_geom()` and the impulse response from the difference equation. The program compares the geometric magnitude with the direct evaluation of B/A.

c) Explore and report (pole-zero map, peak/notch, bandwidth, $|h[n]|$):

i. exponential smoother $a = 0.9$; moving average $M = 8$;

ii. resonator $r = 0.9, 0.97, 0.99$: compare the $-3\,\mathrm{dB}$ bandwidth with $(1 - r)f_s/\pi$;

iii. resonator $r = 1$ and $r = 1.02$: what does the program say, what does the impulse response do?

iv. a notch filter at $6.5\,\mathrm{kHz}$ with zeros at $\mathrm{e}^{\pm\mathrm{j}\theta}$ and poles at $0.95\mathrm{e}^{\pm\mathrm{j}\theta}$. Give the coefficients and the notch width; why is $|H| \approx 1$ far from the notch?

d) Why may the magnitude plot of an unstable system be misleading?

Solution

a)/b) Key parts of `code/solutions/jslinux/sheet09/pz_explorer.c`:

```c
static int poly_roots(const double *p, int n, cpx *r)
{
    cpx c[MAXC], w = cx(0.4, 0.9), num, den, d;
    int i, j, it;
    double change;
    if (n < 1) return 0;
    for (i = 0; i <= n; i++) c[i] = cx(p[i] / p[0], 0.0);   /* monic */
    r[0] = cx(1.0, 0.0);
    for (i = 0; i < n; i++) r[i] = i ? cmul(r[i - 1], w) : cx(1.0, 0.0);
    for (i = 0; i < n; i++) r[i] = cmul(r[i], w);           /* (0.4+0.9j)^(i+1) */
    for (it = 1; it <= 2000; it++) {
        change = 0.0;
        for (i = 0; i < n; i++) {
            num = c[0];                                      /* Horner */
            for (j = 1; j <= n; j++) num = cadd(cmul(num, r[i]), c[j]);
            den = cx(1.0, 0.0);
            for (j = 0; j < n; j++)
                if (j != i) den = cmul(den, csub(r[i], r[j]));
            d = cdiv(num, den);
            r[i] = csub(r[i], d);
            if (cabs_(d) > change) change = cabs_(d);
        }
        if (change < 1e-14) return it;
    }
    return it;
}

/* roots of z^L * (q0 + q1 z^-1 + ... + qK z^-K): exact zeros at z = 0 for
   vanishing trailing coefficients, Durand-Kerner for the rest.
   Leading zero coefficients (q0 = 0, ...) are roots "at infinity" and are
   skipped. Returns the number of finite roots. */
static int tf_roots(const double *q, int K, int L, cpx *r)
{
    double p[MAXC];
    int i, lead = 0, n, nz = 0;
    for (i = 0; i <= L; i++) p[i] = i <= K ? q[i] : 0.0;
    while (lead <= L && p[lead] == 0.0) lead++;          /* roots at infinity */
    if (lead > L) return 0;
    n = L - lead;                                        /* degree */
    while (n > 0 && p[lead + n] == 0.0) { r[nz++] = cx(0.0, 0.0); n--; }
```

```
    poly_roots(p + lead, n, r + nz);
    return nz + n;
}
```

```
static double freq_geom(double g, const cpx *z, int nz, const cpx *p, int np, double W)
{
    cpx e = cx(cos(W), sin(W));
    double m = fabs(g);
    int k;
    for (k = 0; k < nz; k++) m *= cabs_(csub(e, z[k]));
    for (k = 0; k < np; k++) m /= cabs_(csub(e, p[k]));
    return m;
}
```

The geometric and the direct magnitude agree to 10^{-14} dB (notch: 10^{-8} dB, relative error near the zero). Durand–Kerner converges quadratically for simple roots; for multiple roots (e.g. the $M-1$ poles at $z=0$ of the moving average, here found exactly by `tf_roots`) it is only linear.
c) Program results ($f_s = 40$ kHz, grid 10 Hz):

- Smoother $a = 0.9$: pole 0.9, zero 0; $|H|$ 0 dB at DC, -25.58 dB at $f_s/2$; $h[n] = 0.1 \cdot 0.9^n$ ($|h[59]| = 2.0 \cdot 10^{-4}$). Moving average $M = 8$: zeros at ±5, ±10, ±15, 20 kHz on the unit circle, 7 poles at 0; $h[n] = 1/8$ for $n = 0..7$, then 0.
- Resonator (b_0 normalised, poles at exactly 6500.0 Hz):

r	B measured	$(1-r)f_s/\pi$	$\lvert H\rvert$ at DC	$\lvert H\rvert$ at $f_s/2$
0.90	1370 Hz	1273 Hz	-14.6 dB	-24.6 dB
0.97	400 Hz	382 Hz	-25.3 dB	-35.4 dB
0.99	140 Hz	127 Hz	-34.9 dB	-45.0 dB

  The approximation is good for $r \to 1$ (the remaining difference for $r = 0.99$ is the 10 Hz grid). Smaller B means a longer impulse response: decay $\propto r^n$, time constant $-1/(f_s \ln r) = 0.82$ ms for $r = 0.97$ (cf. $\tau = 0.8$ ms of the knock burst).
- $r = 1$: "MARGINALLY STABLE (not BIBO stable)", $h[n] = \sin((n+1)\theta)$, a sinusoidal oscillator with constant amplitude; the magnitude plot shows an (on the 10 Hz grid finite) infinite peak. $r = 1.02$: "UNSTABLE", $|h[n]|$ grows like 1.02^n ($|h[71]| = 0.157$ starting from 0.034).
- Notch: $b = \{1, -2\cos\theta, 1\} = \{1, -1.044997, 1\}$, $a = \{1, -1.9\cos\theta, 0.9025\} = \{1, -0.992748, 0.9025\}$. Zeros 0.5225 ± 0.8526j ($|z| = 1$), poles 0.4964 ± 0.8100j ($|p| = 0.95$), $|H| = -116$ dB at 6.5 kHz, $+0.42$ dB at DC, $+0.44$ dB at $f_s/2$; -3 dB stop band 6200–6800 Hz, width 600 Hz $\approx (1-0.95)f_s/\pi = 637$ Hz. Far from θ each zero and its neighbouring pole have almost the same distance to $e^{j\Omega}$, the ratios cancel ($|H| \approx 1$).

d) For poles outside the unit circle the ROC of the *causal* system does not contain the unit circle; $H(e^{j\Omega})$ is then a formal evaluation of the rational function, not the response to a sinusoid (which grows without bound). The program prints a warning in this case.

Exercise 9.3 Digital resonator in float and Q15

WOKWI JSLINUX

The resonator (3) is implemented twice in the header `resonator.h`: in `float` and in fixed point with signals in Q15 (`int16_t`, 1 LSB $= 2^{-15}$), coefficients in Q14 and an `int32_t` accumulator. The *same file* is used on JSLinux and on the Uno; you find it in both folders `code/jslinux/sheet09/` and `code/wokwi/sheet09_resonator/`.

a) **(paper)** Why can a_1 not be stored in Q15 here? In which format is the accumulator, and how many bits does it need at most? Which quantisation step does $a_2 = r^2$ get, and how large is

the resulting change of the pole radius for $r = 0.9999$?

b) Implement the functions `res_design`, `res_f_step` and `res_q_step` (rounding or truncation, saturation or wrap-around, and count the overflows). Note: `int` has 16 bits on the AVR – cast to `int32_t` *before* multiplying. Run the program `limitcycle.c` and compare quantised and exact pole positions (part (a) of the program) and the impulse responses (b).

c) **Zero-input limit cycles.** Starting from $y[-1] = 0.5$ with $x = 0$, the float resonator decays to zero. Part (c) of the program measures the amplitude of the Q15 output after 5 s. For rounding, a limit cycle can persist as long as rounding maps $a_2 y$ back to y: derive the dead-band bound $|y| \le \frac{0.5}{1-a_2}$ LSB. Compare with the measured amplitudes for $r \to 1$, rounding and truncation, and explain the frequency of the limit cycle.

d) **Overflow.** Without normalisation ($b_0 = 1$) the gain at f_0 is about 20 (Exercise 9.1 f). Part (d) feeds a knock burst of amplitude $A = 0.1$ and 0.5 into the resonator with $r = 0.99$. Compare wrap-around and saturation. What is an *overflow oscillation*?

e) **Uno.** Open the Wokwi project (Uno, potentiometer = r from 0.90 to 1.00, switch SW1 = rounding/truncation, SW2 = saturation/wrap, button = next test T0/T1/T2, see the sketch header) with your `resonator.h`. Reproduce the results of c) and d) for $r \approx 0.97$, 0.99, 0.999. Are the integer results identical to JSLinux? Could you also expect identical float results?

f) Draw conclusions for a fixed-point knock filter in an ECU (choice of r, rounding, overflow handling, scaling).

Hint: The Uno computes the test signals itself; its ADC (about 9 kHz maximum, Sheet 5) could not sample a 6.5 kHz knock signal anyway. The tests are not real-time – we study the arithmetic; real-time knock filtering follows on Sheet 10.

Solution

a) $a_1 = -2r\cos\theta = -1.0136$ for $r = 0.97$ and up to -1.045 for $r \to 1$: outside the Q15 range $[-1, 1)$. Q14 ($[-2, 2)$, step 2^{-14}) is used for all coefficients. Products Q14×Q15 are Q29; the sum of three products is bounded by $(|b_0|+|a_1|+|a_2|)\,2^{29} < 4\cdot 2^{29} = 2^{31}$: an `int32_t` is just sufficient. Step of a_2: $2^{-14} = 6.1\cdot 10^{-5}$, $\Delta r \approx \Delta a_2/(2r) \le 3.1\cdot 10^{-5}$ – comparable with $1 - r = 10^{-4}$ for $r = 0.9999$: the bandwidth (1.3 Hz) is then only known to about ±30 %. Program output (a):

r	b_0	a_1	a_2	Q14: b_0, a_1, a_2	r_q	f_q [Hz]
0.9	0.162086	−0.940497	0.810000	2656, −15409, 13271	0.899999	6500.0
0.97	0.050393	−1.013647	0.940900	826, −16608, 15416	0.970009	6499.9
0.99	0.016968	−1.034547	0.980100	278, −16950, 16058	0.990001	6500.0
0.999	0.001704	−1.043952	0.998001	28, −17104, 16351	0.998992	6500.0
0.9999	0.000171	−1.044892	0.999800	3, −17120, 16381	0.999908	6499.9

Note also the coarse b_0 for $r \to 1$ (3 LSB: gain error up to 17 %).
b) Key part of `resonator.h`:

```
static int16_t res_q_step(res_q_t *s, int16_t x)
{
    int32_t acc, y;
    acc = (int32_t)s->b0 * x - (int32_t)s->a1 * s->y1 - (int32_t)s->a2 * s->y2; /* Q29 */
    if (s->rnd) acc += (int32_t)1 << 13;      /* round to nearest          */
    y = acc >> 14;                            /* arithmetic shift: floor   */
    if (y > 32767 || y < -32768) {
        s->novf++;
        if (s->sat) y = y > 0 ? 32767 : -32768;
        else y = (int16_t)(uint16_t)(y & 0xFFFF);  /* two's complement wrap */
    }
    s->y2 = s->y1;
    s->y1 = (int16_t)y;
```

```
14      return (int16_t)y;
15  }
```

Impulse $0.5\delta[n]$, $r = 0.97$, rounding: the Q15 output follows the float output with a maximum deviation of 7.2 LSB (peak 837 LSB); after 400 samples the float output is 0.000 LSB, the Q15 output still −2 LSB.
c) Zero input after $y[-1] = 0.5$: a limit cycle needs $|y[n]|$ to stay constant although the exact recursion multiplies the amplitude by $r < 1$ per sample. With rounding the oscillation behaves as if a_2 were 1 as long as $|\mathrm{round}(a_2 y) - a_2 y| \le 0.5$ can compensate the decay $(1 - a_2)|y|$, i.e. $(1 - a_2)|y| \le 0.5$ (Jackson's dead band). Measured after 200 000 samples:

r	$a_{2,q}$	bound [LSB]	round: amp. [LSB]	f [Hz]	trunc.: amp. [LSB]	f [Hz]
0.9	13271	2.6	2	6680	1	0 (DC)
0.97	15416	8.5	7	6660	0	–
0.99	16058	25.1	11	6660	3	5720
0.995	16221	50.3	11	6660	0	–
0.999	16351	248.2	11	6660	0	–
0.9999	16381	2730.7	98	6500	172	6500
1.0	16384	∞	19217	6500	19207	6500

All rounding limit cycles respect the bound; they grow with r but stay far below it for $r \ge 0.99$. Their frequency is not f_0: the limit cycle is a periodic integer sequence, here of period 6 samples ($f_s/6 = 6667$ Hz; the zero-crossing count gives 6660–6680). Truncation (arithmetic shift = rounding towards $-\infty$) usually lets the output die out but can lock into a constant −1 LSB (DC limit cycle, $r = 0.9$) or into oscillations ($r = 0.99$: period 7, 5714 Hz; $r = 0.9999$). Example ($r = 0.99$, rounding): $\ldots, -6, -11, -5, 6, 11, 5, -6, \ldots$ For $r = 1$ the system is an oscillator (amplitude $0.5/\sin\theta \cdot 32768 \approx 19217$ LSB), not a limit cycle.
d) $r = 0.99$, $b_0 = 1$, burst $A\mathrm{e}^{-t/\tau}\sin(2\pi f_0 t)$ for 10 ms, then 0:

case	float peak	overflows	max $\lvert y\rvert$ in the last 25 ms [LSB]
$A = 0.1$, wrap-around	1.08	39967	21740
$A = 0.1$, saturation	1.08	5	11
$A = 0.5$, wrap-around	5.42	24	11
$A = 0.5$, saturation	5.42	34	11

With two's complement wrap-around, an overflow turns a large positive value into a large negative one; for $|a_1| + |a_2| > 1$ this can sustain a full-scale oscillation without input (*overflow oscillation*): for $A = 0.1$ the filter overflows in practically every sample for the whole second. Whether it happens depends on the trajectory ($A = 0.5$ recovered by chance). Saturation limits the error to a clipping and the filter recovers to the small rounding limit cycle (11 LSB).
e) The integer path uses only `int16_t`/`int32_t` arithmetic with identical rounding, so the Uno must produce *bit-identical* Q15 results to JSLinux for the same r (the potentiometer gives $r = 0.9 + 0.1 \cdot \mathrm{adc}/1023$, so use the printed r and Q14 coefficients for the comparison, e.g. adc = 921 → $r = 0.99003$). The float path is single precision on both, but `sinf`/`expf` of avr-libc and the evaluation order may differ in the last bit; also a 32-bit x86 compiler (JSLinux) may keep intermediates in 80-bit registers. Small float differences are therefore possible, integer differences are not.
f) For $B \approx 400$ Hz ($r \approx 0.97$) the rounding limit cycle is 7 LSB (−73 dBFS), harmless compared with the sensor noise. Rules: use rounding, always saturate (never wrap), scale the input with the normalised b_0 (or reduce the input range) so that the output cannot exceed full scale, store coefficients in Q14 when $|a_1| > 1$, avoid $r \to 1$ (narrow filters need more bits: 32-bit state or error feedback), or use floating point on the ESP32 (Sheet 10).

Deliverables

Solutions of 9.1 (partial fractions with all ROCs, stability triangle, bandwidth formula and r for 400 Hz); the pole-zero explorer with the screenshots (ASCII plots) and the result table of 9.2 c); the tables of 9.3 a), c) and d) from JSLinux and the corresponding Uno outputs (T0–T2 for at least two values of r) with your conclusions 9.3 f).

Digital Signal Processing
Digital Filters
Exercise Sheet 10

Script: Chapter 10 (FIR and IIR filter design, filter structures)

with solutions

Learning goals

- Turn a verbal requirement ("extract the knock resonance") into a quantitative filter specification and estimate the necessary order.
- Design linear-phase FIR filters with the window method and IIR filters with the bilinear transform (including pre-warping); realise IIR filters as biquad cascades.
- Judge the effects of finite word length: coefficient quantisation, scaling, overflow, saturation; Q15 arithmetic on an 8-bit controller.
- Run the knock band-pass sample by sample in real time on the ESP32 (float) and on the Arduino Uno (fixed point) and compare FIR and IIR in terms of cost and delay.

Background and conventions

The knock sensor signal of the virtual engine contains the knock resonance at $f_1 = 6.5\,\text{kHz}$ (plus a weaker mode at 10.5 kHz), valve impacts at 8 kHz, a low-frequency combustion vibration and broadband noise. The first block of every knock detector is a band-pass filter around f_1. On this sheet we use

sampling rate	$f_s = 25\,\text{kHz}$ ($T_s = 40\,\mu\text{s}$)
pass band	6.0 kHz to 7.0 kHz, loss at most $A_p = 1\,\text{dB}$
stop bands	$f \le 5.0\,\text{kHz}$ and $f \ge 8.0\,\text{kHz}$, attenuation at least $A_s = 40\,\text{dB}$

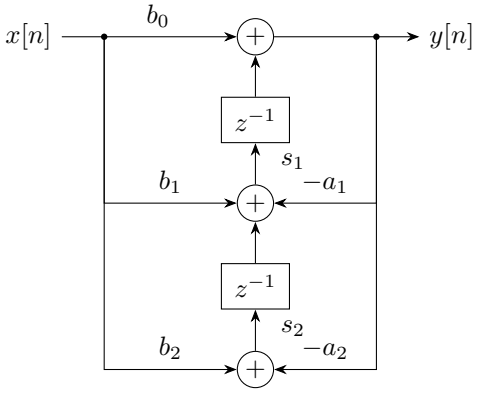
A biquad (second-order section) is written

$$H_k(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}},$$

a cascade is $H(z) = \prod_k H_k(z)$. Two structures are used: *direct form I* (DF I, states $x[n-1], x[n-2], y[n-1], y[n-2]$, one accumulator – preferred in fixed point) and the *transposed direct form II* (TDF II, two states s_1, s_2 – preferred in floating point):

$$y = b_0 x + s_1, \qquad s_1 \leftarrow b_1 x - a_1 y + s_2, \qquad s_2 \leftarrow b_2 x - a_2 y.$$

Q15 denotes a 16-bit two's complement number $v = q/2^{15} \in [-1, 1)$, *Q2.14* a 16-bit number $v = q/2^{14} \in [-2, 2)$.



Biquad in transposed direct form II.

Exercise 10.1 Design by hand

PAPER

a) *Sampling rate.* The engine-sim chip updates its analog outputs every 20 µs (50 kHz, zero-order hold). Explain why the ESP32 samples the knock signal at $f_s = 25\,\text{kHz}$ and not, e.g., at 40 kHz. Estimate the worst-case phase error of the 6.5 kHz component caused by the timing mismatch at 40 kHz. Is 25 kHz sufficient for both knock modes and for the valve disturbance? What happens to the white noise of the chip (generated at 50 kHz) when only every second value is sampled?

b) *FIR order.* Normalise the specification ($\Delta F = \Delta f / f_s$). Estimate the length N of a window-design FIR filter for a Hamming window ($N \approx 3.3/\Delta F$) and with Kaiser's formula $N - 1 \approx (A_s - 8)/(2.285\,\Delta\omega)$, $\beta = 0.5842(A_s - 21)^{0.4} + 0.07886(A_s - 21)$. Where do you put the cut-off frequencies of the ideal band-pass?

c) Derive the impulse response $h_d[m]$ of the ideal (zero-phase) band-pass with cut-offs $\omega_{c1} < \omega_{c2}$, and state how the causal FIR filter $h[n]$, $n = 0, \ldots, N-1$, is obtained from it. What is its group delay in samples, in ms, and in degrees crank angle at 3000 rpm and 6000 rpm? Why does this matter for a knock measurement window of 10°–70° after TDC?

d) *IIR order.* Pre-warp the band edges, $\Omega = 2f_s \tan(\pi f / f_s)$, and compute the stop-band frequency $\lambda_s = \min |\Omega_s^2 - \Omega_0^2| / (B\,\Omega_s)$ of the normalised low-pass prototype ($\Omega_0^2 = \Omega_{p1}\Omega_{p2}$, $B = \Omega_{p2} - \Omega_{p1}$). Which Butterworth prototype order n is needed, how many biquads does the band-pass have, and how many multiplications per sample does it cost compared with the FIR filter?

e) *Second-order band-pass.* The knock burst decays with $\tau = 0.8\,\text{ms}$. Show that its spectrum has a $-3\,\text{dB}$ bandwidth of $B_k = 1/(\pi\tau)$ and argue why a band-pass with about this bandwidth is a good compromise (matched filter). Design $H(s) = Bs/(s^2 + Bs + \Omega_0^2)$ with centre $f_0 = 6.5\,\text{kHz}$ and digital bandwidth 400 Hz via the bilinear transform $s = 2f_s\,(1 - z^{-1})/(1 + z^{-1})$ with pre-warping (use $B \approx 2\pi \cdot 400\,\text{Hz}/\cos^2(\pi f_0 / f_s)$). Compute b_0, b_1, b_2, a_1, a_2, the pole radius r and the decay time of the filter's impulse response.

Solution

a) The chip output is a staircase on a 20 µs grid. With $f_s = 25\,\text{kHz} = 50\,\text{kHz}/2$ every ADC sample sees a fresh chip value, always at the same position within the 20 µs step; the sequence is exactly the model signal sampled uniformly at 25 kHz. At 40 kHz the sampling instants slide over the grid: some chip values are read twice, others never, and the value read is up to 20 µs old. This is a non-uniform sampling with a timing error of up to $\Delta t = 20\,\mu\text{s}$, i.e. a phase error of up to $2\pi \cdot 6.5\,\text{kHz} \cdot 20\,\mu\text{s} = 0.82\,\text{rad} = 47°$ – a strong, deterministic distortion (spurious spectral lines). The Nyquist frequency at 25 kHz is 12.5 kHz, above both knock modes (6.5, 10.5 kHz) and the valve frequency (8 kHz), so no aliasing of signal components occurs. The chip noise is white with variance σ^2 at 50 kHz; decimation by 2 folds the band 12.5–25 kHz onto 0–12.5 kHz. White noise stays white with the same variance σ^2, only its PSD per Hz doubles – nothing is lost that a band-pass could have rejected anyway. (A real ECU would use an analog anti-aliasing filter, Sheet 5.)

b) Transition width $\Delta f = 1\,\text{kHz}$, $\Delta F = 0.04$, $\Delta\omega = 0.2513$. Hamming: $N \approx 3.3/0.04 = 82.5 \Rightarrow N = 83$ (odd, so that the delay is an integer and the filter is of type I). Kaiser: $\beta = 0.5842 \cdot 19^{0.4} + 0.07886 \cdot 19 = 3.395$, $N - 1 \approx 32/(2.285 \cdot 0.2513) = 55.7$, $N = 57$. The ideal cut-offs lie in the middle of the transition bands: $f_{c1} = 5.5\,\text{kHz}$, $f_{c2} = 7.5\,\text{kHz}$, because the window smears the ideal edge symmetrically over the transition band.

c) The ideal band-pass is the difference of two ideal low-passes:

$$h_d[m] = \frac{1}{2\pi}\int_{\omega_{c1} \le |\omega| \le \omega_{c2}} \mathrm{e}^{\mathrm{j}\omega m}\,\mathrm{d}\omega = \frac{\sin(\omega_{c2} m) - \sin(\omega_{c1} m)}{\pi m}, \qquad h_d[0] = \frac{\omega_{c2} - \omega_{c1}}{\pi}.$$

It is infinitely long and non-causal; we truncate it to $|m| \le (N-1)/2$, multiply with the window and shift by $(N-1)/2$: $h[n] = h_d[n - \frac{N-1}{2}]\,w[n]$. $h[n]$ is symmetric, hence linear phase with

a constant group delay of $(N-1)/2 = 41$ samples $= 1.64\,\text{ms}$. At 3000 rpm (18000°/s) this is 29.5°, at 6000 rpm 59° crank angle! The filter output belonging to a 10°–70° ATDC window appears up to 59° later; the evaluation window must be shifted by the (rpm dependent) filter delay, otherwise most of the knock energy is missed (see 10.3).
d) Pre-warped edges (rad/s): $\Omega(5\,\text{kHz}) = 36327$, $\Omega(6\,\text{kHz}) = 46953$, $\Omega(7\,\text{kHz}) = 60440$, $\Omega(8\,\text{kHz}) = 78787$; $\Omega_0 = \sqrt{46953 \cdot 60440} = 53271$ (maps back to 6.502 kHz), $B = 13486$. $\lambda(5\,\text{kHz}) = 3.099$, $\lambda(8\,\text{kHz}) = 3.171$, so $\lambda_s = 3.099$. Butterworth:

$$n \geq \frac{\log_{10}[(10^{A_s/10}-1)/(10^{A_p/10}-1)]}{2\log_{10}\lambda_s} = \frac{\log_{10}(9999/0.2589)}{2\log_{10}3.099} = \frac{4.587}{0.983} = 4.67 \;\Rightarrow\; n = 5.$$

The band-pass has order $2n = 10$, i.e. 5 biquads. Cost: 5 biquads with $b_1 = 0$ need $5 \cdot 4 = 20$ (general: 25) multiplications per sample versus 83 for the FIR filter (or 41 if the symmetry $h[n] = h[N-1-n]$ is exploited).
e) The burst $Ae^{-t/\tau}\sin(2\pi f_1 t)$, $t \geq 0$, has the spectrum magnitude $|X(f)| \approx \frac{A}{2}/\sqrt{(1/\tau)^2 + (2\pi(f-f_1))^2}$ near f_1; $|X|^2$ drops by 3 dB where $2\pi|f-f_1| = 1/\tau$, so the full $-3\,\text{dB}$ width is $B_k = 1/(\pi\tau) = 398\,\text{Hz}$. A filter matched to the burst has the same shape: a narrower band-pass rejects more noise but rings longer than the burst and starts late, a wider one passes more noise. A second-order band-pass with $B = 400\,\text{Hz}$ has exactly the decay time of the knock burst.
Pre-warping: $\Omega_0 = 2f_s\tan(\pi\cdot 0.26) = 53245\,\text{rad/s}$, $c = 2f_s = 50000$, $B \approx 2\pi\cdot 400/\cos^2(0.8168) = 5363\,\text{rad/s}$. Inserting $s = c(1-z^{-1})/(1+z^{-1})$ and multiplying by $(1+z^{-1})^2$:

$$H(z) = \frac{Bc\,(1-z^{-2})}{(c^2 + Bc + \Omega_0^2) + 2(\Omega_0^2 - c^2)z^{-1} + (c^2 - Bc + \Omega_0^2)z^{-2}}.$$

With $d = c^2 + Bc + \Omega_0^2$: $b_0 = Bc/d = 0.04786$, $b_1 = 0$, $b_2 = -b_0$, $a_1 = 2(\Omega_0^2 - c^2)/d = 0.11957$, $a_2 = (c^2 - Bc + \Omega_0^2)/d = 0.90428$. (The tool of 10.2, which places the edges exactly, gives $b_0 = 0.047898$, $a_1 = 0.119566$, $a_2 = 0.904204$.) Poles: $r = \sqrt{a_2} = 0.9509$, angle $\arccos(-a_1/2r) = 93.6° \mathrel{\hat{=}} 6.5\,\text{kHz}$. Decay time of $h[n] \propto r^n$: $\tau_f = -T_s/\ln r = 0.80\,\text{ms}$ – the filter "rings" exactly like the knock burst. Note that at $f_s = 25\,\text{kHz}$ the knock poles lie close to $\pm\text{j}$, so $|a_1|$ is small.

Exercise 10.2 Filter design tool

JSLINUX

Files in `code/jslinux/sheet10/`: `filtdesign.h` (design library), `filterdesign.c` (design, check, export), `coefquant.c` (word-length study), `filtertest.c` (test on the virtual engine).

```
gcc -O2 -o filterdesign filterdesign.c -lm && ./filterdesign    # writes knock_filter.h
gcc -O2 -o coefquant coefquant.c -lm && ./coefquant && ./coefquant 200000
gcc -O2 -o filtertest filtertest.c -lm && ./filtertest
```

a) *FIR window method.* Complete `fd_window()` (Hann, Hamming, Blackman, Kaiser) and `fd_fir_bandpass()`. `filterdesign` prints, for $N = 83$ and every window, the worst pass-band loss and stop-band gain, and the smallest odd N that meets the specification. Explain the ranking of the windows. Is the Kaiser estimate of 10.1 b) met?

b) *IIR Butterworth band-pass.* Complete `fd_prewarp()` and the pole transformation in `fd_butter_bandpass()`: every prototype pole p gives the band-pass poles $s = \frac{1}{2}(pB \pm \sqrt{p^2B^2 - 4\Omega_0^2})$, which are mapped with $z = (2f_s + s)/(2f_s - s)$. Check the specification, list the pole radii and frequencies of the sections, and compare the group delay at 6.5 kHz with the FIR filter. What goes wrong without pre-warping?

c) Complete `fd_bp2()` and compare with your hand calculation of 10.1 e). Report the attenuation of the 400-Hz and a 1000-Hz version at 8 kHz and 10.5 kHz.

d) *Export.* Complete the Q15/Q2.14 conversion (rounding, saturation) and generate `knock_filter.h`. Why are the biquad coefficients exported in Q2.14 rather than in Q15? Why are the sections scaled to a peak gain of 1 (L_∞ scaling) and ordered by pole radius?

e) *Coefficient quantisation.* Complete `quant_vec()` and the quantisation of a_1, a_2 in `coefquant.c`. The program realises the filter (1) as a biquad cascade and (2) as one 10th-order direct-form polynomial, quantises the coefficients to $B = 8 \ldots 32$ bits and reports the pole displacement, the largest pole radius, the pass-band deviation and the stop-band gain. Interpret the results for $f_s = 25\,\text{kHz}$ and for the same analog specification at $f_s = 200\,\text{kHz}$. Which word length do you need for the cascade, which for the direct form?

f) *Test on the engine signal.* Complete `sos_step()` (TDF II, float) and `sosq_step()` (DF I, Q15 data, Q2.14 coefficients, 32-bit accumulator, rounding, saturation) in `filtertest.c`. Compare the window energies with and without knock for FIR and IIR, the delay of the filtered burst, and the error of the Q15 implementation.

Solution

a) Core of the FIR design (`code/solutions/jslinux/sheet10/filtdesign.h`):

```
FD_STATIC void fd_fir_bandpass(double *h, int N, double f1, double f2, double fs,
                               int win, double beta)
{
    double w1 = 2.0 * M_PI * f1 / fs, w2 = 2.0 * M_PI * f2 / fs, g;
    int n;
    for (n = 0; n < N; n++) {
        double m = n - 0.5 * (N - 1);
        double hd = (fabs(m) < 1e-9) ? (w2 - w1) / M_PI
                                     : (sin(w2 * m) - sin(w1 * m)) / (M_PI * m);
        h[n] = hd * fd_window(win, n, N, beta);
    }
    g = c_abs(fd_fir_resp(h, N, 0.5 * (f1 + f2), fs));
    for (n = 0; n < N; n++) h[n] /= g;
}
```

Output of `filterdesign` (cut-offs 5.5/7.5 kHz):

window ($N = 83$)	pass min [dB]	pass max [dB]	stop max [dB]	min. N meeting spec
rectangular	−0.798	0.004	−25.0	> 201
Hann	0.000	0.076	−44.0	77
Hamming	−0.026	0.016	−52.9	77
Blackman	−0.301	0.001	−29.3	103
Kaiser ($\beta = 3.395$)	−0.154	0.016	−41.2	63

The rectangular window has the narrowest main lobe but side lobes of only $\approx -21\,\text{dB}$ (Gibbs), it never reaches 40 dB. Hann and Hamming have a main lobe twice as wide and side lobes of $-31/-41\,\text{dB}$; their stop-band attenuation ($\approx 44/53\,\text{dB}$) is sufficient. Blackman has $-58\,\text{dB}$ side lobes but the widest main lobe ($\approx 5.5/N$): at $N = 83$ the transition band is too wide and the stop-band edge is violated ($-29\,\text{dB}$ at 5 kHz); it needs $N = 103$. The Kaiser window trades main-lobe width against side-lobe level by β and reaches the specification with the shortest filter, $N = 63$; the empirical formula ($N = 57$) is slightly optimistic because it ignores the pass-band ripple and the grid evaluation. We keep the Hamming design with $N = 83$ (comfortable margin).

b) Pole transformation (`code/solutions/jslinux/sheet10/filtdesign.h`):

```
    for (k = 0; k < n; k++) {
        cplx p = c_scale(c_expj(M_PI * (2.0 * k + n + 1) / (2.0 * n)), rp);
        cplx pB, disc, sq, s1, s2;
        if (p.im < -1e-12) continue;              /* use upper half + real */
        pB = c_scale(p, B);
        disc = c_sub(c_mul(pB, pB), c_make(4.0 * W0 * W0, 0.0));
```

```
        sq = c_sqrt(disc);
        s1 = c_scale(c_add(pB, sq), 0.5);
        s2 = c_scale(c_sub(pB, sq), 0.5);
        if (fabs(p.im) < 1e-12) {                  /* real prototype pole:  */
            zp[ns++] = fd_bilinear(s1.im >= 0 ? s1 : s2, fs);  /* one conj pair */
        } else {                                   /* complex: two pairs    */
            zp[ns++] = fd_bilinear(s1, fs);
            zp[ns++] = fd_bilinear(s2, fs);
        }
    }
```

Result ($\lambda_s = 3.099$, $n = 5$, 5 biquads, each $b = [b_0, 0, -b_0]$):

section	b_0	a_1	a_2	$\lvert p\rvert$	f_p [Hz]
0	0.12634	0.11059	0.74733	0.8645	6505
1	0.10456	−0.03989	0.79089	0.8893	6161
2	0.10355	0.26523	0.79290	0.8905	6845
3	0.04236	−0.14173	0.91528	0.9567	5955
4	0.68429	0.37993	0.91667	0.9574	7045

Check: pass band $-1.000 \ldots 0$ dB, stop band ≤ -43.3 dB – specification met. Attenuation at 8 kHz: FIR 52.9 dB, IIR 44.3 dB; at 10.5 kHz: FIR 69.5 dB, IIR 106.7 dB (the IIR filter has 5-fold zeros at $z = -1$). Group delay at 6.5 kHz: IIR 22.4 samples (0.90 ms), FIR 41 samples (1.64 ms). The IIR filter is faster and needs 20 instead of 83 multiplications, but its phase is non-linear (group delay largest near the band edges) – irrelevant for an energy detector. Without pre-warping the analog edges $2\pi f$ are mapped by the bilinear transform to $f' = \frac{f_s}{\pi}\arctan(\pi f/f_s)$: 6 kHz → 5.09 kHz, 7 kHz → 5.69 kHz. The band would be shifted and compressed by about 1 kHz, i.e. the knock frequency would lie outside the pass band.

c) `fd_bp2`: $B = 400$ Hz: $b_0 = 0.047898$, $a_1 = 0.119566$, $a_2 = 0.904204$, $|p| = 0.95090$, peak at 6500 Hz, attenuation 18.1 dB at 8 kHz, 30.5 dB at 10.5 kHz, 38.3 dB at 1 kHz, group delay 19.9 samples. $B = 1000$ Hz: $b_0 = 0.112160$, $a_1 = 0.111496$, $a_2 = 0.775680$, $|p| = 0.8807$; only 10.4 dB at 8 kHz and 22.5 dB at 10.5 kHz, group delay 7.9 samples. A single biquad cannot meet the 40 dB specification; it is the cheap option for the Uno.

d) Q15 covers $[-1, 1)$ only, but biquad coefficients satisfy $|a_1| < 2$ ($a_1 = -2r\cos\theta$) and, after gain scaling, b_0 may exceed 1 as well. Q2.14 (one more integer bit) covers $[-2, 2)$ for all stable biquads. With Q15 data the product is Q29; shifting the 32-bit accumulator by 14 gives Q15. (For our filter at $f_s = 25$ kHz all coefficients happen to be < 1 because the poles lie near $\pm$j, but the header format must work for any design.) L_∞ scaling: every section has peak gain 1, so for sinusoidal inputs no intermediate signal exceeds the input amplitude – the condition for overflow-free fixed-point processing. Ordering by pole radius puts the most resonant sections (largest gain peaks, longest ringing, most sensitive to rounding noise) at the end, where the signal is already band-limited.

e) Output of `coefquant` for $f_s = 25$ kHz (max $|a_k|$ of the direct form: 7.03):

	biquad cascade, Q2.$(B-2)$				direct form, order 10			
B	max $\lvert\Delta p\rvert$	max $\lvert p\rvert$	pass dev	stop	max $\lvert\Delta p\rvert$	max $\lvert p\rvert$	pass dev	stop
8	4.8e-3	0.9601	0.68	−44.3	0.29	1.172	unstable	
10	1.1e-3	0.9581	0.38	−43.0	0.17	1.080	unstable	
12	2.0e-4	0.9576	0.06	−43.2	0.16	1.058	unstable	
14	7.8e-5	0.9575	0.01	−43.2	0.091	1.000	unstable	
16	1.6e-5	0.9574	0.00	−43.3	0.050	0.9623	1.99	−43.4
20	1.0e-6	0.9574	0.00	−43.3	3.5e-3	0.9579	0.10	−43.2
24	6.6e-8	0.9574	0.00	−43.3	7.1e-5	0.9574	0.00	−43.3

(pass dev = worst deviation in dB from the exact response in the pass band; stop = worst stop-band gain in dB.) The pole displacement of the cascade scales like the quantisation step

$2^{-(B-2)}$; already 10–12 bits are enough, 16 bits (Q2.14) are exact for all practical purposes. The direct form is unstable up to 14 bits and still shows 2 dB error at 16 bits: the roots of a high-order polynomial with clustered roots are extremely sensitive to its coefficients ($\partial p_i/\partial a_k = p_i^{N-k}/\prod_{j\neq i}(p_i - p_j)$, and the poles of a narrow band-pass are close to each other). At $f_s = 200\,\text{kHz}$ (the same analog filter, now $n = 6$, 12th order) the poles crowd near $z = 1$ (angle 10.7°, radius 0.9958): the cascade needs about 14 bits for 0.3 dB accuracy (12 bits: 1.8 dB, 8 bits: 25 dB error), and the direct form ($\max|a_k| = 806$) is unstable at *every* tested word length, even with 32 bits (and even in double precision its frequency response cannot be evaluated accurately). Lessons: always use second-order sections, and do not oversample a narrow-band IIR filter unnecessarily. For the single biquad ($B = 400\,\text{Hz}$, $f_s = 25\,\text{kHz}$) 8 bits already give $r = 0.95197$ instead of 0.95090 and a shift of 11 Hz; at $f_s = 200\,\text{kHz}$ 8 bits move the peak to 5651 Hz, 6 bits put the pole on the unit circle.

f) TDF II (float) and DF I (Q15) in `code/solutions/jslinux/sheet10/filtertest.c`:

```c
static float sos_step(sos_t *f, float x)
{
    int k;
    for (k = 0; k < KF_NSEC; k++) {
        const float *c = kf_sos[k];
        float y = c[0] * x + f->s1[k];
        f->s1[k] = c[1] * x - c[3] * y + f->s2[k];
        f->s2[k] = c[2] * x - c[4] * y;
        x = y;
    }
    return x;
}

/* ---- Q15 biquad cascade, direct form I, saturating ---------------- */
typedef struct { int16_t x1[KF_NSEC], x2[KF_NSEC], y1[KF_NSEC], y2[KF_NSEC]; long nsat; }
    sosq_t;
static int16_t sat16(int32_t v, long *nsat)
{
    if (v > 32767) { (*nsat)++; return 32767; }
    if (v < -32768) { (*nsat)++; return -32768; }
    return (int16_t)v;
}
static int16_t sosq_step(sosq_t *f, int16_t x)
{
    int k;
    for (k = 0; k < KF_NSEC; k++) {
        const int16_t *c = kf_sos_q14[k];
        int32_t acc = (int32_t)c[0] * x + (int32_t)c[1] * f->x1[k] + (int32_t)c[2] * f->x2[k]
                    - (int32_t)c[3] * f->y1[k] - (int32_t)c[4] * f->y2[k];
        int16_t y = sat16((acc + (1 << 13)) >> 14, &f->nsat);  /* Q29 -> Q15, rounded */
        f->x2[k] = f->x1[k]; f->x1[k] = x;
        f->y2[k] = f->y1[k]; f->y1[k] = y;
        x = y;
    }
    return x;
}
```

Output (3000 rpm, knock amplitude 0.3 V, onset 15° ATDC, noise 0.05 V):

window energy 10°–70° ATDC [V²]	knock	no knock	ratio
FIR (float, $N = 83$)	0.379	0.0199	19.0
IIR biquads (float, 5 sections)	0.327	0.0116	28.1
IIR biquads (Q15/Q2.14)	0.327	0.0116	28.2

The IIR filter separates knock and noise better: its pass band is narrower (noise bandwidth 1.16 kHz versus ≈ 2 kHz for the FIR filter whose −6 dB points are 5.5/7.5 kHz) and its delay is

smaller, so more of the burst falls into the window. Peak of the filtered burst after the knock onset: FIR 1.92 ms, IIR 1.44 ms (group delay plus build-up). The Q15 implementation deviates from the float filter by $5.6 \cdot 10^{-3}$ (-45 dB) relative rms error; no saturation occurred. The error is dominated by the rounding noise of the last, most resonant sections.

Exercise 10.3 Real-time filtering on the microcontroller

WOKWI

a) *ESP32, float.* Open `code/wokwi/sheet10_esp32_filter/` and replace the placeholder `knock_filter.h` by your generated header. The sketch samples the KNOCK pin at 25 kHz (deadline loop) and records 300 samples after each TDC of cylinder 1 (TDC pulse while CAM is HIGH). Complete `fir_step()` (circular buffer stored twice, so the newest N samples are always contiguous) and `iir_step()` (TDF II).

i. Run at 3000 rpm with knock probability 100 % and 0 %. Copy one record into JSLinux and plot x, y_{FIR}, y_{IIR} and the KNK flag. Compare the window energies printed in the `# rec` lines.

ii. Repeat at 6000 rpm. Explain the drastic change of the FIR window energy and propose a fix.

iii. Why does the sketch use `float` and not `double` on the ESP32? Compare the indicative benchmark of FIR and IIR with the operation counts.

b) *Uno, Q15.* Open `code/wokwi/sheet10_uno_q15/`. Timer1 interrupts at 25 kHz; the ISR reads the ADC, starts the next conversion and filters with the single biquad `kf_bp2_q14`.

i. Complete `biquad_q15()`: DF I, 32-bit accumulator (Q29), rounding, conversion to Q15, overflow detection, saturation or wrap-around (`SATURATE`). Verify it in `MODE 0`, where a recorded block is filtered with float and with Q15 arithmetic.

ii. Cycle budget: how many CPU cycles are available per sample? Measure the ISR duration (column `isr_cycles`, Timer1 count at the end of the ISR, or the pulse width on D8 with the logic analyzer). How many biquads, how many FIR taps would fit? Could the 10th-order cascade of 10.2 run on the Uno at 25 kHz?

iii. Scaling: the input is mapped by $x = (\mathrm{adc} - 512) \cdot 2^{\mathtt{SHIFT}}$. Which `SHIFT` uses the full Q15 range? Show that the band-pass output can still overflow when the *input* clips (hint: a clipped sine approaches a square wave). Run `MODE 1` with knock amplitude 3 V and `SHIFT` = 6, 8, 10 and compare saturation with wrap-around.

iv. Complete the window energy `energy += y*y >> 8`. Show that the 32-bit accumulator cannot overflow down to 600 rpm. Why is $y^2 \gg 8$ much cheaper on the AVR than $y^2 \gg 12$?

Solution

a) FIR with doubled circular buffer and cascade in TDF II (`code/solutions/wokwi/sheet10_esp32_filter/sketch.ino`):

```
static float fir_step(float x)
{
    fir_buf[fir_pos] = x;
    fir_buf[fir_pos + KF_FIR_N] = x;
    if (++fir_pos == KF_FIR_N) fir_pos = 0;
    const float *p = &fir_buf[fir_pos + KF_FIR_N - 1];   /* newest */
    float acc = 0.0f;
    for (int k = 0; k < KF_FIR_N; k++) acc += kf_fir[k] * p[-k];
    return acc;
}

/* ---------------- IIR: biquad cascade, transposed direct form II ---- */
static float s1[KF_NSEC], s2[KF_NSEC];
```

```
14
15 static float iir_step(float x)
16 {
17     for (int k = 0; k < KF_NSEC; k++) {
18         const float *c = kf_sos[k];          /* b0 b1 b2 a1 a2 */
19         float y = c[0] * x + s1[k];
20         s1[k] = c[1] * x - c[3] * y + s2[k];
21         s2[k] = c[2] * x - c[4] * y;
22         x = y;
23     }
24     return x;
25 }
```

i. The following values were obtained by running the sketch against the chip model in a PC test harness; Wokwi gives different random realisations but the same picture. 3000 rpm, knock probability 100 %, knock amplitude 0.5 V: window 10°–70° = samples 13–97, $E_{\mathrm{FIR}} = 0.45 \ldots 1.86$, $E_{\mathrm{IIR}} = 0.43 \ldots 1.79\,\mathrm{V}^2$. Without knock: $E_{\mathrm{FIR}} = 0.024 \ldots 0.040$, $E_{\mathrm{IIR}} = 0.015 \ldots 0.027\,\mathrm{V}^2$. In the plot the IIR output rises about 0.5 ms earlier than the FIR output; the KNK flag is HIGH for $3\tau = 2.4$ ms from the knock onset.
ii. 6000 rpm, knock amplitude 1 V: window = samples 6–48 (1.67 ms); $E_{\mathrm{FIR}} = 0.02 \ldots 0.51$, $E_{\mathrm{IIR}} = 0.81 \ldots 5.2\,\mathrm{V}^2$. The FIR delay of 41 samples equals 59° at 6000 rpm: the filtered burst leaves the window before it has fully arrived. Fix: shift the window by the group delay, i.e. evaluate samples $[n_{10} + 41,\ n_{70} + 41)$ of the FIR output (equivalently $[10° + \Delta\theta, 70° + \Delta\theta)$ with $\Delta\theta = \tau_g \omega$), or use the IIR filter with half the delay – and shift its window by ≈ 22 samples as well.
iii. The ESP32 FPU supports single precision only; `double` is emulated in software and is roughly an order of magnitude slower. For a 16-bit-class signal (12-bit ADC) float (24-bit mantissa) is more than sufficient. Per sample the FIR needs 83 multiply-adds, the cascade 25 multiplications (20 non-zero); the benchmark should show roughly this ratio (Wokwi timing is only indicative). At 240 MHz even the FIR uses only a few percent of the 40 µs sampling period.
b) i. Q15 biquad (`code/solutions/wokwi/sheet10_uno_q15/sketch.ino`):

```
1  static inline int16_t biquad_q15(int16_t x)
2  {
3      /* kf_bp2_q14 = {b0, b1, b2, a1, a2}, b1 = 0 for the band-pass     */
4      int32_t acc = (int32_t)kf_bp2_q14[0] * x
5                  + (int32_t)kf_bp2_q14[2] * bx2
6                  - (int32_t)kf_bp2_q14[3] * by1
7                  - (int32_t)kf_bp2_q14[4] * by2;      /* Q29 */
8      acc += 1L << 13;                                  /* rounding     */
9      /* Q29 -> Q15: bits 14..29 of acc. (acc << 2) >> 16 needs only byte
10        moves on the AVR, a loop ">> 14" would cost ~100 cycles.          */
11     int16_t y = (int16_t)(((uint32_t)acc << 2) >> 16);  /* wraps around */
12     if (acc >= (1L << 29) || acc < -(1L << 29)) {    /* |y| > 1: overflow */
13         n_ovf++;
14 #if SATURATE
15         y = acc > 0 ? 32767 : -32768;
16 #endif
17     }
18     bx2 = bx1; bx1 = x;
19     by2 = by1; by1 = y;
20     return y;
21 }
```

Since $b_1 = 0$ only 4 multiplications are needed. Block test (`MODE 0`, 400 samples with knock): maximum deviation from the float filter 1.8 LSB (Q15), rms 0.58 LSB – the rounding noise of the output, slightly amplified by the recursive part.
ii. 16 MHz/25 kHz = 640 cycles per sample, shared by the ISR, the serial interrupt and `loop()`. From the disassembly the ISR has about 180 instructions (of which $\approx$ 100 cycles are register

save/restore) plus 5 calls of the $16 \times 16 \to 32$ multiplication helper of ≈ 40 cycles each – an estimate of 450–500 cycles, i.e. about 75 % CPU load; measure it with `isr_cycles` or the D8 pulse. Each additional biquad costs $\approx 4 \cdot 40 + 50 \approx 200$ cycles, each FIR tap ≈ 45 cycles: at most one biquad or $\approx$ 3–4 FIR taps fit. The 5-section cascade (≈ 1000 cycles) or the 83-tap FIR (≈ 3700 cycles) are impossible at 25 kHz – one reason why the Uno is limited to the single matched biquad (and why production ECUs use DSP cores with single-cycle MAC).
iii. $|\text{adc} - 512| \le 512 = 2^9$; `SHIFT` $= 6$ maps the ADC range onto the full Q15 range. The band-pass has peak gain 1, so for any *sinusoidal* input with $|x| < 1$ its output stays below 1. With a larger shift the input saturates; a strongly clipped sine approaches a square wave whose fundamental has the amplitude $\frac{4}{\pi} \approx 1.27$ – the narrow band-pass extracts this fundamental and overflows. In the test (knock slider 3 V, 12 combustions, knock in 8 of them): `SHIFT` $= 6$ and 8: no overflow (the decaying burst is too short for the filter to build up to the square-wave fundamental); `SHIFT` $= 10$: 2 overflow events. With saturation the affected combustion has the energy $9.13 \cdot 10^7$, with wrap-around $6.28 \cdot 10^7$: a wrapped sample jumps from $+1$ to -1, which adds a large broadband error and falsifies the result, whereas saturation only clips. Saturation is therefore mandatory; the shift should be chosen such that the largest expected signal does not clip (`SHIFT` $= 6$ here, the small signals are still resolved with 2^6 Q15 units per ADC LSB).
iv. $|y| \le 2^{15}$, so $y^2 \gg 8 \le 2^{22}$. At 600 rpm 180° take 50 ms = 1250 samples, the window (60°) 417 samples: $417 \cdot 2^{22} = 1.75 \cdot 10^9 < 2^{32}$. A shift by 8 (or 16) bits is a byte move on the 8-bit AVR; a shift by 12 is compiled as a loop of 12 four-byte shifts (≈ 80 cycles).

Deliverables

10.1 on paper. `knock_filter.h` and the output of `filterdesign` and `coefquant` (both sampling rates); plot of one ESP32 record (x, y_{FIR}, y_{IIR}, KNK) at 3000 and 6000 rpm; Uno: block-test error, measured ISR cycles, table of overflow events for the three shifts with a short discussion.

Digital Signal Processing Exercise Sheet 11

Maximum Entropy Spectral Estimation

Script: Chapter 11 (AR models, Yule–Walker, Levinson–Durbin, Burg) **with solutions**

Learning goals

- Describe a resonance by an autoregressive (AR) model and relate the AR poles to resonance frequency and damping.
- Derive and solve the Yule–Walker equations with the Levinson–Durbin recursion; implement Burg's method; explain the maximum entropy principle.
- Compare MEM (AR) spectra with windowed periodograms on the very short records available in a knock window (32–128 samples): frequency accuracy, peak shape, resolution, artefacts.
- Select the model order (FPE, AIC, MDL) and recognise the symptoms of a wrong order (spurious peaks, line splitting).

Background

A knock window is short: 60° crank angle are 3.3 ms at 3000 rpm and only 1.7 ms at 6000 rpm, i.e. 42–83 samples at $f_s = 25\,\text{kHz}$ (the rate of Sheet 10). The DFT of N samples has the bin spacing f_s/N (781 Hz for $N = 32$), and windowing widens the peaks further. Parametric methods fit a model to the data and are not bound to this "Rayleigh limit".

An AR(p) process is white noise $e[n]$ (variance σ^2) filtered by an all-pole filter:

$$x[n] = -\sum_{i=1}^{p} a_i\, x[n-i] + e[n], \qquad A(z) = 1 + \sum_{i=1}^{p} a_i z^{-i}, \qquad P(f) = \frac{\sigma^2\, T_s}{\left|A(\mathrm{e}^{\mathrm{j}2\pi f/f_s})\right|^2}.$$

Multiplying by $x[n-m]$ and taking expectations gives the *Yule–Walker equations* $\sum_{i=1}^{p} a_i\, r[|m-i|] = -r[m]$, $m = 1, \ldots, p$, and $\sigma^2 = r[0] + \sum_i a_i r[i]$. The *Levinson–Durbin recursion* solves them order by order in $O(p^2)$ operations ($E_0 = r[0]$):

$$k_m = -\frac{r[m] + \sum_{i=1}^{m-1} a_i^{(m-1)} r[m-i]}{E_{m-1}}, \quad a_i^{(m)} = a_i^{(m-1)} + k_m\, a_{m-i}^{(m-1)}, \quad a_m^{(m)} = k_m, \quad E_m = (1-k_m^2)E_{m-1}.$$

Burg's method uses the same order recursion, but estimates every reflection coefficient k_m directly from the data by minimising the sum of forward and backward prediction error powers,

$$k_m = -\frac{2\sum_{n=m}^{N-1} f_{m-1}[n]\, b_{m-1}[n-1]}{\sum_{n=m}^{N-1}\left(f_{m-1}^2[n] + b_{m-1}^2[n-1]\right)}, \qquad \begin{aligned} f_m[n] &= f_{m-1}[n] + k_m b_{m-1}[n-1], \\ b_m[n] &= b_{m-1}[n-1] + k_m f_{m-1}[n], \end{aligned}$$

with $f_0[n] = b_0[n] = x[n]$. No autocorrelation estimate (and hence no implicit windowing of the data) is needed. Order selection criteria (E_p = prediction error power of order p, N = record length):

$$\mathrm{FPE}(p) = E_p \frac{N+p+1}{N-p-1}, \qquad \mathrm{AIC}(p) = N \ln E_p + 2p, \qquad \mathrm{MDL}(p) = N \ln E_p + p \ln N.$$

Code: `code/jslinux/sheet11/`: `ar.h` (AR library), `fft.h` (radix-2 FFT with the API of Sheet 8, `fft(re, im, n, inverse)` – you may use your own), `memspec.c` (Exercise 11.2), `resolve.c` (Exercise 11.3).

Exercise 11.1 AR models and the maximum entropy principle PAPER

a) AR(1): derive a_1 and σ^2 from $r[0], r[1]$ and the PSD. For which sign of a_1 is the process low-pass, for which high-pass?

b) AR(2): write down the Yule–Walker equations and solve them for a_1, a_2 in terms of $\rho_1 = r[1]/r[0]$, $\rho_2 = r[2]/r[0]$. Then run the Levinson–Durbin recursion by hand for the normalised autocorrelation $r = [1,\ -0.0627,\ -0.8973]$ and give $k_1, k_2, a_1, a_2, E_1, E_2$.

c) Show that an AR(2) model with complex poles $z_{1,2} = r\mathrm{e}^{\pm \mathrm{j}\theta}$ has $a_1 = -2r\cos\theta$, $a_2 = r^2$ and describes a resonance. Compute r, θ, a_1, a_2 for the knock mode ($f_1 = 6.5\,\mathrm{kHz}$, decay time $\tau = 0.8\,\mathrm{ms}$, $f_s = 25\,\mathrm{kHz}$) and compare with b). Estimate the $-3\,\mathrm{dB}$ bandwidth of the AR(2) peak from r.

d) *Maximum entropy.* Given $r[0], \ldots, r[p]$, there are infinitely many PSDs consistent with these values. Burg proposed to choose the one with maximum entropy rate $h = \frac{1}{4\pi}\int_{-\pi}^{\pi} \ln P(\omega)\,\mathrm{d}\omega$ (+const.). Show with a Lagrange argument that this PSD has the form $P(\omega) = 1/\sum_{|m|\le p} \lambda_m \mathrm{e}^{-\mathrm{j}\omega m}$, i.e. it is an AR($p$) spectrum. How does the MEM spectrum treat the unknown lags $|m| > p$, compared with a (windowed) periodogram?

e) Show that Burg's reflection coefficients always satisfy $|k_m| \le 1$. Why does this guarantee a stable (minimum-phase) $A(z)$ and why is that important? Does the Yule–Walker method with the *biased* autocorrelation estimate have the same property?

Solution

a) $r[1] = -a_1 r[0] \Rightarrow a_1 = -\rho_1$, $\sigma^2 = r[0] + a_1 r[1] = r[0](1-\rho_1^2)$, $P(f) = \sigma^2 T_s/|1 + a_1 \mathrm{e}^{-\mathrm{j}\omega}|^2 = \sigma^2 T_s/(1 + a_1^2 + 2a_1\cos\omega)$. For $a_1 < 0$ (pole $z = -a_1 > 0$, positive correlation) the PSD is largest at $\omega = 0$: low-pass; for $a_1 > 0$ (pole on the negative real axis) high-pass.

b) $\begin{pmatrix} r_0 & r_1 \\ r_1 & r_0 \end{pmatrix} \binom{a_1}{a_2} = -\binom{r_1}{r_2}$, hence

$$a_1 = -\frac{\rho_1(1-\rho_2)}{1-\rho_1^2}, \qquad a_2 = -\frac{\rho_2 - \rho_1^2}{1-\rho_1^2}.$$

Levinson: $E_0 = 1$; $k_1 = -\rho_1 = 0.0627$, $a_1^{(1)} = 0.0627$, $E_1 = (1-k_1^2) = 0.99607$; $k_2 = -(r_2 + a_1^{(1)} r_1)/E_1 = -(-0.8973 - 0.00393)/0.99607 = 0.9048$; $a_2 = k_2 = 0.9048$, $a_1 = a_1^{(1)} + k_2 a_1^{(1)} = 0.0627 \cdot 1.9048 = 0.1194$; $E_2 = (1-k_2^2)E_1 = 0.1806$. The prediction error drops from 1 to 0.18: the process is well predictable (narrow resonance).

c) $A(z) = (1 - z_1 z^{-1})(1 - z_2 z^{-1}) = 1 - 2r\cos\theta\, z^{-1} + r^2 z^{-2}$. The impulse response of $1/A(z)$ is $\propto r^n \sin((n+1)\theta)$: a damped oscillation with $\theta = 2\pi f/f_s$ and $r = \mathrm{e}^{-T_s/\tau}$. Knock mode: $\theta = 2\pi \cdot 0.26 = 93.6°$, $r = \mathrm{e}^{-0.05} = 0.9512$, $a_1 = -2 \cdot 0.9512 \cos 93.6° = 0.1195$, $a_2 = 0.9048$ – exactly the values of b): the autocorrelation given there is that of the knock resonance excited by white noise. For r close to 1 the $-3\,\mathrm{dB}$ bandwidth is $\Delta f \approx (1-r) f_s/\pi = 388\,\mathrm{Hz}$, in agreement with $1/(\pi\tau) = 398\,\mathrm{Hz}$ (Sheet 10).

d) Maximise $\int \ln P\,\mathrm{d}\omega$ subject to $\frac{1}{2\pi}\int P(\omega)\mathrm{e}^{\mathrm{j}\omega m}\,\mathrm{d}\omega = r[m]$, $|m| \le p$. With Lagrange multipliers λ_m the variation with respect to $P(\omega)$ gives $1/P(\omega) - \sum_{|m|\le p}\lambda_m \mathrm{e}^{\mathrm{j}\omega m} = 0$ (up to constants), so $1/P(\omega)$ is a non-negative trigonometric polynomial of degree p. By the Fejér–Riesz theorem it can be factorised as $|A(\mathrm{e}^{\mathrm{j}\omega})|^2/\sigma^2$ with a minimum-phase A of degree p: the maximum entropy PSD is the AR(p) spectrum, and the constraints are exactly the Yule–Walker equations. The MEM spectrum *extrapolates* the autocorrelation beyond lag p in the "most random" way consistent with the known lags (by the AR recursion). The periodogram implicitly sets all lags $|m| \ge N$ to zero and tapers the others (triangular/window lag weighting); this truncation causes the sinc main-lobe width $\approx f_s/N$ and the side lobes (leakage). MEM has neither, hence its sharp peaks from short records.

e) With $F = \sum f^2$, $B = \sum b^2$, $C = \sum fb$: Cauchy–Schwarz gives $|C| \le \sqrt{FB} \le \frac{1}{2}(F+B)$ (geometric $\le$ arithmetic mean), so $|k_m| = 2|C|/(F+B) \le 1$. In the Levinson structure $|k_m| < 1$

for all m is equivalent to all zeros of $A(z)$ inside the unit circle (Schur–Cohn), i.e. the synthesis filter $1/A(z)$ is stable, the prediction errors $E_m = (1-k_m^2)E_{m-1}$ stay positive and the spectrum is well defined. The biased autocorrelation sequence is positive semidefinite, so Yule–Walker with the biased estimate also yields $|k_m| \le 1$; the unbiased estimate (division by $N-m$) does not guarantee this and may produce unstable models.

Exercise 11.2 MEM vs. periodogram on short knock windows

JSLINUX

```
gcc -O2 -o memspec memspec.c -lm && ./memspec [N] [p]     # default N = 64, p = 8
```

a) Complete `ar_levinson()`, `ar_burg()` and `ar_psd()` in `ar.h`. Test them with the AR(2) example of 11.1 b) (you must get $a_1 = 0.1194$, $a_2 = 0.9048$).

b) Part (a) of `memspec` takes one knock window (N samples starting 10° after TDC, 3000 rpm, mean removed) and computes the Hann periodogram (zero padded to 1024), the Yule–Walker and the Burg spectrum. Plot `mem.csv` for $N = 32, 64, 128$ and describe the differences.

c) Part (b) estimates the knock frequency (maximum between 5.5 and 7.5 kHz) in 300 windows for $N = 32, 64, 128$ with four methods and reports mean, standard deviation, rms error, the -3 dB width of the peak and how often the second mode (10.5 kHz) is found. Interpret the table: in which respect is MEM better than the periodogram, in which not? Why does Burg's frequency estimate scatter more than the periodogram's?

d) Complete `ar_crit()`. Part (c) applies FPE, AIC and MDL to Burg models up to order 20. Which orders are chosen, and why are about 6–8 poles plausible for the knock signal?

e) Part (d) uses Burg with orders 2 to 32 for $N = 64$. Explain the growing number of peaks and the "split" knock peak, and give a rule of thumb for the order.

Solution

a) Levinson–Durbin (`code/solutions/jslinux/sheet11/ar.h`):

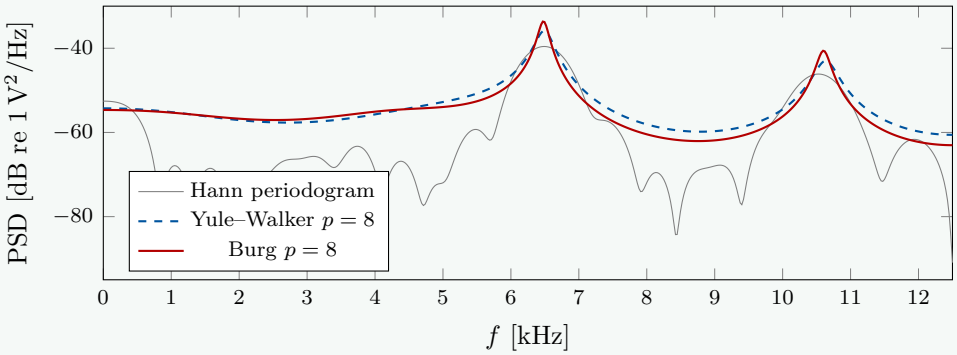
```
    for (m = 1; m <= p; m++) {
        double acc = r[m];
        for (i = 1; i < m; i++) acc += a[i] * r[m - i];
        k[m] = -acc / E[m - 1];
        for (i = 1; i < m; i++) tmp[i] = a[i] + k[m] * a[m - i];
        for (i = 1; i < m; i++) a[i] = tmp[i];
        a[m] = k[m];
        E[m] = (1.0 - k[m] * k[m]) * E[m - 1];
        if (fabs(k[m]) >= 1.0) return -1;
    }
```

Burg:

```
    for (m = 1; m <= p; m++) {
        double num = 0.0, den = 0.0;
        for (n = m; n < N; n++) {
            num += f[n] * b[n - 1];
            den += f[n] * f[n] + b[n - 1] * b[n - 1];
        }
        k[m] = den > 0.0 ? -2.0 * num / den : 0.0;
        /* update errors: go backwards in n so b[n-1] is still the old value */
        for (n = N - 1; n >= m; n--) {
            double fo = f[n];
            f[n] = fo + k[m] * b[n - 1];
            b[n] = b[n - 1] + k[m] * fo;
        }
        /* Levinson update of the polynomial */
        for (i = 1; i < m; i++) tmp[i] = a[i] + k[m] * a[m - i];
        for (i = 1; i < m; i++) a[i] = tmp[i];
```

```
        a[m] = k[m];
        E[m] = (1.0 - k[m] * k[m]) * E[m - 1];
    }
```

Note the backward loop over n: $b[n-1]$ must still hold the error of order $m-1$ when $f[n]$ and $b[n]$ are updated. With $r = [1, -0.0627, -0.8973]$ `ar_levinson` returns $a_1 = 0.1194$, $a_2 = 0.9048$, $E_2 = 0.181$.
b) Example window ($N = 64$, 2.56 ms, burst onset at 14.3° ATDC, $p = 8$): peak frequencies 6490 Hz (Hann periodogram), 6491 Hz (Yule–Walker), 6479 Hz (Burg); second mode at 10527/10634/10602 Hz.



The periodogram shows the two modes as broad main lobes (Hann, $N = 64$: main lobe $4f_s/N = 1.56$ kHz wide) with side lobes and a deep, ragged background. The AR spectra are smooth, have narrow peaks at both modes and a low-frequency peak (the combustion vibration that is not removed by subtracting the mean). Yule–Walker is smoother and its peaks are lower and broader than Burg's: the biased autocorrelation estimate corresponds to an implicitly windowed ("pre- and post-windowed") record. For $N = 32$ the periodogram lobes become twice as wide, while the Burg peaks remain sharp; for $N = 128$ all methods agree well.
c) Output of part (b):

N	method	mean [Hz]	std [Hz]	rms err [Hz]	−3 dB width [Hz]	10.5 kHz found
32	periodogram rect	6469	99	103	1043	83 %
32	periodogram Hann	6451	130	139	1540	75 %
32	Yule–Walker $p = 8$	6516	74	76	630	72 %
32	Burg $p = 8$	6487	106	107	376	74 %
64	periodogram rect	6494	21	22	519	100 %
64	periodogram Hann	6496	20	21	666	100 %
64	Yule–Walker $p = 8$	6515	37	40	382	86 %
64	Burg $p = 8$	6486	85	86	277	90 %
128	periodogram rect	6493	34	35	417	100 %
128	periodogram Hann	6496	35	35	377	98 %
128	Yule–Walker $p = 8$	6542	39	57	464	90 %
128	Burg $p = 8$	6532	84	90	354	89 %

Peak shape: the true knock spectrum has a −3 dB width of $1/(\pi\tau) \approx 400$ Hz. Burg reproduces it already from 32 samples (376 Hz); the periodograms show the width of their window main lobe (1043 Hz and 1540 Hz for $N = 32$) and approach the true width only for $N = 128$. This is the advantage of MEM: the spectral *shape* and the resolution are not limited by f_s/N.
Frequency of an isolated, strong peak: here the (zero-padded, interpolated) periodogram maximum is as good or better (rms error 21 Hz for $N = 64$); for a single sinusoid in white noise it is the maximum-likelihood estimator. A wide main lobe does not shift the maximum. Burg's estimate scatters by 80–100 Hz: (i) its frequency estimate of a sinusoid depends on the initial phase (known bias of the Burg method for short records), (ii) the forward–backward averaging

assumes a stationary signal, but the knock burst decays – backwards it grows, so the fitted poles are pushed towards the unit circle and their angles are disturbed; (iii) with $p = 8$ the model has to share its poles between two modes, the low-frequency vibration and the noise. A noise-free test (single damped sinusoid, $N = 64$, $p = 2$) confirms (i)/(ii): Burg finds 6300–6698 Hz depending on the phase and $r \approx 0.995$ instead of 0.951. For knock *detection* (Sheet 12) the exact frequency is irrelevant; what matters is the energy near 6.5 kHz relative to the background – and there MEM's clean, leakage-free spectrum of a 32-sample window is attractive.
d) Order selection (Burg, orders 1–20, 300 windows):

N	FPE: mean / most frequent	AIC	MDL
32	7.2 / 7	7.8 / 7	5.0 / 3
64	8.8 / 7	8.9 / 7	6.8 / 7
128	10.1 / 7	10.2 / 7	7.2 / 7

The signal contains two resonances (2 pole pairs), the low-frequency combustion vibration (1 pair) and noise: 6–8 poles are physically plausible, and all criteria mostly choose $p = 7$. FPE and AIC are asymptotically equivalent; they penalise the order weakly and tend to overestimate it for larger N (mean 10). MDL penalises with $\ln N$ and is consistent; for $N = 32$ it often selects only $p = 3$ (one resonance), because a short record does not justify more parameters.
e) Burg, $N = 64$:

p	peaks within 20 dB of max	knock peak split	rms error [Hz]
2	1.00	0 %	479
4	1.00	0 %	128
8	2.08	0 %	80
16	3.66	17 %	97
24	5.89	30 %	100
32	8.42	52 %	111

With $p = 2$ one pole pair has to describe everything (the estimate is pulled away from 6.5 kHz). For $p \gg$ (number of components) the surplus poles are placed close to the unit circle to fit noise details of the particular record: spurious peaks appear, and the knock peak splits into two (*line splitting*, typical for Burg at high order and high SNR). At $p = N/2$ the AR spectrum approaches the erratic behaviour of the periodogram (the model has as many parameters as there are independent lags). Rule of thumb: $p \approx 2 \times$ (number of expected resonances) + 2, and $p \leq N/3$; check with FPE/AIC/MDL.

Exercise 11.3 Resolving two close resonances

JSLINUX

A temperature-dependent shift of the knock mode in one cylinder, or a second structural mode of the engine block, can create two resonances only a few hundred Hz apart. Can the ECU still separate them within one knock window?

```
gcc -O2 -o resolve resolve.c -lm && ./resolve 500 && ./resolve 250 && ./resolve 1000
```

a) Complete `resolved()`: the two components count as resolved if there is a local maximum within $\Delta f/2$ of f_1 and one within $\Delta f/2$ of f_2, and the spectrum dips by at least 3 dB between them.

b) The program generates two sinusoids at $f_1 = 6.5$ kHz and $f_1 + \Delta f$ (random phases) in white noise with an SNR of 40, 20 and 10 dB per component and records how often the rectangular periodogram, the Hann periodogram and Burg ($p = N/3$, at most 32) resolve them for $N = 16 \ldots 256$. Determine the minimum record length for $\geq 90\,\%$ resolution for $\Delta f = 500$ Hz and compare with the Rayleigh limit $N \approx f_s/\Delta f$. How does the SNR influence the result?

c) Repeat for $\Delta f = 250$ and 1000 Hz. At $\Delta f = 500$ Hz and 6000 rpm, a 60° window has 42 samples: which method can resolve the two modes?

d) Part (b) of the program uses *damped* modes with $\tau = 0.8$ ms, as in a real knock burst. Explain why now even long records and high SNR do not help for $\Delta f = 250$ Hz and why Burg's success rate saturates at about 50–60 % for $\Delta f = 500$ Hz.

Solution

a) See `code/solutions/jslinux/sheet11/resolve.c`:

```c
static int resolved(const double *P, int nf, double dfr)
{
    double f2 = F1 + df_g, lo = F1 - df_g, hi = f2 + df_g;
    int i, i1 = -1, i2 = -1, k0 = (int)(lo / dfr), k1 = (int)(hi / dfr), kmid, imin;
    if (k1 > nf - 2) k1 = nf - 2;
    for (i = k0 + 1; i <= k1; i++) {
        double f = i * dfr;
        if (P[i] > P[i - 1] && P[i] >= P[i + 1]) {          /* local max */
            if (fabs(f - F1) < 0.5 * df_g && (i1 < 0 || P[i] > P[i1])) i1 = i;
            if (fabs(f - f2) < 0.5 * df_g && (i2 < 0 || P[i] > P[i2])) i2 = i;
        }
    }
    if (i1 < 0 || i2 < 0) return 0;
    imin = i1;
    for (kmid = i1; kmid <= i2; kmid++) if (P[kmid] < P[imin]) imin = kmid;
    return P[imin] < 0.5 * (P[i1] < P[i2] ? P[i1] : P[i2]);   /* 3 dB dip */
}
```

b) $\Delta f = 500$ Hz, percentage of resolved trials (200 trials each):

		SNR 40 dB			SNR 20 dB			SNR 10 dB			damped, 40 dB		
N	p	rect	Hann	Burg	rect	Hann	Burg	rect	Hann	Burg	rect	Hann	Burg
16	5	0	0	32	0	0	1	0	0	0	0	0	28
24	8	0	0	60	0	0	22	0	0	0	0	0	34
32	10	4	0	84	2	0	58	0	0	6	1	0	44
48	16	37	10	100	39	11	100	37	4	92	31	6	46
64	21	56	25	100	49	21	100	56	24	100	34	26	50
96	32	100	50	100	100	46	100	100	46	100	39	42	58
128	32	100	100	100	100	100	100	100	100	100	36	54	58
256	32	100	100	100	100	100	100	100	100	100	35	100	58

Minimum N for ≥ 90 %: periodogram rect 96, Hann 128 (independent of the SNR), Burg 48 at 40 dB and 20 dB (84 % already at $N = 32$ for 40 dB), 48 at 10 dB. The Rayleigh limit $f_s/\Delta f = 50$ samples is the record length at which the main lobes (width $2f_s/N$ for rect, $4f_s/N$ for Hann between the first zeros) start to separate; with random phases and the 3-dB criterion about twice this length is needed (rect: 56 % at $N = 64$, Hann: about 2× longer). The periodogram's resolution is a property of the window and does not improve with SNR. Burg beats the Rayleigh limit by a factor 2 at moderate SNR – the "super-resolution" of MEM –, but its advantage shrinks with decreasing SNR: at 10 dB it needs $N = 48$ and fails completely below $N = 32$, because the noise has to be modelled by the same few poles.

c) $\Delta f = 250$ Hz (Rayleigh limit 100 samples): rect needs $N = 256$, Hann 256, Burg 64 (94 %) at 40 dB, 96 at 20 and 10 dB. $\Delta f = 1000$ Hz (Rayleigh 25): rect $N = 48$, Hann 64, Burg 24 at 40/20 dB and 32 (99 %) at 10 dB. With 42 samples (60° at 6000 rpm) and $\Delta f = 500$ Hz, only Burg can separate the modes (and only at SNR $\gtrsim 10$ dB); the periodogram resolves them in less than 40 % of the cases.

d) A damped mode has a Lorentzian line of width $1/(\pi\tau) = 400$ Hz – this is a property of the signal, not of the estimator. Two such lines 250 Hz apart overlap into a single peak, so no method can show two maxima: the success rate stays at 0–28 % for all N (output of `./resolve 250`,

damped part). At $\Delta f = 500\,\mathrm{Hz}$ the lines are just separable; a long rectangular window does not help either (the signal has decayed after $\approx 3\tau = 60$ samples, more samples only add noise), the Hann window at $N = 256$ succeeds because it suppresses the discontinuity at the burst onset. Burg saturates at 50–60 %: the forward–backward error criterion assumes stationarity, so for decaying signals it biases the pole radii towards 1 and the pole angles depending on the phases (cf. 11.2 c). For damped exponentials, methods that model the decay explicitly (forward linear prediction / covariance method, Prony) are more appropriate. For $\Delta f = 1000\,\mathrm{Hz}$ Burg resolves the damped modes in 90–98 % of the cases from $N = 24$ on.

Deliverables

11.1 on paper. Completed `ar.h`; plots of `mem.csv` for $N = 32, 64, 128$; the tables of `memspec` parts (b)–(d) and of `resolve` for $\Delta f = 250, 500, 1000\,\mathrm{Hz}$ with a half-page interpretation: when would you use MEM in a knock controller, and when not?

Digital Signal Processing Exercise Sheet 12

Knock Detection

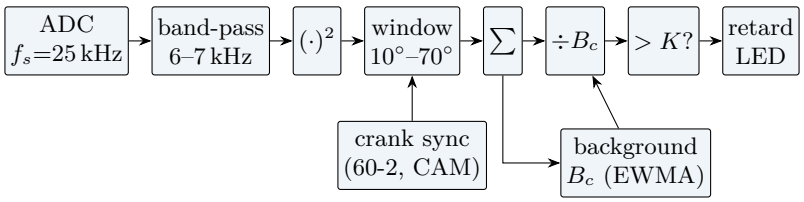
Script: Chapter 12 (capstone: angle-synchronous knock detection) **with solutions**

Learning goals

- Assemble the methods of the previous sheets into a complete, angle-synchronous knock detector and justify every block quantitatively.
- Evaluate a detector statistically on thousands of labelled combustions: ROC curve, area under the curve, detection rate at a fixed false-alarm rate, choice of the threshold.
- Compare energy-, FFT- and MEM-based knock features and quantify the contribution of band-pass, angle window and background normalisation.
- Run the detector in real time on the ESP32 with crank synchronisation, per-cylinder knock indication and ignition retard; optionally port it to the 8-bit Uno in fixed point.

The signal chain

The ECU does not see the engine. It sees only signals. The ECU never sees the knock – only a vibration signal that also contains noise, the combustion vibration and valve impacts. The standard knock detector of a production ECU is the chain below; every block is one of the previous sheets.



Conventions: $f_s = 25\,\text{kHz}$ (Sheet 10), band-pass = the 10th-order Butterworth biquad cascade of `knock_filter.h` (Sheet 10; group delay 22 samples at 6.5 kHz) or the single biquad `kf_bp2` ($B = 400\,\text{Hz}$, 20 samples), window 10°–70° after the TDC of the firing cylinder, feature $E_c = \sum_{\text{window}} y^2[n]$ for cylinder c, normalised $R_c = E_c/B_c$, knock if $R_c > K$. Ground truth: `e.knock_flag[c]` in the C model, the KNK pin of the engine-sim chip in Wokwi. Code: `code/jslinux/sheet12/` (`knockeval.c` plus `knock_filter.h`, `ar.h`, `fft.h` – use your own versions from Sheets 8, 10, 11 if you like) and `code/wokwi/sheet12_knock_esp32/`, `code/wokwi/sheet12_knock_uno/`.

Exercise 12.1 Detector design

PAPER

a) Justify every block of the chain with the respective chapter: why sampling at 25 kHz (and what about anti-aliasing), why an angle window and why exactly 10°–70° ATDC, why a band-pass, why squaring (instead of a linear operation), why summation, why a normalisation and why a threshold. Which blocks are LTI, which are not?

b) Compute the window length in samples at 1500, 3000 and 6000 rpm. By how many degrees must the window be shifted to compensate the group delay of the band-pass (22 samples)? Why is a fixed sample shift correct although the window is defined in degrees?

c) Energy budget. Show that a knock burst $A\mathrm{e}^{-t/\tau}\sin(2\pi f_1 t)$ that lies completely in the window and is passed with gain 1 contributes $E_k \approx f_s A^2 \tau/4$ (in V^2, sum over samples). White noise with standard deviation σ contributes on average $E_n = N_w \sigma^2 \cdot 2B_n/f_s$, where N_w is the window length and B_n the noise bandwidth of the band-pass ($B_n = \int_0^{f_s/2} |H(f)|^2\,\mathrm{d}f = 1161\,\text{Hz}$ for the cascade). Evaluate E_k for $A = 0.2\,\text{V}$ and E_n for $\sigma = 0.05$ and $0.15\,\text{V}$ at the three speeds. What do you expect for the detection performance?

d) Without knock, E_c is approximately $E_n \cdot \chi^2_\nu/\nu$ distributed with $\nu \approx 2B_nT_w$ degrees of freedom (T_w = window duration). Compute ν for the three speeds and, using the 99 % quantiles $\chi^2_{\nu,0.99}/\nu \approx 2.0$ ($\nu = 15.5$), 2.6 ($\nu = 7.7$), 3.4 ($\nu = 3.9$), the threshold for 1 % false alarms in units of the mean noise energy. Why does a fixed threshold K give an rpm-dependent false-alarm rate?

e) Background normalisation: propose an update rule for B_c that follows slow changes (sensor ageing, rpm, load) but is not pulled up by knocking combustions. Why one background per cylinder?

f) Control action: on a detected knock the ignition of that cylinder is retarded by Δ_r, without knock it is advanced again by $\Delta_a \ll \Delta_r$ per combustion. Explain the asymmetry and what a false alarm costs.

Solution

a) *Sampling* (Ch. 4/5): the knock modes (6.5/10.5 kHz) need $f_s > 21$ kHz; 25 kHz fits the 50 kHz update of the chip (Sheet 10). A real ECU needs an analog anti-aliasing low-pass below 12.5 kHz, otherwise broadband vibration above $f_s/2$ folds into the knock band. *Crank sync and angle window* (Ch. 1, 4): knock can only occur after the spark, during the combustion of one cylinder, typically 5–30° ATDC, and decays within a few ms; the window excludes the noise of the rest of the cycle and the valve impacts at 90° ATDC (and, in a real engine, injector and piston-slap noise). Since the events are fixed in *angle*, the window must be angle based (time-based windows would move with rpm). 10° start: the combustion vibration and spark noise before; 70° end: before the valve impact. *Band-pass* (Ch. 10): selects the knock resonance and rejects the low-frequency combustion vibration (0.2 V, much larger than light knock) and most of the broadband noise (gain in SNR $\approx (f_s/2)/B_n \approx 11$, i.e. 10 dB). *Squaring* (Ch. 2, nonlinear memoryless system): the knock oscillation has random phase, its linear average is zero; the energy (or the rectified amplitude) is phase independent. *Summation* (Ch. 3, LTI, moving sum): averages the fluctuations of y^2 over the window (matched to the burst duration). *Normalisation*: the absolute energy depends on rpm (window length), sensor sensitivity, mounting position, engine load and ageing; the ratio to the background is dimensionless and comparable. *Threshold*: a binary decision (Ch. 1: from signal to information) with a trade-off between misses and false alarms. LTI: band-pass, summation (with a fixed window). Not LTI: windowing (time-variant, angle-synchronous), squaring (nonlinear), normalisation and threshold (nonlinear, adaptive).
b) $f_s = 25$ kHz, 60° take $60/(6 \cdot \text{rpm})$ s: 6.67 ms = 167 samples at 1500, 3.33 ms = 83 at 3000, 1.67 ms = 42 at 6000 rpm. The delay of $22T_s = 0.88$ ms corresponds to 7.9°, 15.8°, 31.7°. The filter delays the signal by a fixed *time*: the output sample $y[n]$ belongs to the input at $n - 22$; hence the window decision for $y[n]$ must use the crank angle of sample $n - 22$ (in code: a ring buffer of the last angles). Expressed in degrees the shift grows with rpm.
c) $\sum_n y^2[n] \approx f_s \int_0^\infty A^2 \mathrm{e}^{-2t/\tau} \sin^2(2\pi f_1 t)\,\mathrm{d}t = f_s A^2 \cdot \frac{\tau}{2} \cdot \frac{1}{2} = f_s A^2 \tau/4 = 5A^2$ (V^2) $\Rightarrow E_k = 0.20\,\text{V}^2$ for $A = 0.2$ V (the knock amplitude of the model varies between 0.5 and 1.5 times this value, i.e. $E_k = 0.05 \ldots 0.45$, and part of a late burst falls outside the window). Noise: $E_n = N_w\sigma^2 \cdot 2 \cdot 1161/25000$:

rpm	N_w	E_n, $\sigma = 0.05$ V	E_n, $\sigma = 0.15$ V
1500	167	0.039	0.348
3000	83	0.019	0.174
6000	42	0.010	0.087

At $\sigma = 0.05$ V the knock energy is 5–20 times the mean noise energy: good detection, better at high rpm (shorter window, less noise). At $\sigma = 0.15$ V the ratio is only 0.6–2.3: weak knocks are buried in the noise, the detection rate at low false-alarm rates will be poor, especially at low rpm.
d) $\nu = 2B_nT_w = 15.5$, 7.7, 3.9 at 1500, 3000, 6000 rpm; threshold for 1 % false alarms ≈ 2.0,

2.6, 3.4× the mean noise energy. With few degrees of freedom (short window) the noise energy fluctuates more, the χ^2 distribution has a heavier tail, and a higher threshold is needed for the same false-alarm rate. A fixed K is therefore conservative at low rpm and produces more false alarms at high rpm; production ECUs use rpm-dependent thresholds (maps). Note that normalisation removes the *mean* dependence on rpm but not the *shape* of the distribution.
e) Exponentially weighted moving average with clipping: $B_c \leftarrow B_c + \lambda(\min(E_c, 3B_c) - B_c)$, $\lambda \approx 0.05$ (time constant ≈ 20 combustions of that cylinder, i.e. about 1.6 s at 3000 rpm), larger λ during the start (warm-up). The clipping limits the influence of a knocking combustion (alternative: update only with combustions classified as non-knocking – but then a wrong decision can lock the background). One background per cylinder because the transfer path from each cylinder to the sensor differs (distance, block structure): the same knock intensity gives different sensor amplitudes.
f) Knock can damage the engine within few cycles – fast reaction ($\Delta_r \approx 1.5°$ per event). Retard costs efficiency and torque, so the ignition is advanced again slowly ($\Delta_a \approx 0.05°$ per combustion) towards the knock limit; the controller then oscillates closely below the limit. A false alarm costs efficiency (Δ_r that is recovered only after $\Delta_r/\Delta_a = 30$ combustions of that cylinder); a miss risks damage. Hence the threshold is chosen for a low but non-zero false-alarm rate (e.g. 1 %).

Exercise 12.2 Offline evaluation with ground truth

JSLINUX

`knockeval.c` runs the virtual engine at 1500, 3000 and 6000 rpm with sensor noise $\sigma = 0.05$ and 0.15 V (6 scenarios, 1000 evaluated combustions each, knock probability 0.2, knock amplitude 0.2 V×(0.5 . . . 1.5), light knock) and computes for every combustion eight features, each normalised by its own per-cylinder background:

WB-seg / WB-win	wide-band energy $\sum x^2$ over the whole 180° segment / in the window
BP-seg / BP-win	band-pass (cascade) energy over the segment / in the delay-compensated window
BP2-win	as BP-win with the single biquad ($B = 400$ Hz)
FFT / FFT-rect	power 6–7 kHz of the zero-padded FFT of the raw window samples, Hann / no window
MEM	power 6–7 kHz of the Burg AR(8) spectrum of the raw window samples

```
gcc -O2 -o knockeval knockeval.c -lm && ./knockeval [knock_amp] [ncomb]
```

a) Complete the window logic: integrate x^2 in the window 10°–70° ATDC and y^2 (cascade, single biquad) while the *delayed* angle is in the window.

b) Complete the background update (clipped EWMA, faster during the first 16 combustions of each cylinder).

c) Complete the ROC computation in `roc()`: the combustions are sorted by decreasing feature value; every threshold between two values gives a point (FPR, TPR). Compute the AUC (trapezoidal rule) and the detection rate at ≤ 1 % false alarms.

d) Complete the FFT and MEM features (`fft.h`, `ar.h`).

e) Run the program and interpret the AUC and detection tables: what is the contribution of the band-pass, of the angle window, of the delay compensation, of the normalisation (compare the pooled ROC with and without)? Plot `roc.csv`.

f) Choose a threshold K for 1 % false alarms over all scenarios and compare the per-scenario thresholds with your prediction from 12.1 d).

g) Compare the energy, FFT and MEM features. Why is the Hann-windowed FFT so much worse at 1500 rpm than the rectangular one? Which feature would you implement on the microcontroller, and why?

Solution

a)–d) Window logic and background update (`code/solutions/jslinux/sheet12/knockeval.c`):

```
                /* background update: knock-robust EWMA (values clipped at
                   3 x background), faster adaptation during the warm-up */
                for (j = 0; j < NFEAT; j++) {
                    double v = f[j] < 3.0 * bg[cur][j] ? f[j] : 3.0 * bg[cur][j];
                    bg[cur][j] += (nbg[cur] < WARMUP ? LAMBDA_W : LAMBDA) * (v - bg[cur][j]);
                }
                nbg[cur]++;
            }
            first = (cur < 0);                /* the first segment is incomplete */
            cur = lc;
            label = e.knock_flag[cur];
            nw = 0;
            for (j = 0; j < NFEAT; j++) acc[j] = 0.0;
        }
        d = es_angle_diff(e.theta, es_tdc_deg[cur]);     /* deg ATDC */
        dh[n & 31] = d;
        dS = dh[(n - D_SOS) & 31];                        /* angle D samples ago */
        dB = dh[(n - D_BP2) & 31];
        n++;
        acc[0] += x * x;
        acc[2] += y * y;
        if (d >= WIN0 && d < WIN1) {
            acc[1] += x * x;
            if (nw < MAXW) xw[nw++] = x;
        }
        if (dS >= WIN0 && dS < WIN1) acc[3] += y * y;
        if (dB >= WIN0 && dB < WIN1) acc[4] += y2 * y2;
```

ROC (after sorting by decreasing value; ties are merged):

```
    for (i = 0; i < m; i++) {
        double fpr, tpr;
        if (v[i].l) tp++; else fp++;
        if (i < m - 1 && v[i + 1].v == v[i].v) continue;   /* ties */
        fpr = (double)fp / Nn; tpr = (double)tp / P;
        auc += 0.5 * (tpr + last_tpr) * (fpr - last_fpr);
        if (fpr <= 0.01) { *tpr1 = tpr; *thr1 = v[i].v; }
        if (tpr_grid)
            for (g = 0; g < NGRID; g++) if (fpr <= g / (NGRID - 1.0) && tpr > tpr_grid[g])
    tpr_grid[g] = tpr;
        last_fpr = fpr; last_tpr = tpr;
    }
```

e) Output (6000 combustions, 1210 with knock = 20.2 %):

AUC		WB-seg	WB-win	BP-seg	BP-win	BP2-win	FFT	FFT-rect	MEM
1500	0.05	0.799	0.788	0.957	**0.981**	0.978	0.804	0.973	0.974
1500	0.15	0.600	0.619	0.738	**0.789**	0.784	0.620	0.764	0.761
3000	0.05	0.811	0.819	0.985	**0.998**	0.996	0.955	0.997	0.997
3000	0.15	0.606	0.652	0.747	**0.828**	0.818	0.717	0.824	0.818
6000	0.05	0.912	0.922	0.988	**0.999**	0.998	0.997	0.999	0.999
6000	0.15	0.698	0.726	0.822	**0.886**	0.864	0.826	0.878	0.858
	pooled	0.740	0.752	0.897	**0.934**	0.927	0.846	0.927	0.924
det. @1 % FA									
1500	0.05	20.7	26.4	77.7	82.9	82.4	34.2	80.3	80.8
1500	0.15	3.5	2.0	7.0	20.4	14.4	5.5	15.4	18.9
3000	0.05	29.7	28.2	80.9	95.7	90.0	66.5	92.8	91.9
3000	0.15	2.1	3.7	16.8	30.9	24.6	14.1	27.7	24.6
6000	0.05	52.6	56.5	87.1	96.2	94.3	93.3	97.1	96.7
6000	0.15	4.3	8.7	29.0	37.2	34.8	24.6	31.9	24.6
	pooled	11.8	9.3	44.5	59.1	55.9	40.3	57.1	56.4

Band-pass: the decisive block. Without it (WB) the combustion vibration and the noise of all frequencies dominate; AUC 0.74–0.75 pooled, only 9–12 % detection at 1 % false alarms. With it (BP-seg) the AUC rises to 0.90. *Angle window:* BP-win vs. BP-seg: AUC 0.897 → 0.934, detection 44.5 % → 59.1 %; integrating over 180° collects three times more noise energy (and the valve burst, which however is strongly attenuated by the cascade, −44 dB at 8 kHz). For the wide-band features the window helps little because the low-frequency combustion vibration lies inside the window. *Delay compensation:* included in BP-win; without it, at 6000 rpm a 32° portion of the burst would be lost (cf. Sheet 10, 10.3). *Normalisation:* pooled over all scenarios the raw BP-win energy reaches only AUC 0.812 and 9.8 % detection at 1 % false alarms, the normalised ratio 0.934 and 59.1 %: the mean noise energy differs by a factor of 36 between the scenarios (Table in 12.1 c), so no single absolute threshold fits. *Noise:* at $\sigma = 0.15$ V even the best feature detects only 20–37 % of the (light) knocks at 1 % false alarms – consistent with the energy budget of 12.1 c). The ROC curves (`roc.csv`) of BP-win lie far above those of WB-win; at FPR = 5 % the pooled TPR of BP-win is 74 % (WB-win: 28 %).

f) Pooled threshold for 1 % false alarms: $R > 2.00$ (detection 59.1 %) – this is the value $K = 2$ used in 12.3. Per scenario: $R > 1.57$, 1.54, 2.02 ($\sigma = 0.05$ V; 1500/3000/6000 rpm) and 1.79, 2.04, 2.49 ($\sigma = 0.15$ V). The mean ratio of the non-knocking combustions is below 1 (0.54–0.91), because the background contains (clipped) knock energy; in units of the mean noise energy the thresholds are 2.3/2.0 (1500), 2.7/2.4 (3000) and 3.7/3.2 (6000 rpm) – in good agreement with the χ^2 prediction 2.0, 2.6, 3.4 of 12.1 d): the shorter the window, the higher the threshold must be for the same false-alarm rate.

g) The band-pass energy (cascade) is the best and cheapest feature; the single biquad (BP2) is almost as good (AUC 0.927 vs. 0.934): the matched 400-Hz band-pass rejects noise as well as the steep cascade – the Uno version is therefore not much worse. FFT-rect and MEM are comparable to BP-win at $\sigma = 0.05$ V and slightly worse at high noise; they cost much more (FFT of 256 points, or Burg + spectrum evaluation, per combustion) and give no extra information here, since there is only one knock mode in the band. The Hann-windowed FFT is poor at 1500 rpm (AUC 0.80 vs. 0.97): the window is 6.7 ms long, but the burst starts at the beginning of the window (onset 5°–25° ATDC) and has decayed after ≈ 2 ms – exactly where the Hann window is close to zero. At 6000 rpm (window 1.7 ms) the burst fills the window and the difference vanishes. Spectral features become interesting if several knock modes must be weighted, if the knock frequency varies, or to distinguish knock from disturbances at nearby frequencies with very short windows (MEM, Sheet 11). Recommendation for the microcontroller: band-pass energy in the delay-compensated angle window, normalised per cylinder.

Limitation of the model: the valve impacts and the combustion vibration of the virtual engine are deterministic (same amplitude every cycle); a constant disturbance only shifts the feature

and is largely removed by the normalisation. In a real engine these disturbances vary from cycle to cycle and the angle window becomes even more important.

Exercise 12.3 Real-time knock detector

WOKWI

Open `code/wokwi/sheet12_knock_esp32/` (ESP32 + engine-sim chip, 4 knock LEDs, retard LED, logic analyzer). Pins: KNOCK → GPIO34, CRANK → 18, CAM → 19, TDC → 21 (only to check the synchronisation), KNK → 22 (ground truth, only for the evaluation); LEDs cylinder 1–4 on 25, 26, 27, 14; retard PWM on 13. The sketch samples at 25 kHz (deadline loop), filters with `kf_sos` and prints one line `cyl,rpm,R,knock,truth,retard` per combustion and every 200 combustions the confusion counts.

a) *Crank sync.* Complete the 60-2 decoder in the CRANK interrupt (gap detection: a tooth period more than twice the previous one; revolution from the CAM level at tooth 10) and the angle interpolation `crank_angle()`. The sketch compares the decoded angle at every TDC pulse with 120/300/480/660° and prints the mean error. What limits the accuracy?

b) *Window.* Complete the integration of y^2 while the delayed angle (ring buffer `th_hist`, `KF_DELAY` = 22 samples) is in the window.

c) *Decision and action.* Complete `finish_combustion()`: $R = E/B_c$, knock if $R > K$ ($K =$ 2 from 12.2), background update, ignition retard (+1.5° per knock, −0.05° per knock-free combustion, max. 9°).

d) Run the detector at 1500, 3000, 6000 rpm with knock amplitude 1.0 and 0.2 V and noise 50 and 150 mV (knock probability 20 %). Report detection and false-alarm rates and compare them with 12.2. Watch the LEDs and the retard output.

e) Estimate the processing time per sample on the ESP32 and the UART load at 6000 rpm. Why must the sketch not print the samples?

f) The engine-sim chip has no spark input: the retard does not influence the knock probability. Describe how a closed knock control loop would behave and how you would test it.

Solution

a) Decoder (`code/solutions/wokwi/sheet12_knock_esp32/sketch.ino`):

```
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
volatile uint32_t t_edge = 0;      /* time of the last rising edge, us   */
volatile uint32_t t_tooth = 0;     /* duration of one 6 deg tooth, us    */
volatile int tooth = -1;           /* 0..57, -1: no gap seen yet         */
volatile int rev = -1;             /* 0: theta 0..360, 1: 360..720       */

void IRAM_ATTR on_crank()
{
    uint32_t t = micros();
    portENTER_CRITICAL_ISR(&mux);
    uint32_t dt = t - t_edge;
    t_edge = t;
    if (t_tooth > 0 && dt > 2 * t_tooth) {   /* gap: 18 deg instead of 6 */
        tooth = 0;
        if (rev >= 0) rev ^= 1;               /* next revolution          */
    } else {
        if (tooth >= 0) tooth++;
        if (tooth > 57) tooth = -1;           /* lost sync                */
        t_tooth = dt;
    }
    if (tooth == 10) rev = digitalRead(PIN_CAM) ? 0 : 1;   /* phase from CAM
                                        (HIGH for 0..180 deg), checked every rev. */
    portEXIT_CRITICAL_ISR(&mux);
}
```

```
25
26 /* crank angle at time t (deg, 0..720) or -1 if not synchronised */
27 static float crank_angle(uint32_t t)
28 {
29     portENTER_CRITICAL(&mux);
30     uint32_t te = t_edge, tt = t_tooth;
31     int k = tooth, r = rev;
32     portEXIT_CRITICAL(&mux);
33     if (k < 0 || r < 0 || tt == 0) return -1.0f;
34     float frac = (float)(int32_t)(t - te) / tt;   /* teeth since the last edge */
35     float lim = (k == 57) ? 3.0f : 1.0f;     /* tooth 57: 18 deg gap follows */
36     if (frac > lim) frac = lim;
37     if (frac < -1.0f) frac = -1.0f;
38     return 360.0f * r + 6.0f * (k + frac);
39 }
```

After the first gap (at most one revolution) the tooth counter is valid; the revolution flag is toggled at every gap and confirmed by the CAM level at tooth 10 (CAM HIGH for 0°–180°). Mean TDC sync error (PC test of the sketch with the chip model): $\leq 0.01°$ at all speeds – the TDC pulses coincide with the rising edges of teeth 20 and 50, so this check verifies the tooth counting and the phase (a wrong revolution flag would give 360°). Note that the TDC flag must be discarded while the decoder is not yet synchronised (stale pulse). In Wokwi the interrupt latency (a few µs, i.e. $\approx 0.1°$ at 6000 rpm) adds to this. Between the teeth the angle is interpolated with the last tooth period; the 1 % second-order speed ripple changes the speed by only ≈ 0.2 % within one tooth, so the interpolation error stays below 0.01°; the largest error occurs over the gap (18° extrapolated).
b), c) Window and decision:

```
1     float th = crank_angle(t);
2     th_hist[n_samp & 31] = th;
3     float thd = th_hist[(n_samp - KF_DELAY) & 31];   /* angle of the sample the
4                                                         filter output belongs to */
5     n_samp++;
6     if (th < 0.0f || thd < 0.0f || n_samp < 32) {          /* not synchronised */
7         tdc_flag = false;
8         return;
9     }
10
11    /* ground truth: KNK HIGH in 0..90 deg after TDC of the current cylinder */
12    float a_now;
13    int c_now = segment(th, &a_now);
14    if (c_now != last_c_now) { truth[c_now] = false; last_c_now = c_now; }
15    if (a_now < 90.0f && digitalRead(PIN_KNK)) truth[c_now] = true;
16
17    /* crank sync check: decoded angle at the TDC pulse */
18    if (tdc_flag) {
19        float at, thc = crank_angle(t_tdc);
20        tdc_flag = false;
21        if (thc >= 0.0f) {
22            segment(thc, &at);
23            tdc_err_sum += fabsf(at < 90.0f ? at : at - 180.0f);
24            tdc_n++;
25        }
26    }
27
28    /* window on the delayed angle */
29    float a;
30    int c = segment(thd, &a);
31    if (a >= WIN0 && a < WIN1) {
32        if (win_cyl != c) { win_cyl = c; E = 0.0f; }
33        E += y * y;
34    } else if (win_cyl == c && a >= WIN1) {
```

```
        finish_combustion(c);
        win_cyl = -1;
    }
```

```
static void finish_combustion(int c)
{
    float R = (nB[c] > 0) ? E / B[c] : 1.0f;
    bool knock = (nB[c] >= WARMUP) && (R > K_THR);
    /* background: EWMA of E, clipped at 3 B so that knocking combustions
       hardly raise it; faster adaptation during the warm-up */
    if (nB[c] == 0) {
        B[c] = E;
    } else {
        float v = E < 3.0f * B[c] ? E : 3.0f * B[c];
        B[c] += (nB[c] < WARMUP ? LAMBDA_W : LAMBDA) * (v - B[c]);
    }
    nB[c]++;
    /* ignition retard: fast retard, slow recovery */
    if (knock) retard[c] = fminf(retard[c] + RET_STEP, RET_MAX);
    else retard[c] = fmaxf(retard[c] - RET_REC, 0.0f);
    float rmean = 0.25f * (retard[0] + retard[1] + retard[2] + retard[3]);
    ledcWrite(PIN_RETARD, (int)(255.0f * rmean / RET_MAX));
    int led = PIN_LED[cyl_no[c] - 1];
    if (knock) { digitalWrite(led, HIGH); led_off[c] = millis() + 100; }
```

d) Results (PC test harness running the sketch against the engine-sim chip code; counts after the warm-up; Wokwi results differ in detail):

rpm	knock amp.	noise	combustions	TP	FP	FN	detection	false alarms
3000	1.0 V	50 mV	600	115	1	0	100 %	0.2 %
6000	1.0 V	50 mV	1400	279	4	0	100 %	0.4 %
1500	0.2 V	50 mV	200	32	0	12	72.7 %	0.0 %
3000	0.2 V	50 mV	600	104	1	11	90.4 %	0.2 %
6000	0.2 V	50 mV	1400	267	6	12	95.7 %	0.5 %
1500	0.2 V	150 mV	200	4	1	40	9.1 %	0.6 %
3000	0.2 V	150 mV	600	36	10	79	31.3 %	2.1 %
6000	0.2 V	150 mV	1400	131	30	148	47.0 %	2.7 %

The real-time detector reproduces the offline results of 12.2 (same chain, same K): strong knock is always detected; light knock (0.2 V) at 50 mV noise in 73–96 % of the cases with < 1 % false alarms, and at 150 mV noise the detection collapses while the false-alarm rate rises above the design value at high rpm (fewer degrees of freedom, 12.1 d). The LED of a cylinder flashes for 100 ms at each detection, the retard LED brightness follows the mean retard (sawtooth: fast up, slow down).

e) Per sample: `analogRead`, 5 biquads (25 multiply-adds in float), angle interpolation (one division), a few comparisons – some hundred instructions, i.e. $\approx$ 1–2 µs at 240 MHz plus the ADC conversion, well below 40 µs (Wokwi timing is only indicative). UART: at 6000 rpm 200 combustions/s $\times \approx 22$ characters $= 4400$ characters/s, 38 % of the 11 520 characters/s at 115 200 baud; the 4-kB TX buffer decouples the printing from the sampling loop. Printing the raw samples (25 000/s $\times \geq 5$ characters) would exceed the UART capacity tenfold.

f) In a closed loop the retard lowers the cylinder pressure and temperature and thus the knock probability; the controller then settles into a limit cycle around the knock limit (retard steps after each knock, slow advance in between). Testing requires a knock model that depends on the spark advance, e.g. $p_{\text{knock}} = f(\text{spark_adv} - \text{retard})$, i.e. a SPARK input or a serial/I^{2}C command input of the chip. One would then record retard and knock rate per cylinder over time and check stability (no persistent heavy knock, small mean retard) for different noise levels and false-alarm rates.

Exercise 12.4 Port to the Arduino Uno

CHALLENGE

Port the detector to the Uno (`code/wokwi/sheet12_knock_uno/`, starting from the Q15 biquad of Sheet 10): Timer1 ISR at 25 kHz, single biquad `kf_bp2_q14`, window from the TDC pulse (period in samples of the previous combustion, cylinder 1 = TDC while CAM is HIGH), energy $\sum(y^2 \gg 8)$ in 32 bit, integer background $B \mathrel{+}= (\min(E, 3B) - B) \gg 4$, decision $E > 2B$ without division. Report detection and false-alarm rates as in 12.3 and the limits of the port.

Solution

See `code/solutions/wokwi/sheet12_knock_uno/sketch.ino`. The ISR adds only the window bookkeeping and the KNK check (`PIND`) to the ISR of Sheet 10 (estimated ≈ 500 of 640 cycles per sample); the decision, background, LEDs and printing run in `loop()` once per combustion. Results of the PC test (knock probability 20 %): 1.0 V/50 mV at 3000 rpm: 107 TP, 5 FP, 0 FN, 488 TN; 0.2 V/50 mV: 87 % detection / 0 % false alarms at 1500 (34/39), 92.5 % / 1.2 % at 3000 (99/107), 91.9 % / 0.1 % at 6000 rpm (260/283); 0.2 V/150 mV at 3000: 36 % / 3.4 %. That is about the performance of the ESP32 – consistent with 12.2, where the single biquad (BP2-win) was almost as good as the cascade. Limits: CPU load ≈ 75–80 % (no room for a second filter, FFT or MEM), no 60-2 decoder (the TDC pulse is a ground-truth helper of the simulator – a real implementation must decode the crank wheel, e.g. with the input capture unit of Timer1, which competes with the sampling interrupt), window positions from the previous combustion (lag at fast rpm changes), 10-bit ADC (resolution 4.9 mV/0.6 ≈ 8 mV at the sensor, sufficient for knock amplitudes ≫ noise), and no floating point for diagnostics.

Deliverables

12.1 on paper (block justification, numbers for b)–d)). `knockeval`: AUC and detection tables, pooled ROC plot (BP-win, WB-win, FFT, MEM), chosen threshold with justification. Wokwi: the confusion counts for the settings of 12.3 d) and a screenshot of the LEDs; a short discussion (max. one page) of the remaining weaknesses of your detector.